

UNIVERSITÀ DI PISA

Department of Computer Science

Bachelor's Degree in Computer Science

BACHELOR'S THESIS

Implementation of a Relational API for the WindFlow Stream Processing Library

Supervisor:
Prof. Gabriele Mencagli

Candidate:
Duccio Lorenzoni

ACADEMIC YEAR 2025/2026

For "those who know not, that they are not" and their journey towards awareness.

Contents

Introduction

1 Background **5**

1.1 Stream Processing 5

1.1.1 Timestamps and Out-of-Order Data 6

1.1.2 Working with Unbounded Streams 7

1.1.3 Event-Time Progress and Watermarks 8

1.2 Continuous Relational Models and CQL 9

1.2.1 Limits of Standard Relational Algebra on Streams 10

1.2.2 Streams and Time-Varying Relations 11

1.2.3 The Continuous Operator Triad 11

1.2.4 Practical Relevance and Execution Semantics 14

1.3 Dataflow Execution Models and Parallelism 14

1.3.1 Logical Plans versus Physical Dataflow Graphs 15

1.3.2 Parallel Execution: Subtasks and Data Routing 16

1.4 Apache Flink 18

1.4.1 The Layered API Ecosystem 18

1.4.2 Subtask Execution and Parallel Worker Model 19

1.4.3 Event-Time Synchronization via Stream Punctuations 20

1.5 Flink's Table API 21

1.5.1 Supported Operations and Plan Lowering 24

1.5.2 The Changelog Stream and RowKind Encoding 27

1.5.3 Streaming Sink Semantics 27

1.6 Introduction to WindFlow 28

1.6.1 PipeGraphs and Execution Topologies 28

1.6.2 Time Policies and Synchronization 30

1.6.3 Core Operators: Builders and Functors 32

1.6.4 Current Limitations 34

2 Overall API Architecture **35**

2.1 Python API 35

2.1.1 The Python API Subsystem 36

2.1.2 The Code Generation Subsystem 36

2.1.3 The Runtime Subsystem . . . 37
2.2 WindFlow Table Operations . . . 38

3 Python API Syntax and Semantics 39
3.1 Execution Context and Environment Setup . . . 39
3.1.1 Time Policies . . . 40
3.2 Stream Sources . . . 41
3.2.1 Schemas and Types . . . 42
3.2.2 Extracting a Timestamp in EventTime . . . 43
3.2.3 Out-of-Order Data and Durations . . . 44
3.2.4 Source Parallelization . . . 45
3.3 Customizable Expressions and Type Checking . . . 47
3.3.1 Expression Grammar and Operator Overloading . . . 47
3.3.2 Static Type Checking and Type Coercion . . . 49
3.4 Query Composition and the Draft Abstraction . . . 52
3.4.1 Projections, Filters and Deduplication . . . 53
3.4.2 Query Objects and Table References . . . 56
3.5 Windows and Intervals . . . 59
3.5.1 Window Specifications . . . 59
3.5.2 Relative Intervals . . . 60
3.6 GroupBy and Stateful Aggregations . . . 60
3.6.1 Keys, Windows, and Global Aggregations . . . 62
3.6.2 Aggregation Dependencies and Static Validation . . . 63
3.7 Multi-Stream Operators . . . 64
3.7.1 Relational Joins . . . 65
3.7.2 Set Operators . . . 67
3.8 Query Plan Serialization and Execution . . . 69
3.8.1 Query Dependencies and FromOp Resolution . . . 70

4 Code Generation and Operations 72
4.1 Parsing the Json into the Operations Tree . . . 73
4.1.1 Operation Tree and the Factory Pattern . . . 73
4.1.2 Linking Table References and Subquery Caching . . . 74
4.1.3 Post parsing Optimizations . . . 75
4.2 Operation Builders . . . 76
4.2.1 Logic Encapsulation in Lambdas . . . 78
4.2.2 Operation Peculiarities . . . 79
4.3 Struct Generation . . . 80
4.3.1 Struct Deduplication . . . 81
4.3.2 KeyBy, Hashing and Windowing Requirements . . . 82

4.4 Visiting the Operation Tree . . . 85
4.4.1 VisitContext and VisitResult . . . 85
4.4.2 Graph Construction and Pipe Routing . . . 86
4.5 Compilation and Execution . . . 88
4.5.1 The Runtime Subsystem . . . 88

5 Performance and Benchmarks 90
5.1 NEXMark Benchmark . . . 90
5.1.1 Data-Skew and Hot Entities . . . 91
5.1.2 Time Dilation and Dataset Generation . . . 91
5.2 Query Selection and Limitations . . . 91
5.2.1 Expressive Boundaries and Out-of-Scope Queries . . . 92
5.2.2 Supported Relational Subset . . . 93
5.2.3 Successive Aggregations and Boundary Slicing . . . 98
5.2.4 Topological Bottlenecks . . . 101
5.3 Experimental Setup . . . 103
5.4 Benchmark Results . . . 104
5.4.1 TIDE Results and Data Skew . . . 105
5.4.2 Native WindFlow Comparisons . . . 114

Conclusions 117

List of Figures

1.1 Example of a simple dataflow graph with two intermediate operators 15

1.2 Condensed logical streaming dataflow versus its expanded physical execution topology with parallel operator subtasks and stream partitions [10]. 17

1.3 Flink's API stack for stream processing. Adapted from [4]. 18

1.4 A Flink cluster's architecture. Adapted from [4]. 20

1.5 Watermark propagation via explicit punctuation events 21

1.6 MultiPipe built by the topology in Listing 1.10 29

1.7 Window Parallelism in WindFlow 33

2.1 Overall architecture showing how different subsystems interact 36

3.1 Topological evolution of the Draft during query composition. The final Query object encapsulates the root operator of the chain, while the initial `TableRefOp` symbolically links to the parent table. 57

3.2 The `Draft` of my_joined_query from 3.18 pointing to the previously defined Queries with a `TabRef`. 58

3.3 Physical plan view of an unkeyed-global-aggregation with forced parallelism to one. 63

3.4 Serialized view of the proposed query in Figure 3.2 70

4.1 Codegen Module Architecture 73

4.2 Logical Operation Tree derived from Listing 4.1. At the logic level, the `base_q` subgraph is represented exactly once. 75

4.3 An example of *Identity Select Erasure*, the bottom graph is the result of the optimization. 76

4.4 Table detailing the what operator maps to each table-operation and physical operator. 77

4.5 Automatically inserted maps in the physical plan to unify schemas before a join. 80

5.1 Performance profile for Query 1 on the 70M dataset. (A) Maximum processing throughput scaling linearly up to 5 threads. (B) Perfectly uniform data distribution at the sink, validating the source chunk-splitting mechanism. 105
5.2 Performance profile for Query 2. (A) Linear throughput scaling up to 1.8M tuples/s, proving that parallel filtering introduces negligible overhead. (B) Extreme data skew at the sink, where a single worker emits 100% of the surviving tuples. 106
5.3 Internal view of filtered tuples. 107
5.4 Performance profile for Query 3. (A) Throughput scaling across parallelism configurations. (B) Data skew at the sink, revealing significant load imbalance driven by hot entities during the join phase. 108
5.5 Internal view of join skew. 108
5.6 Performance profile for Query 5 on the 10M dataset. (A) Exceptionally low throughput caused by unkeyed topological bottlenecks. . . . 109
5.7 Performance profile for Query 8. (A) Throughput volatility across parallelism configurations. (B) Data skew at the sink, demonstrating a direct correlation between hash-based load balance and absolute pipeline throughput. 110
5.8 Performance profile for Query 9 on the 30M dataset. (A) Throughput peaking at 3 threads before experiencing degradation. (B) Data skew at the sink, exhibiting a deceptive output balance that masks internal computational bottlenecks. 111
5.9 Internal view of the skew by the join that is nullified by the keyed windows operator. 112
5.10 Performance profile for Query 12. (A) Perfect linear throughput scaling up to nearly 1.9M tuples/s. (B) Perfectly uniform data skew, demonstrating optimal hash distribution across the `bidder` keys. . . . 113
5.11 Performance profile for Query 20 under maximum dataset load (64.4M Bids × 4.2M Auctions). (A) Peak throughput achieved at `par = 3`, bounded by the sheer volume of memory state allocations. (B) Severe data skew in the C++ join buffers driven by clustered "Hot Auctions" matching the target category. 113
5.12 Performance comparison between TIDE auto-generated pipelines and native WindFlow C++ implementations. The efficiency metric highlights the near-zero overhead of the declarative abstraction. 114
5.13 Simplified topologies for Q2 in TIDE (top) and native WindFlow (bottom). The filter and the map are fused. 115

5.14 Simplified topologies for Q3 in TIDE (top) and native WindFlow (bottom). The select and unifier maps are fused and the join returns the already mapped struct. 115

List of Listings

1.1 CQL Query Equivalence for Stream Filtering . . . 14
1.2 Example of Imperative DataStream API Pipeline . . . 19
1.3 Flink TableAPI Environment . . . 22
1.4 Flink TableAPI Sources . . . 22
1.5 Flink TableAPI Sinks . . . 23
1.6 Flink TableAPI Query Construction . . . 23
1.7 Logical AST . . . 25
1.8 Optimized Physical Dataflow Plan . . . 26
1.9 WindFlow Topology . . . 28
1.10 WindFlow Pipe Insertions . . . 28
1.11 WindFlow Topology Merge . . . 30
1.12 WindFlow Topology Execution . . . 30
1.13 WindFlow assigning Timestamps and Watermarks . . . 31
1.14 Example of a Map Functor . . . 32
1.15 Example of Builder use . . . 32
3.1 TableEnvironment concept . . . 39
3.2 Registering a File Source . . . 41
3.3 Building a Schema . . . 42
3.4 Registrating the Time-Baseline in a TableEnvironment . . . 44
3.5 Configuring Source Delay . . . 45
3.6 Configuring Interleaved Splitting . . . 46
3.7 Expression Composition . . . 48
3.8 JSON AST Expression Serialization . . . 48
3.9 Type Inference Dispatch Registry . . . 51
3.10 Numeric Type Coercion . . . 51
3.11 Running Example: Where . . . 53
3.12 Running Example: Select . . . 53
3.13 Predefined Selection methods . . . 54
3.14 Running Example: Distinct . . . 54
3.15 Wrong Distinct Usage . . . 54
3.16 Running Example: Implicit Distinct . . . 55
3.17 Naming and Finalizing a Query . . . 56
3.18 Query Nesting . . . 58

3.19 Defining Windows 59
3.20 Defining Intervals 60
3.21 GroupBy Example 61
3.22 Having Emulation 61
3.23 GroupBy Examples by Key and Window 62
3.24 Static Grouping Validation 63
3.25 Pre-Join Branch Composition 65
3.26 Window and Interval Joins 66
3.27 Join with an Internal Predicate 66
3.28 Set Operations 68
3.29 Query Execution Dispatch 69
4.1 Replicated Subquery Example 74
4.2 SelectBuilder Table-Operation Example 77
4.3 Generated C++ Lambda for a Filter Operator 79
4.4 Schema Equivalence 81
4.5 Decoupled Struct 82
4.6 Automatic Key Extraction 82
4.7 Key Struct with Hashing 83
4.8 Output Struct with Window Constructors 84
4.9 Common Visit Signature 85
4.10 Visit Result 86
4.11 Pipeline Composition 87
5.1 CQL formulation for Query 1. 93
5.2 TIDE implementation for Query 1. 93
5.3 CQL formulation for Query 2. 94
5.4 TIDE implementation for Query 2. 94
5.5 CQL formulation for Query 3. 94
5.6 TIDE implementation for Query 3. 95
5.7 CQL formulation for Query 8. 95
5.8 TIDE implementation for Query 8. 96
5.9 Flink SQL formulation for Query 20. 96
5.10 TIDE implementation for Query 20. 97
5.11 Flink SQL formulation for Query 12. 98
5.12 TIDE implementation for Query 12. 98
5.13 Flink SQL formulation for Query 9. 99
5.14 TIDE implementation for Query 9. 99
5.15 CQL formulation for Query 4. 100
5.16 TIDE implementation for Query 4. 100
5.17 CQL formulation for Query 5. 101

5.18 TIDE implementation for Query 5. 102

Introduction

Context and Motivation

The evolution of real-time analytics has established Data Stream Processing [3] as a fundamental pillar of modern software architectures. As the complexity of continuous queries increases, the stream processing ecosystem has progressively shifted away from low-level, procedural programming models in favor of high-level declarative abstractions. Industry standards, such as Apache Flink, have demonstrated that decoupling the logical intent of a query from its physical execution not only drastically accelerates the development lifecycle, but also delegates complex topological optimizations, routing algorithms, and memory-safe state management to the underlying engine.

Achieving this level of developer productivity, however, typically implies relying on distributed frameworks running on managed runtimes. In contrast, ultra-low latency use cases, ranging from algorithmic trading to high-frequency sensor monitoring, often demand the raw execution speed and fine-grained hardware control provided by low level systems deployed on shared-memory multi-core architectures.

Problem Statement

Within the landscape of high-performance shared-memory architectures, the C++ WindFlow [12] library offers an exceptionally fast stream processing framework capable of fully exploiting multi-core parallelism based on FastFlow [2]. However, its native execution model presents significant ergonomic and architectural hurdles for application developers.

Operating WindFlow requires low-level imperative programming and manual composition of pipeline graphs. Developers are burdened with explicitly handling partitioning constructs (such as manual key hashing), managing out-of-band watermarks, and physically instantiating complex, tightly-coupled data structures. More importantly, WindFlow natively lacks pure relational abstractions and a dynamic schema-based type system. Every transformation step forces the developer to manually define C++ structs and handle casting operations.

These friction points drastically steepen the learning curve, increase the likelihood of application errors, and ultimately restrict the adoption of high-performance C++ engines in agile, modern data-science workflows.

Proposed Solution and Architectural Decisions

To bridge the gap between the flexibility of declarative programming and the raw throughput of native execution, this thesis presents the design, implementation, and empirical validation of a declarative relational API for the WindFlow library, named **TIDE** (*Table Interface for Declarative Execution*) [9].

Rather than building a monolithic compiler, TIDE adopts a rigorously decoupled, three-tier architecture designed to maintain a clean separation of concerns:

- **The Python Frontend (`api`):** Python was explicitly chosen as the frontend interface due to its undisputed dominance in the data engineering ecosystem. Relying on a lazy evaluation paradigm, it provides users with an SQL-like Domain Specific Language (DSL) to express continuous operations naturally.
- **The Intermediate Representation (`AST`):** The logical pipeline is serialized into a language-agnostic JSON Abstract Syntax Tree (AST). This decision ensures that the Python API remains completely oblivious to the underlying hardware execution, paving the way for future interoperability.
- **The Translation and Runtime Engine (`codegen` and `runtime`):** A Python-based compiler consumes the AST to dynamically synthesize native C++ code (via Jinja2 templating), seamlessly lowering the logical intent into a heavily optimized physical WindFlow execution graph, ready to be compiled and dispatched.

Methodology and Implementation

The implementation of TIDE addresses the complexities of physical stream processing by shifting the burden of validation from runtime to compile time. The frontend enforces strict, fail-fast semantic validation through static type checking and dynamic schema inference. This ensures that illegal operations, such as incompatible stream joins or alias collisions, are caught instantly before a single line of C++ is generated.

During the lowering phase, the translation engine bridges the semantic gap between Python's dynamic intent and C++'s strict execution model. The compiler recursively traverses the declarative symbolic expressions, such as boolean filters or mathematical projections, and dynamically translates them into isolated, strongly typed C++ lambda functions (`std::function` closures). Because C++ is statically typed, TIDE implements an automated struct generation algorithm, synthesizing minimalistic, shared C++ structs while transparently injecting the necessary hashing routines and windowing constructors entirely under the hood.

Experimental Validation and Results

To rigorously assess the physical capabilities of TIDE and quantify the overhead of its abstraction layer, the framework was evaluated using the industry-standard NEXMark benchmark. We implemented a robust subset of NEXMark [16] queries and introduced a *time-dilation* methodology during dataset generation to accurately stress temporal progression, window closures, and interval joins over 10 to 70 million tuple datasets.

The experimental phase yielded highly positive results, demonstrating that TIDE successfully abstracts complexity without sacrificing performance. Key findings include:

- **Negligible Runtime Overhead:** The auto-generated TIDE pipelines were directly compared against aggressively optimized, handwritten native C++ baselines. TIDE maintained over 91%, and peaking at over 98%, of the raw throughput of native implementations, seamlessly processing hundreds of thousands of tuples per second.

- **Scalability and Parallelism:** The framework successfully leveraged WindFlow's pipeline parallelism, scaling efficiently up to 40 logical cores on a shared-memory server environment without suffering from topological bottlenecks.

Thesis Outline

This thesis is organized to reflect the architecture and lifecycle of the framework, progressing from theoretical foundations to empirical validation:

- **Chapter 1 (Background):** Defines the principles of Data Stream Processing, formalizes continuous relational models (CQL), and analyzes industry standards like Apache Flink. It concludes by examining WindFlow's architecture and delineating the operational limitations that motivate this project.

- **Chapter 2 (Overall System Architecture):** Provides a high-level overview of the three-tier decoupled architecture (`api`, `codegen`, `runtime`), illustrating the component responsibilities and the end-to-end logical flow of the application.

- **Chapter 3 (Python API Syntax and Semantics):** Explores the logical frontend (the `api` module), detailing the core role of the `TableEnvironment`, the `Draft` relational semantics, the static type inference system, and the modeling methodologies for continuous streams and stateful multi-stream operations.

- **Chapter 4 (Code Generation and Operations):** Details the C++ Code Generation engine and runtime backend. It documents the JSON AST parsing, dependency resolution, dynamic synthesis of deduplicated structures, and the encapsulation of algebraic logic using Jinja2 templates.
- **Chapter 5 (Performance and Benchmarks):** Presents the empirical evaluation of the framework using the NEXMark industry standard. It analyzes parallel scalability and data skew, verifying that the TIDE abstraction successfully lowers queries without introducing runtime overhead compared to natively authored WindFlow pipelines.

Chapter 1

Background

In recent years, the widespread adoption of the Internet of Things (IoT), cyber-physical systems, and high-frequency real-time monitoring has led to an exponential increase in the volume and velocity of generated data. Traditional batch processing systems, designed to operate periodically on finite, static datasets, are inherently ill-suited for scenarios requiring instantaneous response times. Consequently, Data Stream Processing [3] has emerged as a crucial computing paradigm capable of ingesting, analyzing, and transforming potentially infinite, unbounded sequences of data in continuous time.

This chapter provides the foundation required to contextualize the design of the declarative table-based for WindFlow [12]. First, it explores the core principles of stream processing, including continuous relational models and strategies for handling unbounded data via time semantics and watermarks. Next, it examines Apache Flink's [8] multi-layered API ecosystem, briefly introducing its procedural DataStream API and native SQL interface while focusing primarily on its declarative Table API, highlighting the architectural patterns that served as key inspiration for this work. Finally, it introduces the WindFlow library, analyzing its execution model, and details the specific limitations that motivated many decisions made during the development of WinfFlow TIDE (Table Interface for Declarative Execution) [9] presented in this thesis.

1.1 Stream Processing

Data Stream Processing [3] is a computing approach designed to ingest, process, and analyze high-velocity and high-volume data continuously as soon as they become available. Unlike traditional batch architectureswhich assume that datasets are bounded, statically stored on disk and queried afterwards, stream processing handles data generated in continuous motion. Because the data source produces events incrementally over time, there is no prior knowledge of the total volume of data that will be generated or when the stream will end. Consequently, data stream processing systems must be designed to operate indefinitely over potentially unbounded

sequences of data under stringent latency constraints.

1.1.1 Timestamps and Out-of-Order Data

Formally, a *data stream* S is modeled as a potentially infinite, linearly ordered sequence of data items:

$$S = \{e_1, e_2, e_3, \dots\} \tag{1.1}$$

where each element e_i represents a discrete event, typically structured as a pair:

$$e_i = \langle \mathbf{d}_i, t_i \rangle \tag{1.2}$$

Here, $\mathbf{d}_i$ denotes the *data payload* (or *tuple*), which conforms to a predefined relational schema composed of typed attributes. The second member of the pair $t_i \in \mathcal{T}$ is known as *timestamp*; it represents the instant at which $\mathbf{d}_i$ has been generated in the time domain $\mathcal{T}$.

It is important to state that although the elements of a stream are strictly sequenced by their arrival, their timestamps do not necessarily follow the same order:

$$i < j \not\Rightarrow t_i \leq t_j \tag{1.3}$$

Such a stream is referred to as *out-of-order* as the ingestion order is different from the chronological order in which events were originally generated.

This is usually the result of variable network latency, localized buffering, or transmission retries across distributed sources (IoT devices or monitoring sensors) recording data and the stream processing engine.

When there can be out-of-order data, stream processing systems primarily distinguish between *Event Time*, *Ingestion Time* (often referred to as *Ingress Time*) and *Processing Time* [1]:

- **Event Time:** Denotes the physical instant at which an event originally occurred at the data source (for instance, when an IoT sensor recorded a measurement or a transaction was generated). This timestamp is embedded directly within the event payload and must be explicitly extracted during ingestion.

- **Ingestion Time (Ingress Time):** Corresponds to the instant an event physically enters the stream processing pipeline. It acts as a special case of Event Time where the system monotonically assigns timestamps upon reading the records at the source. While it frees the user from explicitly handling timestamp extraction and watermark generation logic (see 1.1.3), the system internally still relies on automatically generated watermarks. This is necessary

because, even if tuples are perfectly ordered at the source, network routing and parallel shuffles can still introduce out-of-order arrivals at downstream operators.

- **Processing Time:** Refers to the local wall-clock time of the specific machine or thread executing the operator. Since computation relies purely on the local clock, events are processed strictly in the order they arrive at the operator. Consequently, Processing Time entirely eliminates the concept of out-of-order data and removes the need for watermarks. While this approach yields the lowest possible latency, it provides non-deterministic results that heavily depend on network speeds and execution scheduling.

1.1.2 Working with Unbounded Streams

Since there is no limit to the volume of data that can be processed, stream processing systems encounter problems regarding memory management. Stateful operators, such as aggregations, duplicate detectors, and multi-stream joins, must preserve historical data across events. However, preserving an unbounded history in memory is physically unfeasible, as it would inevitably exhaust the available storage resources.

In streaming queries, calculating statistics (such as moving averages, sums, or extreme values) cannot yield a definitive result at any given time since new tuples might alter it.

Modern stream processing systems address this challenge through two primary paradigms:

1. **Continuous Aggregations (The Changelog Model):** In this approach, stateful operators maintain running accumulators across the unbounded timeline. Every incoming tuple immediately mutates the operator's internal state, emitting an updated result downstream as a continuous change log (typically structured as insert, update, or retraction records). While this approach provides near-zero latency, it introduces two significant limitations: downstream operators must continuously adapt to mutable data streams, and state storage grows monotonically with the cardinality of the grouping keys unless explicit eviction policies (such as Time-To-Live expiration) are enforced.

2. **Stream Discretization (The Windowing Model, see 1.2.3):** Instead of maintaining an unbounded global state, the infinite stream is partitioned into finite, manageable subsets known as *windows*. The system buffers arriving records within their designated window boundaries and performs the aggregation only when the window interval concludes. Once evaluated, the window

emits a definitive, immutable output record downstream and purges its internal state from memory.

If the aggregation is commutative and associative an incremental approach can be used; the current aggregation state is kept and updated overtime but only emitted when the window interval concludes.

Windows can be categorized based on their dimension into *time-based windows* (discretizing time into discrete temporal spans) and *count-based windows* (segmenting records into fixed batch sizes). Furthermore, based on how adjacent intervals interact, they are typically divided into:

- *Tumbling Windows:* Fixed-length, non-overlapping intervals where each event belongs to exactly one window instance (size = slide).
- *Sliding Windows:* Overlapping intervals where windows advance by a discrete slide duration, allowing records to participate simultaneously in multiple active windows.

While windowing effectively bounds state retention, it introduces a critical operational dilemma in real-world deployments: *the window closing problem.* In the presence of variable network jitter and out-of-order event arrivals, a streaming operator cannot rely on its local clock time to infer whether all events belonging to a temporal window $[t_{\text{start}}, t_{\text{end}})$ have arrived. Closing a window prematurely causes late-arriving tuples to be permanently dropped or misplaced, whereas delaying execution indefinitely degrades query latency and exhausts memory buffers.

To overcome this problem and properly signal progress in event time, stream processing systems use **watermarks**.

1.1.3 Event-Time Progress and Watermarks

The watermark $w \in \mathcal{T}$ is the latest timestamp that guarantees that no more incoming tuples have a timestamp lower than it.

$$\forall\, e = \langle \mathbf{d}, t \rangle \quad \text{arriving after } w, \quad t > w \tag{1.4}$$

When a window-based operator receives a watermark w, it advances its internal event-time clock and initiates the following lifecycle transitions:

1. **Window Firing:** The operator triggers computation for all active windows whose upper boundary has been reached, namely all windows satisfying:

$$t_{\text{end}} \leq w \tag{1.5}$$

2. **Emission and Eviction:** The materialized aggregated state is emitted downstream, and the corresponding buffers are purged from memory, reclaiming storage resources.

3. **Late Data Handling:** Any subsequent tuple $e = \langle \mathbf{d}, t \rangle$ received with $t \leq w$ is formally classified as *late data*. Depending on the system's runtime configuration, late tuples are either unconditionally discarded, accumulated into side-outputs, or accommodated via an allowed lateness threshold.

The policy adopted by a system to generate its watermarks influences *latency*, *state retention*, and *data completeness*.

- **Aggressive Watermarks (Low Delay):** Producing watermarks close to the highest observed timestamp ($\max(t)$) minimizes end-to-end latency and state memory, but significantly increases the risk of discarding legitimate out-of-order tuples.

- **Conservative Watermarks (High Delay):** Introducing an intentional slack delay δ (such that $w = \max(t) - \delta$) ensures robust tolerance against network jitter and out-of-order arrivals, but inflates memory buffers and defers downstream result visibility.

Consequently, watermark generation represents the central control mechanism for orchestrating correct, high-throughput analytics over unbounded streams.

1.2 Continuous Relational Models and CQL

Although data stream processing operates on continuous flows rather than static, batched datasets, adopting a relational abstraction for high-level declarative query formulation remains highly desirable. Its primary advantage lies in the declarative paradigm: users express *what* data they wish to retrieve or compute, allowing the underlying database engine to determine *how* to execute the query efficiently via query planners and optimizers. However, classical relational algebra is inherently ill-suited for continuous streaming: foundational operators such as global sorting and non-windowed aggregations are strictly blocking, requiring the entire dataset to be available before producing output; an impossible premise on potentially unbounded streams.

To adapt relational concepts to continuous streaming, Arasu et al. [6] proposed the *Continuous Query Language* (CQL). CQL formalizes how infinite streams can be converted into relations and queried continuously. Understanding this model is essential, as it inspires modern interfaces such as Apache Flink's Dynamic Tables and

highlights the main challenges of compiling declarative queries down to WindFlow (see Section 1.6.4).

1.2.1 Limits of Standard Relational Algebra on Streams

In practice, standard relational algebra cannot be applied to streaming data without significant adaptations. Traditional Database Management Systems (DBMSs) are fundamentally designed around three implicit assumptions that do not hold in a streaming context:

1. **Data is Finite and Static:** Traditional SQL queries execute against a bounded snapshot of data stored on disk. The computation has a clear beginning and end: once the stored tables are scanned, the query terminates and outputs the final result. In contrast, a data stream is conceptually infinite, open-ended, and append-only.

2. **The Presence of Blocking Operators:** In relational algebra, several fundamental operations are inherently *blocking*. A blocking operator must inspect the entire dataset before it can safely emit its first output record. For example:

   - An unconstrained `ORDER BY` must consume all incoming records to determine which tuple has the smallest sort key.
   - Standard aggregations (such as global `COUNT`, `SUM`, or `AVG`) assume that no further rows will arrive, outputting a single definitive value at query completion.
   - Unconstrained joins (e.g., Cartesian products or non-windowed theta-joins) must retain all past records indefinitely to pair them with potential future arrivals.

   When applied to an unbounded stream, a blocking operator produces a deadlock condition: it waits indefinitely for the stream to conclude, starving subsequent operators in the processing pipeline and never emitting any result.

3. **Unbounded State and Memory Consumption:** Even for operators that could theoretically produce intermediate updates (such as a running count or deduplication), keeping track of an unbounded history without explicit boundaries leads to monotonic state growth. Over time, the internal state inevitably exhausts available memory resources.

To resolve these incompatibilities, stream processing requires a fundamental shift: instead of running a one-time query over static data, the system must support *con-*

tinuous queries [6]. A continuous query remains active indefinitely, processing events incrementally as they arrive and continuously updating its output.

Achieving this requires reconciling two conflicting abstractions: the transient, append-only nature of a *stream* and the stateful, addressable nature of a *relation*. This conceptual bridge is precisely what Arasu et al. [6] formalized through the Continuous Query Language (CQL).

1.2.2 Streams and Time-Varying Relations

To formalize continuous relational computation, Arasu et al. [6] established that streaming queries must operate over two distinct primitive abstractions: the continuous data streams defined in Section 1.1 and an extended relational abstraction termed the *time-varying relation*.

While a stream S represents an append-only sequence of immutable events $\langle \mathbf{d}, t \rangle$, a relation in CQL is modeled dynamically as a function mapping each discrete time instant $\tau \in \mathcal{T}$ to a finite but unbounded, multiset (bag) of tuples:

$$R : \mathcal{T} \longrightarrow \mathcal{B}(\text{Tuples}) \tag{1.6}$$

At any specific instant τ, $R(\tau)$ denotes an *instantaneous relation* representing the snapshot state of the table at that point in time. Unlike classical databases, where relations are static, a time-varying relation evolves as time advances: tuples can be inserted, updated, or removed across successive instants.

A **continuous query** over these structures runs indefinitely: as time advances within domain $\mathcal{T}$, the engine processes input elements up to τ and updates the instantaneous relations or produces new output stream records until explicitly stopped.

1.2.3 The Continuous Operator Triad

Rather than introducing complex stream-to-stream (S2S) operators directly, the CQL abstract semantics decomposes any continuous query into a pipeline composed of three distinct operator classes:

1. Stream-to-Relation (S2R) Operators

Stream-to-Relation operators convert an incoming append-only stream into a time-varying relation by bounding history over sliding windows. The most prominent window constructs include:

- **Time-Based Windows (S[**Range** T]):** Captures all stream elements that fall within the preceding interval of duration T relative to the current evalua-

tion time τ:

$$R(\tau) = \{\mathbf{d} \mid \langle \mathbf{d}, t \rangle \in S \wedge (t \leq \tau) \wedge (t \geq \max\{\tau - T, 0\})\} \tag{1.7}$$

CQL defines two notable boundary cases:

  - **Instantaneous Window (S[**Now**]):** Where $T = 0$, capturing solely the records sharing the current timestamp τ.
  - **Unbounded Window (S[**Range Unbounded**]):** Where $T = \infty$, retaining the complete cumulative history of the stream up to τ.

- **Tuple-Based Windows (S[**Rows** N]):** Captures the N most recent tuples of S relative to τ.

- **Partitioned Windows (S[**Partition By** $A_1, \ldots, A_k$ **Rows** N]):** Partitions the input stream by attributes $A_1, \ldots, A_k$ and independently maintains the last N tuples per partition key.

2. Relation-to-Relation (R2R) Operators

Relation-to-Relation operators ($\mathcal{O}_{\text{rel}}$) correspond directly to standard relational algebra (selection σ, projection π, equi-join $\bowtie$, and grouped aggregation γ). In CQL, these operators are evaluated snapshot-by-snapshot: at each time instant τ, the operator computes its output relation $R_{\text{out}}(\tau)$ by applying standard relational semantics over the instantaneous inputs $R_{\text{in}}(\tau)$:

$$R_{\text{out}}(\tau) = \mathcal{O}_{\text{rel}}(R_1(\tau), R_2(\tau), \ldots, R_n(\tau)) \tag{1.8}$$

3. Relation-to-Stream (R2S) Operators

Because downstream consumers or sinks often require append-only streams rather than mutable tables, CQL provides three explicit relation-to-stream operators to materialize state transitions into event streams:

- **`Istream` (Insert Stream):** Emits a stream element $\langle \mathbf{d}, \tau \rangle$ whenever tuple $\mathbf{d}$ is newly added to the relation at time τ:

$$\text{Istream}(R) = \bigcup_{\tau \geq 0} \Big((R(\tau) - R(\tau - 1)) \times \{\tau\} \Big) \tag{1.9}$$

  where $R(-1) = \emptyset$ by convention.

It is worth noting that the operators $\cup$ and $-$ denote multiset (bag) operations [6]. Consequently, the difference operator accounts for tuple multiplicity: if an instantaneous relation $R(\tau)$ contains m_1 occurrences of tuple $\mathbf{d}$ and $R(\tau - 1)$ contained m_2 occurrences, the resulting difference contains exactly $\max(0, m_1 - m_2)$ instances of $\mathbf{d}$. Thus, if a tuple already present in $R(\tau - 1)$ appears with an increased multiplicity in $R(\tau)$, only the number of new insertions is emitted into the stream, as past occurrences were already captured by the union over prior instants.

- **`Dstream` (Delete Stream):** Emits a stream element whenever tuple $\mathbf{d}$ is evicted or removed from the relation at time τ:

$$\text{Dstream}(R) = \bigcup_{\tau \geq 0} \Big((R(\tau - 1) - R(\tau)) \times \{\tau\} \Big) \tag{1.10}$$

  Symmetrically, under multiset semantics, an element is emitted for each net decrement in tuple multiplicity between instants $\tau - 1$ and τ.

- **`Rstream` (Relation Stream):** Emits the entire instantaneous snapshot $R(\tau)$ as stream records at every time instant τ:

$$\text{Rstream}(R) = \bigcup_{\tau \geq 0} (R(\tau) \times \{\tau\}) \tag{1.11}$$

Query Composition and Continuous Semantics

Any end-to-end stream transformation (S2S) is formally realized through the composition of the three operator classes:

$$\text{Stream} \xrightarrow{\text{S2R}} \text{Relation} \xrightarrow{\text{R2R}} \text{Relation} \xrightarrow{\text{R2S}} \text{Stream} \tag{1.12}$$

This design choice allows the system to reuse standard relational algebra rewrites, predicate pushdowns, and established optimization techniques directly within the R2R intermediate stage.

Finally, the formal result of a continuous query Q at any evaluation instant $\tau \in \mathcal{T}$ depends strictly on its outermost operator. If the query terminates with a Relation-to-Stream operator, it yields an append-only stream emitting newly produced events as time advances; conversely, if the top-level operator is an S2R or R2R operator, the output is an instantaneous relation snapshot $R(\tau)$ evaluated over all inputs observed up to τ.

1.2.4 Practical Relevance and Execution Semantics

While the continuous operator triad serves as a rigorous theoretical foundation, real-world continuous query languages introduce syntactic defaults to simplify routine pipelines. For instance, referencing a stream directly in a query without a window clause implicitly applies an unbounded window, and monotonic subqueries automatically apply an `Istream` operator downstream. Under these defaults, a simple stateless filter on a stream:

```
SELECT * FROM SensorStream WHERE temperature > 30;
--is equal to
SELECT Istream(*) FROM SensorStream [Range Unbounded] WHERE
   temperature > 30;
```

Listing 1.1: Equivalence between standard SQL query defaults and the explicit S2R/R2S operator expansion in CQL.

logically traverses the full triad: the stream is converted into an unbounded relation, filtered using relational selection, and streamed back via `Istream`.

Beyond theoretical formulation, the CQL model became the canonical language used to specify benchmark queries in data stream processing. In particular, the NEXMark benchmark [16], widely adopted to evaluate relational stream systems and utilized in Chapter 5 of this thesis, defines its queries directly in CQL syntax.

By formalizing how stateful windows and incremental change logs reconcile relational algebra with open-ended event streams, Arasu et al. provided the foundational execution model later adopted by modern streaming SQL interfaces.

1.3 Dataflow Execution Models and Parallelism

While continuous query languages abstract processing into compositions of Stream-to-Relation, Relation-to-Relation, and Relation-to-Stream operators, physical execution engines must translate these logical constructs into concrete execution topologies. Understanding how streaming computations are physically structured and scheduled is essential to motivating the architectural decisions and compilation workflows presented in the subsequent chapters.

Modern stream processing engines execute queries by modeling computation as a directed *dataflow graph* [1]. In this paradigm, vertices represent execution units (operators) that encapsulate specific computation logic, while directed edges represent continuous communication channels conveying data items, structured as timestamped records $\langle \mathbf{d}_i, t_i \rangle$, from producer operators to downstream consumers.

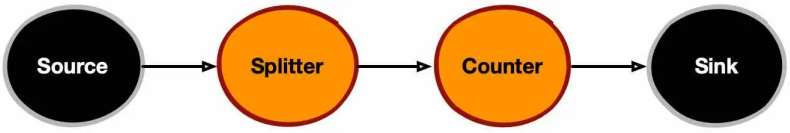


Figure 1.1: Example of a simple dataflow graph with two intermediate operators

As illustrated in Figure 1.1, data traverses three main phases along this pipeline: the *source* operator ingests raw external feeds and inject timestamped events into the system; *intermediate* operators reactively apply stateless transformations (such as filters or projections) or update localized state (such as windows and running aggregations); and finally, terminal *sink* operator consumes the processed stream to materialize results to persistent storage or downstream applications.

Computation within this topology is inherently *data-driven* (or push-based); execution is triggered by the arrival of data elements in operator input queues, where records are processed as soon as they become available.

1.3.1 Logical Plans versus Physical Dataflow Graphs

Similarly to how expressions in classical relational algebra do not map 1-to-1 to physical access paths in traditional database management systems (DBMSs), stream processing engines maintain a strict separation between high-level *logical plans* and executable *physical dataflow graphs*.

A **logical plan** represents the user query as a directed acyclic graph (DAG) or tree of abstract relational operatorssuch as projections (π), selections (σ), windowed aggregations (γ), and joins ($\bowtie$). It defines *what* transformations must be applied to the stream schemas and verifies semantic validity, without committing to specific execution mechanics, memory structures, or hardware scheduling.

Conversely, a **physical plan** defines *how* that computation is concretely carried out over runtime resources. In streaming systems, nodes in the physical plan are concrete operator instances encapsulating explicit state backends, event queues, and execution threads, while edges correspond to memory channels or network buffers.

The divergence between logical intent and physical execution is readily apparent in classical operations:

- **The Batch Analogy:** In a relational DBMS, a declarative statement such as `SELECT DISTINCT` does not correspond to a native CPU instruction. To execute it, the query planner must lower the logical operation into a concrete physical algorithm, typically by inserting an external sorting operator (*Sort-based Distinct*) followed by sequential deduplication.

- **The Streaming Reality:** In stream processing, physical lowering is even more critical because unbounded streams cannot be sorted. A logical `DISTINCT` must therefore be translated physically into an active stateful filter. The physical node must manage an internal in-memory state (e.g., `std::unordered_set`) which conceptually requires state-retention bounds, windowed constraints, or execution over bounded streams to prevent unbounded memory growth.

Beyond duplicate elimination, this structural mismatch governs many streaming transformations.

Bridging this gap requires translating abstract relational expressions into concrete runtime constructs: the compiler must resolve and align data schemas, prune redundant intermediate operations, and synthesize native processing functions, forming the translation pipeline implemented in this thesis.

1.3.2 Parallel Execution: Subtasks and Data Routing

Since streams can generate data at high speeds, maintaining low end-to-end latency on the graph and high throughput is critical to prevent over-saturation of internal buffers. To raise efficiency stream processing systems use parallelization and distributed computing (briefly in Section 1.4).

There are two main parallelism strategies:

- **Inter-Operator (Task/Pipeline) Parallelism:** Individual operators along the physical pipeline execute concurrently on distinct threads or worker processes. Directed channels (such as bounded queues in shared memory or network sockets) act as asynchronous buffering stages, allowing upstream producers and downstream consumers to operate in a pipelined fashion without blocking one another.

- **Intra-Operator (Data) Parallelism:** A single logical operator is horizontally partitioned across multiple concurrent instances, termed *operator subtasks* (or *worker replicas*). Each subtask is assigned a dedicated thread of execution and processes a disjoint partition of the incoming stream.

The degree of intra-operator parallelism is quantified by an integer parameter $P \geq 1$, which can be configured independently for each operator stage. Figure 1.2 contrasts the condensed logical view of a streaming pipeline with its expanded physical execution topology.

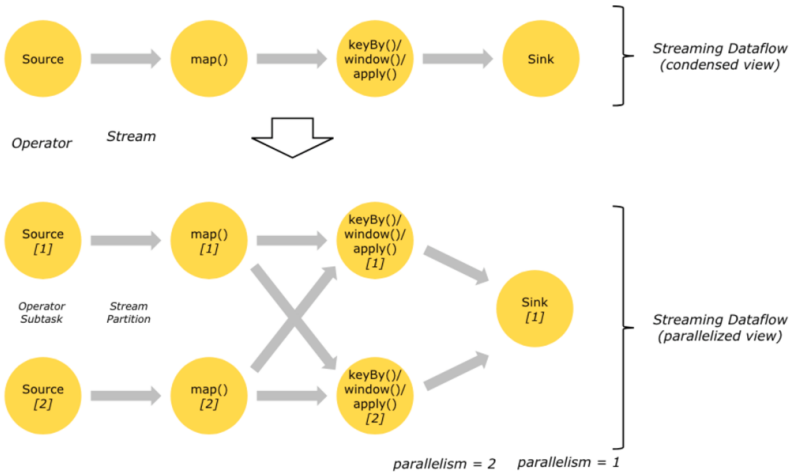


Figure 1.2: Condensed logical streaming dataflow versus its expanded physical execution topology with parallel operator subtasks and stream partitions [10].

As shown in Figure 1.2, scaling an operator across multiple subtasks introduces the challenge of data routing: determining how records emitted by P_{up} upstream subtasks are dispatched across P_{down} downstream subtasks. Physical stream engines rely on three primary channel routing patterns:

1. **Forward (1-to-1):** Each upstream subtask transmits records exclusively to a corresponding downstream subtask ($P_{\text{up}} = P_{\text{down}}$). Because no data redistribution is required, this pattern avoids scheduling overhead and cross-thread contention. It is predominantly used between consecutive stateless transformations (such as the `Source` $\rightarrow$ `map()` transition in Figure 1.2) and forms the basis for *operator chaining*, where chained operators are merged into a single thread to eliminate inter-thread queue traversals entirely.

2. **Key-Based Partitioning (Hash Shuffle / `keyBy`):** Records are routed deterministically based on an extraction function applied to one or more attributes of the tuple payload. The downstream subtask index is resolved via a hashing function, for example:

$$\text{subtask_id} = h(\text{key}) \pmod{P_{\text{down}}} \tag{1.13}$$

   This all-to-all redistribution (illustrated between the `map()` and `keyBy()/...` stages in Figure 1.2) is indispensable for stateful stream processing. By guaranteeing that all records sharing the same key are dispatched to the exact same subtask, each worker maintains its localized window state, accumulators, or hash sets independently, ensuring correctness without requiring expensive synchronization locks or atomic memory barriers across threads.

3. **Round-Robin:** When records must be distributed across downstream subtasks without a partitioning key, tuples are dispatched in a cyclic round-robin fashion. This all-to-all redistribution balances the processing load evenly across worker threads for stateless computations. In the boundary case where a downstream stage has a parallelism of onesuch as the `Sink` operator in Figure 1.2all upstream subtasks simply route their outputs into that single receiving worker.

The combination of intra-operator parallelism and key-based routing enforces *state isolation*: subtasks do not share mutable state in memory. Each replica encapsulates a private, disjoint slice of the global state, allowing the execution engine to achieve throughput scalability as core counts increasewhether scaled out across a cluster or scaled up across multiple cores on a single machine.

Although, these are the core concepts how parallelization is actually achieved may vary from system to system.

1.4 Apache Flink

Apache Flink [8] is an open-source, distributed stream processing framework that has established itself as an industry standard for stateful computations over unbounded and bounded data streams. Built from the ground up as a pure streaming engine, Flink treats batch processing simply as a special case of streaming over finite datasets. Analyzing its architectural stack and declarative relational interfaces provides the conceptual baseline for the design of WindFlow TIDE.

1.4.1 The Layered API Ecosystem

Flink structures its programming model as a hierarchical pyramid of abstractions, balancing developer control against programming productivity.

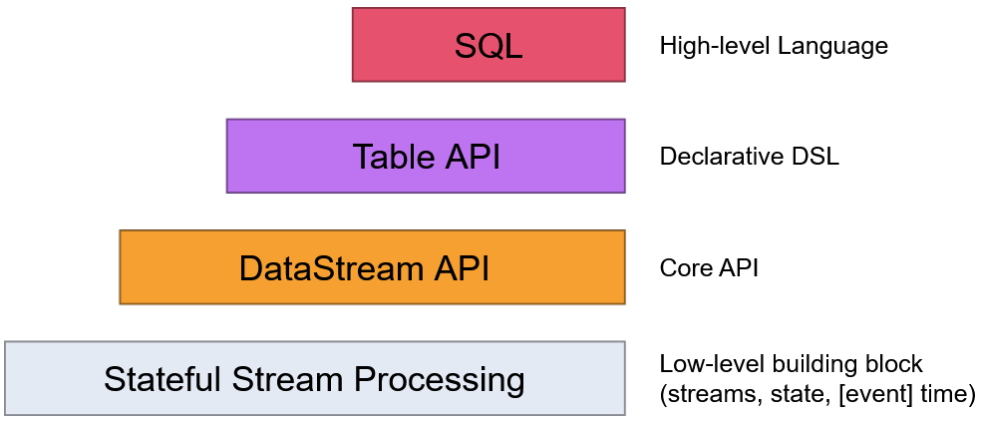


Figure 1.3: Flink's API stack for stream processing. Adapted from [4].

1. **Core Stateful Stream Processing (ProcessFunction):** The lowest-level abstraction. It exposes low-level access to event streams, allowing users to explicitly manipulate keyed state backends, register event-time and processing-time timers, and handle arbitrary callback logic. While offering maximum expressiveness, it requires significant boilerplate code.

2. **The DataStream API:** An intermediate imperative and functional interface. Computation is defined as a series of typed transformations (`map`, `filter`, `keyBy`, `window`) over dataflow streams. The developer explicitly prescribes the execution topology and procedural pipeline:

```
# Imperative stream processing pipeline
env = StreamExecutionEnvironment.get_execution_environment()
events = env.add_source(sensor_source)

windowed_counts = (events
    .filter(lambda x: x.temperature > 30.0)
    .key_by(lambda x: x.sensor_id)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .reduce(lambda a, b: SensorReading(a.id, a.temp + b.temp)))
```

Listing 1.2: Imperative stream transformation in PyFlink using the DataStream API.

3. **The Table API and SQL:** The highest-level declarative tier. Rather than manually wiring pipes and window buffers, developers express *what* data transformation is desired using relational expressions or standard ANSI-SQL queries. Both interfaces share the same unified AST representation and execution engine.

Crucially, these abstraction layers do not represent isolated execution engines. The declarative tiers are systematically compiled and lowered into the underlying physical dataflow runtime. An expression formulated through the Table API or SQL is initially parsed into a logical relational tree, optimized and ultimately translated into concrete streaming operators executed as parallel subtasks.

1.4.2 Subtask Execution and Parallel Worker Model

At the cluster level, Flink coordinates execution through a master-worker architecture:

- **JobManager (Coordinator):** The central master process responsible for accepting submitted applications, converting logical query graphs into physical execution plans, scheduling tasks, and orchestrating distributed checkpoints.

- **TaskManagers (Workers):** Distributed worker processes that execute the actual operator tasks. Each TaskManager allocates a fixed number of execution slots (*Task Slots*), where each slot encapsulates an isolated set of memory resources and executes assigned operator threads.

When an application is executed, each logical operator is mapped into P concurrent physical replicas, termed *operator subtasks*. Via operator chaining, multiple logical operators can be fused into a single subtask. Each chained operator is executed on the same thread (task slot) as the others to reduce thread handover overhead and network traffic.

In accordance with the state-isolated dataflow paradigm, subtasks do not share mutable memory: each worker instance processes its assigned stream partition, maintains its local state independently (stored either in JVM heap memory or embedded RocksDB instances), and exchanges tuples with other TaskManagers via buffered TCP connections managed by Netty.

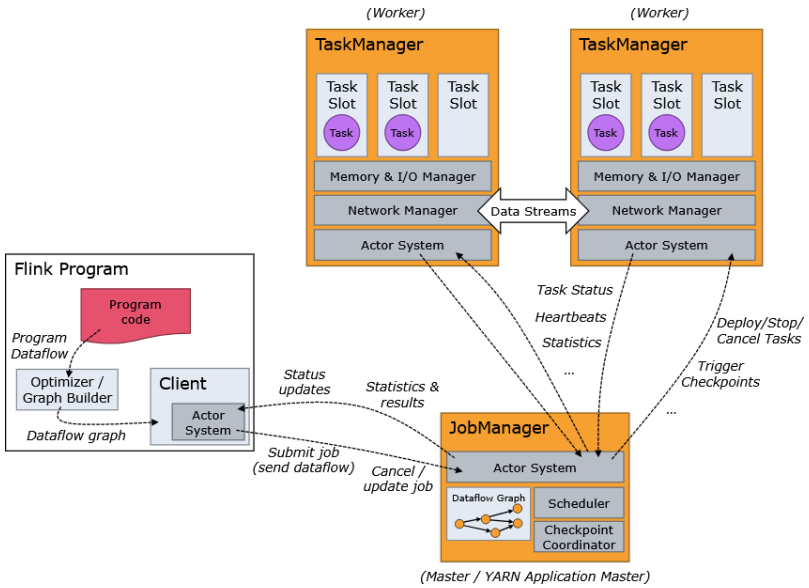


Figure 1.4: A Flink cluster's architecture. Adapted from [4].

1.4.3 Event-Time Synchronization via Stream Punctuations

To handle event time streams Flink models watermarks through the formal concept of *stream punctuations*, first established by Tucker et al. [15].

Unlike out-of-band control signals or global wall-clock synchronization, a watermark is an explicit punctuation record $\mathrm{WM}(t)$ injected directly into the dataflow channels alongside standard data tuples. A punctuation carrying timestamp t acts as a formal assertion that no subsequent records with an event timestamp $t' \leq t$ will arrive on that channel.

When a physical operator consumes data from multiple upstream subtask channels ($i \in [1, P_{\text{in}}]$), each channel advances its event-time progress independently. The receiving subtask tracks the watermark of each incoming channel and computes its own operational event-time clock as the **minimum** over all active channel watermarks:

$$W_{\text{current}} = \min_{i \in [1, P_{\text{in}}]} W_i \tag{1.14}$$

Only when W_{current} advances past a defined threshold does the operator trigger time-based operations (such as closing temporal windows or evaluating state eviction timers), subsequently broadcasting the updated WM(W_{current}) downstream.

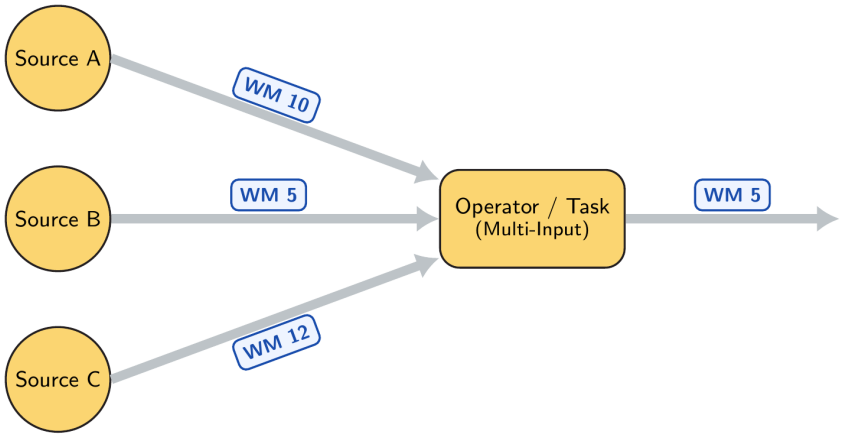


Figure 1.5: Watermark propagation via explicit punctuation events

1.5 Flink's Table API

The declarative foundation of Flink's relational layer is the *Dynamic Table* abstraction, which provides a concrete implementation of the Continuous Relational Model formalised by CQL. A dynamic table is a relation that changes continuously over time as new events arrive.

Continuous query execution across dynamic tables follows a cyclical pipeline:

$$\text{DataStream} \xrightarrow{\text{Ingestion}} \text{Dynamic Table} \xrightarrow{\text{Continuous Query}} \text{Dynamic Table} \xrightarrow{\text{Emission}} \text{Changelog} \tag{1.15}$$

To understand the paradigm shift, it is highly effective to break down the lifecycle of a PyFlink streaming application into its three fundamental phases: environment configuration, schema definition, and fluent query composition.

The entry point of any Table API application is the `TableEnvironment`, which serves as the central execution context and catalog manager.

```
from pyflink.table import EnvironmentSettings, TableEnvironment

settings = EnvironmentSettings.in_streaming_mode()
t_env = TableEnvironment.create(settings)

# Decoupled physical configurations
t_env.get_config().set(
    "table.exec.resource.default-parallelism", "2"
)
t_env.get_config().set("table.exec.state.ttl", "1 h")
```

Listing 1.3: Running Example: Environment setup

Physical configurations, such as intra-operator parallelism, are defined globally on the environment rather than wired into individual operators. Furthermore, because continuous non-windowed aggregations require tracking state for every unique key, memory usage would theoretically grow without bound. Configuring a state Time-To-Live (`table.exec.state.ttl`) ensures that the underlying runtime automatically evicts internal accumulators for idle keys.

Handling event-time semantics requires injecting custom timestamp assigners and watermark generator classes into the dataflow. The Table API abstracts this completely by lifting time semantics into the data definition language (DDL).

```
t_env.execute_sql("""
    CREATE TEMPORARY TABLE sensor_source (
        sensor_id STRING,
        ts_string STRING,
        temperature DOUBLE,
        -- Computed column mapping a string to a system timestamp
        payload_ts AS TO_TIMESTAMP(ts_string, 'yyyy-MM-dd''T''HH:mm:
   ss.SSS''Z''''),
        -- Punctuation-based watermark generator bound to the
   catalog
        WATERMARK FOR payload_ts AS payload_ts - INTERVAL '2' SECOND
    ) WITH (
        -- Where to get data and file configurations
        -- We use filesystem but Flink offers many other options
        'connector' = 'filesystem',
        'path' = '/path/to/sensors_data.csv',
        'format' = 'csv'
    )
""")
```

Listing 1.4: Running Example: Table configuration via DDL

Computed columns and punctuation strategies (`WATERMARK FOR ...`) are bound directly to the catalog. The engine autonomously wraps the underlying physical `Source` with the appropriate `WatermarkAssigner` operator, allowing downstream nodes to track event-time progress transparently without exposing the synchronization logic to the user.

Similarly a sink configuration is specified via DDL.

```
t_env.execute_sql("""
    CREATE TEMPORARY TABLE sensor_summary_sink (
    -- Attributes
        sensor_id STRING,
        avg_temp DOUBLE,
        max_hum DOUBLE,
        reading_count BIGINT
    ) WITH (
        'connector' = 'print',
        'print-identifier' = 'SENSOR_METRICS'
    )
""")
```

Listing 1.5: Running Example: Sink configuration

Once the catalog is populated, developers build their streaming topology using fluent method chains. Unlike procedural APIs that accept opaque language lambdas (e.g., `lambda x: x.temperature > 10.0`), the Table API operates over symbolic expression trees (`col("temperature") > 10.0`).

```
from pyflink.table.expressions import col

source_table = t_env.from_path("sensor_source")

# Constructing the query
result_table = (
    source_table
    .filter(col("temperature") > 10.0)
    .group_by(col("sensor_id"))
    .select(
        col("sensor_id"),
        col("temperature").avg.alias("avg_temp")
    )
)

# Triggering physical lowering and execution
result_table.execute_insert("sensor_summary_sink").wait()
```

Listing 1.6: Running Example: Query construction with fluent methods

Because expressions are first-class objects, the underlying engine can statically inspect column dependencies and optimize operations (such as pruning unused fields) before physical lowering occurs.

Crucially, calling transformation methods like `.filter()` or `.group_by()` does not trigger immediate computation; it merely appends logical nodes to an internal abstract syntax tree (AST). Execution remains strictly *lazy*. The pipeline is compiled and deployed to the dataflow runtime only upon explicitly emitting results to a sink via `execute_insert()`.

1.5.1 Supported Operations and Plan Lowering

The transformation methods used to compose the query do not map 1-to-1 with physical operators. We will be using the terms *operation* (the user's intent) for high level constructs and a physical *operator* for the runtime implementation. A single relational operation often expands into a complex topology of physical nodes.

Supported operations cover all the typical SQL constructs while adapting stateful operations to streams:

- **Stateless Transformations:** Operations such as projections (`Select`) and filters (`Filter`) that evaluate records individually without maintaining historical state.
- **Stateful Aggregations:** Operations that compute metrics across multiple records, either continuously (Global `GROUP BY`) or bounded by time/count intervals (Tumbling, Sliding, and Session Windows).
- **Set Operations:** Operations that combine divergent dataflows possessing identical schemas. While modern engines fully support the standard relational set suite over continuous streams, a strict distinction must be made regarding their physical execution and memory footprint:
  - *Union All:* The only purely topological, stateless set operation. It performs a simple multiplexing merge of upstream channels without any deduplication overhead, making it perfectly suited for unbounded streams at zero memory cost.
  - *Union, Intersect, and Minus (Except):* Inherently stateful, multi-stage operations. To evaluate them on infinite streams, the physical engine must maintain localized hash states (to deduplicate, match, or subtract elements) and emit a dynamic changelog stream with retraction messages.
- **Unbounded Regular Joins (Inner and Outer):** Standard relational joins (Inner, Left, Right, and Full Outer Joins) correlate incoming records based

purely on equality predicates without imposing time limits. Consequently, the required state to compute the query grows infinitely, it can be managed with the environment TTL.

- **Time-Bounded Joins (Interval and Window Joins):** To natively prevent unbounded state accumulation, streaming engines introduce structurally bounded joins.
  - *Interval Joins:* Require a standard equi-join predicate coupled with a relative temporal bounding envelope (e.g., `left.ts BETWEEN right.ts - 5 min AND right.ts + 10 min`). Because the correlation timeframe is finite, the underlying state backend can safely evict historical records as soon as the event-time watermark surpasses their upper temporal bound.
  - *Window Joins:* Discretize the continuous streams into distinct temporal buckets (Tumbling or Sliding) before joining. Records are correlated exclusively if they fall into the same discrete time window.
- **Ordering:** global sorting (`ORDER BY`) is structurally incompatible with unbounded streams unless it is done on a time attribute or has a subsequent `FETCH` operation.

To observe the divergence between logical intent and physical execution, consider the running example from Section 1.5. Before execution, the query planner exposes the **Abstract Syntax Tree (AST)**, representing the pure relational algebra:

```
== Abstract Syntax Tree ==
//SELECTION
LogicalProject(sensor_id=[$0], avg_temp=[$1], max_hum=[$2],
   reading_count=[$3])
//GROUP BY
+- LogicalAggregate(group=[{0}], EXPR$0=[AVG($2)], EXPR$1=[MAX($3)],
    EXPR$2=[COUNT($0)])
    //FILTER
   +- LogicalFilter(condition=[>($2, 1.00E1)])
        //WATERMARKS
      +- LogicalWatermarkAssigner(rowtime=[payload_ts], watermark
   =[-($4, 2000:INTERVAL SECOND)])
        //FROM (implemented via a selection)
         +- LogicalProject(sensor_id=[$0], ts_string=[$1],
   temperature=[$2], humidity=[$3], payload_ts=[TO_TIMESTAMP(...)])
            +- LogicalTableScan(table=[[default_catalog,
   default_database, sensor_source]])
```

Listing 1.7: The Logical Relational Plan generated by the Table API parser.

This tree maps 1-to-1 with the user's declarative statements. However, executing this graph directly would be highly inefficient and would lack distributed routing semantics. The engine's optimizer (Apache Calcite [7] in Flink) applies rule-based transformations to produce the **Optimized Physical Plan**:

```
== Optimized Physical Plan ==
//AGGREGATION and SELECTION
GroupAggregate(groupBy=[sensor_id], select=[sensor_id, AVG(
   temperature) AS EXPR$0, MAX(humidity) AS EXPR$1, COUNT(sensor_id
   ) AS EXPR$2])
//KEYBY
+- Exchange(distribution=[hash[sensor_id]])
    //SELECTION and FILTER
   +- Calc(select=[sensor_id, ts_string, temperature, humidity,
   payload_ts], where=[>(temperature, 1.00E1:DOUBLE)])
    //WATERMAKS
      +- WatermarkAssigner(rowtime=[payload_ts], watermark=[-(
   payload_ts, 2000:INTERVAL SECOND)])
      //FROM
         +- Calc(select=[sensor_id, ts_string, temperature, humidity
   , TO_TIMESTAMP(...) AS payload_ts])
            +- TableSourceScan(table=[[...]], fields=[sensor_id,
   ts_string, temperature, humidity])
```

Listing 1.8: The lowered physical execution plan, featuring operator fusion and injected routing.

Comparing the two plans highlights the core responsibilities of the lowering phase:

1. **Operator Fusion (`Calc`):** The logical `Project` and `Filter/Where` nodes are not instantiated as separate threads. The compiler fuses them into single physical `Calc` operators, eliminating intermediate memory allocations and thread context switches.

2. **Injected Network Routing (`Exchange`):** The user never explicitly requested a data shuffle. However, because the query includes a `GROUP BY sensor_id`, the physical planner automatically injects an `Exchange` node with a `hash[sensor_id]` distribution policy. As discussed in Section 1.3.2, this enforces state isolation by ensuring that all records with the same sensor ID are routed to the same parallel subtask.

This translation pipelinefrom symbolic relational trees down to hardware-aware execution graphsis the architectural foundation that will be adapted and reproduced for the WindFlow library in the following chapters.

1.5.2 The Changelog Stream and RowKind Encoding

Because downstream network channels and terminal sinks require stream records rather than mutable tables, a dynamic table materializes its internal mutations into a *changelog stream*. Flink formalizes state transitions by tagging each physical tuple with an explicit mutation descriptor, termed **RowKind**:

- `+I` **(Insert):** Emits a newly added row into the relation.
- `-D` **(Delete):** Signals the eviction or explicit deletion of an existing row.
- `-U` **(Update_Before):** Retracts the previous state of a record prior to an update.
- `+U` **(Update_After):** Injects the newly updated value of a record.

This four-state retraction model extends CQL's primitive `Istream` and `Dstream` operators, allowing stateful transformations to be chained arbitrarily. For instance, in an unbounded continuous aggregation:

```
SELECT sensor_id, AVG(temperature) FROM SensorTable GROUP BY
   sensor_id;
```

every incoming sensor reading triggers the emission of a retraction pair: a `-U` record retracting the prior moving average, immediately followed by a `+U` record publishing the updated average.

1.5.3 Streaming Sink Semantics

Depending on the operations present in the physical plan, the resulting changelog stream dictates the compatibility of terminal sink connectors:

- **Append-Only Sinks:** Accept exclusively `+I` records. Queries that perform only selections, projections, or windowed aggregations (which emit finalized results once at window closure) can write directly to append-only destinations such as standard CSV files or Kafka topics.
- **Retract and Upsert Sinks:** Capable of consuming `-U`, `+U`, and `-D` mutations. When continuous non-windowed aggregations generate an evolving state, the sink must either process retraction messages explicitly or maintain an upsert key to overwrite prior state within external stores (such as key-value databases).

1.6 Introduction to WindFlow

WindFlow [12] is a C++17 library designed for high-performance data stream processing on multicore, scale-up architectures; as such there is no support for distributed computing. Built on top of the FastFlow [2] parallel programming framework, WindFlow aims to maximize throughput and minimize latency by embracing hardware-aware optimizationssuch as lock-free queues, thread pinning, and fine-grained data routing. Unlike high-level engines like Apache Flink, which abstract away the physical execution topology, WindFlow exposes a low-level, native interface that provides developers with granular control over thread placement, memory management, and stream partitioning.

However, this raw performance and structural flexibility come at the cost of developer ergonomics. Understanding WindFlow's execution model and its structural constraints is essential to motivate the design of TIDE introduced in this work.

1.6.1 PipeGraphs and Execution Topologies

In WindFlow, a streaming application is orchestrated by a top-level execution container called a *PipeGraph*. The actual processing topology is constructed by composing one or more macroscopic building blocks known as *MultiPipes*.

Rather than acting merely as simple communication edges, a `MultiPipe` encapsulates a set of parallel pipelines. These internal pipelines can either operate as strictly independent data channels or support complex all-to-all data routing between discrete computational stages. By leveraging asynchronous, lock-free mechanisms, a `MultiPipe` transports data items across threads without introducing centralized bottlenecks, while the `PipeGraph` governs the merging of these overarching blocks.

```
1    // pipegraph setup
2    wf::PipeGraph topology(
3        "topology_name",
4        wf::Execution_Mode_t::DEFAULT,
5        wf::Time_Policy_t::EVENT_TIME
6    );
```

Listing 1.9: Pipegraph creation and configuration

From a developer's perspective, building a `MultiPipe` simply requires appending or chaining single logical operators to the graph. Internally, however, the `MultiPipe` automatically maps each logical operator to a physical stage composed of P concurrent threads, where $P \geq 1$ represents the degree of intra-operator parallelism.

```
1    auto& pipe = topology.add_source(src1)  //par = 2
2                      .add(op1)             //par = 3
```

```
3                       .add(op2)                   //par = 2
4                       .chain(op3)                 //same threads as op2
5                       .add_sink(sink1);           //par = 1
```

Listing 1.10: Adding an operator to the MultiPipe doesn't require explicit intra-operator routing

As illustrated in Figure 1.6, mapping sequential logical operators onto varying degrees of physical parallelism (P) introduces the architectural challenge of data distribution. When a computational stage consumes data from an upstream producer, the `MultiPipe` routes the tuples across the available downstream thread replicas.

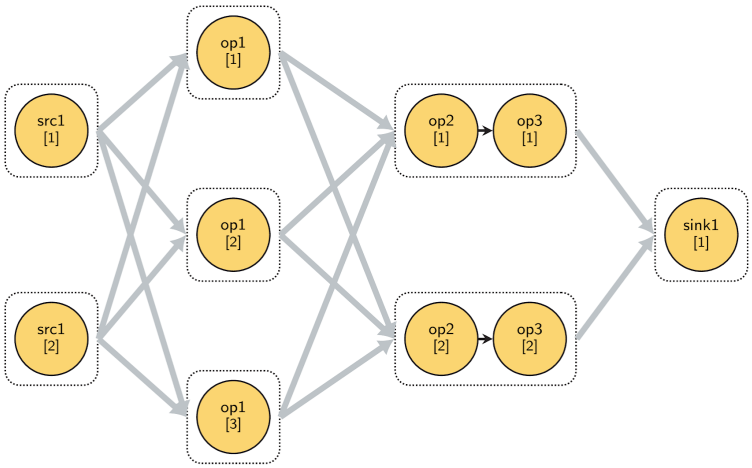


Figure 1.6: MultiPipe built by the topology in Listing 1.10

To govern how this physical distribution interacts with event ordering and temporal synchronization, WindFlow provides three configurable execution modes:

- **DETERMINISTIC:** in this mode, the user must generate inputs in the Sources ordered by timestamp. The run-time environment guarantees that all operator replicas will process inputs in timestamp order. This execution mode ensures deterministic execution, often at the expense of performance.
- **PROBABILISTIC:** in this mode, Source replicas may generate outputs in any order, but the run-time system still sorts items before applying the operators' functional logic. To maintain ordering, some items may be dropped. This execution mode can offer a reasonable compromise between ordering requirements and performance when the application semantics tolerate the loss of a small number of inputs.
- **DEFAULT:** this is the default mode, which provides no ordering guarantees.

Inputs are processed in a non-deterministic order, and temporal window-based computations rely on watermarks for progress.

In **DEFAULT** mode all operators (except sinks) can be configured to push tuples into the topology in custom-sized batches. Each tuple will be processed as a standalone by downstream operators, but while in the pipegraph they will move together to reduce traffic. This strategy can improve throughput at the cost of increased latency. If no batch size is specified, the default is zerowhich disables batchingand operators emit outputs individually.

A topology can contain more MultiPipes within it (*branches*); it is possible to merge them into one pipe acting as a unified stream.

```
1   //separate source streams
2   auto& pipe_1 = topology.add_source(src1).add(op1);
3   auto& pipe_2 = topology.add_source(src2).add(op2);
4
5   std::vector<wf::MultiPipe*> branches = {&pipe_1, &pipe_2};
6
7   //merge and adding the final operator
8   auto merged_pipe = wf::merge_multipipes_func(
9       &topology, branches
10  );
11  merged_pipe->add_sink(sink);
```

Listing 1.11: Merging two streams into a single sink

Once the topology is fully built the execution needs to be explicitly triggered:

```
1   // synchronous execution of the dataflow
2   topology.run();
3   //indefinite waiting for execution termination
4   //topology.wait_end();
```

Listing 1.12: Explicit execution trigger

1.6.2 Time Policies and Synchronization

As already presented, time semantics in WindFlow are declared globally at the topology level. The engine supports both `INGRESS_TIME` (where events are timestamped by the system clock upon entering the graph) and `EVENT_TIME` (where timestamps are extracted directly from the event payload). Unlike Apache Flink, which relies predominantly on explicit punctuation records injected into the data stream based on user-defined frequencies, WindFlow employs a hybrid, autonomous watermarking strategy.

By default, watermarks circulate directly with the data tuples as *pointstamps*. This effectively elevates the stream element from a simple pair to a triple:

$$e_i = \langle \mathbf{d_i}, t_i, w_i \rangle \tag{1.16}$$

Source operators emit records using a `Source_Shipper`, which explicitly pairs each tuple with an unsigned 64-bit integer timestamp (in microseconds) and periodically advances the stage's watermark by invoking `setNextWatermark()`. Downstream stateful operators track these internal metadata boundaries to trigger temporal evaluations (such as window closures) accurately.

```
1     //inside the Source Operator
2     //usually inside a loop to ship all incoming data
3
4             //user-defined record extraction
5             RecordT record = ....
6             uint64_t ts_us = ....
7             uint64_t wm_us = ....
8
9             //required timestamping
10            shipper.pushWithTimestamp(record, ts_us);
11
12            //depending on the users extraction policy wm could be
    different from ts
13            shipper.setNextWatermark(wm_us);
```

Listing 1.13: An abstract example of shipping a tuple in event time

In `INGRESS_TIME` tuples are shipped with a simple `push()` method since timestamp and watermark are automatically assigned.

However, relying exclusively on pointstamps introduces a temporal synchronization issue on sparse streams. If an intermediate operator (such as a highly selective `Filter`) drops a large number of consecutive tuples, the physical network channel becomes idle. Without outgoing tuples to carry the new pointstamps, downstream operators would suffer from event-time starvation, preventing window closures.

To overcome this, WindFlow automatically detects when a physical channel has not propagated data for a certain period and dynamically falls back to injecting explicit punctuation records. While Flink requires the programmer to explicitly configure the emission logic and frequency of punctuations, WindFlow manages this dual mechanism (pointstamps and idle-channel punctuations) entirely autonomously.

1.6.3 Core Operators: Builders and Functors

To define a processing stage, WindFlow utilizes a builder-based design pattern that cleanly separates physical configuration from user-defined logic. Every node in the graph is constructed using two primary components:

1. **The Functor:** A C++ `struct` or `class` that encapsulates the actual computation logic (typically implemented by overloading the `operator()`). The functor is instantiated within the builder and executed independently by each thread replica.

```
class Kelvin_Map_Functor {
    public:
    SensorRecord operator()(const SensorRecord &input) {
        SensorRecord output(input);
        output.temperature += 273.15;
        return output;
    }
};

```

Listing 1.14: Logic needed to change a temperature from Celsius to Kelvin

2. **The Builder:** A configuration object (e.g., `Map_Builder`, `Reduce_Builder`) that dictates the physical properties of the stage, such as its identifier, its parallelism degree (P), and, crucially, the *Key Extractor* function required for deterministic routing.

```
//user-defined logic
Kelvin_Map_Functor kelvin_logic;

//operator building with physical parameters
auto kelvin_op = wf::Map_Builder(kelvin_logic)
    .withName("to_Kelvin")
    //intra-operator parallelism
    .withParallelism(2)
    .build();

```

Listing 1.15: Creating a Map Operator with the user-defined logic

Stateful operators also need a `.withKeyBy()` with a key extraction function to properly route tuples between replicas.

WindFlow supports a varied catalog of operators, which can be categorized by their semantics and state retention:

- **Boundary Operators:** `Source` and `Sink` nodes act as the entry and exit points of the PipeGraph, interfacing with external systems such as files, or message brokers (Apache Kafka, RocksDB).

- **Tuple-at-a-time Transformations:** `Map`, `Filter`, and `FlatMap` apply transformations, predicates, or one-to-many record expansions to individual tuples independently.

- **Global Stateful Aggregations:** The `Reduce` operator enables continuous aggregations over unbounded streams. It evaluates running aggregates continuously, typically emitting updates with an append-only changelog semantic (where each state mutation generates a new independent output record).

- **Windowed Aggregations:** For discrete, time-bound intervals, the primary abstraction is the `KeyedWindows` operator. Windows of different keys are executed in parallel (key-based) while windows of the same key are executed sequentially and each from scratch.

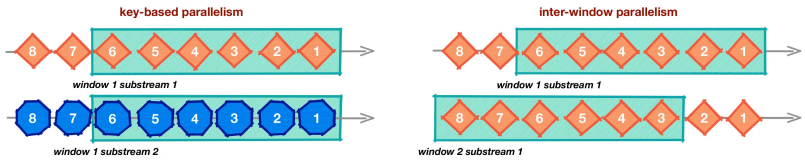


Figure 1.7: Architectural distinction between *Key-Based* and *Inter-Window* parallelism strategies in WindFlow.

There are more operators which allow for better window parallelism but those won't be relevant for this thesis (e.g. PanedWindows [11]).

- **Stream Joins:** To correlate divergent dataflows, WindFlow provides structurally bounded operators (Window and Interval Joins). The engine implements two distinct join evaluation algorithms to optimize the correlation based on the strictness of the temporal bounds and the presence of theta-predicates.

- **Heterogeneous GPU Operators:** For computationally intensive tasks, WindFlow extends its catalog with specialized nodes capable of offloading execution to GPU accelerators.

In the next chapters relevant operators to TIDE implementation will be discussed in further detail.

1.6.4 Current Limitations

While WindFlow achieves state-of-the-art performance through its C++ architecture, deploying continuous queries directly via its native API exposes several structural and ergonomic limitations:

- **Lack of Relational Abstractions:** The API operates purely on raw C++ structs and pointers. There is no native concept of "Tables," "Schemas," or SQL-like data types, forcing developers to manually define and cast data structures for every intermediate processing step.
- **Absence of Unbounded Joins:** WindFlow lacks native, high-level operators for joining data streams. Correlating streams requires manually implementing complex state tracking and temporal bounding logic to prevent memory exhaustion.
- **Pure Streaming Paradigm:** As a pure stream processor, WindFlow lacks inherent batching capabilities outside of internal pipe routing optimizations. Global operations across finite datasets must be manually emulated.
- **Manual State Management and Absence of TTL:** While WindFlow efficiently handles time-bounded windows, it natively lacks automated *State Retention* (Time-To-Live) policies for continuous, non-windowed aggregations. In stateful operators like `Reduce`, the internal state grows monotonically with the cardinality of the grouping keys. Without a built-in TTL mechanismsuch as the one provided by Flink's execution environmentunbounded queries risk infinite memory accumulation.

Formulating complex queries requires extensive C++ boilerplate, handling timestamp (and watermark) extraction, and enforcing correct routing. These limitations directly motivate the development of TIDE presented in the subsequent chapters.

Chapter 2

Overall API Architecture

Rather than adopting a monolithic structure, the system is designed around a strict separation of concerns, ensuring that the frontend logical representation remains agnostic of the physical C++ backend. To achieve this, the software ecosystem is logically partitioned into three main subsystems: the frontend `api` for query composition, the `codegen` engine for C++ translation, and the `runtime` manager for automated execution.

This chapter provides a bird's-eye view of these subsystems, detailing their core responsibilities and illustrating how they cohesively interact to seamlessly transform a user's Python script into a native stream processing application. The subsequent chapters will explore these components in detail: Chapter 3 is dedicated to the frontend Python API and its logical semantics, while Chapter 4 delves into the code generation engine and the native C++ runtime backend.

2.1 Python API

To provide a seamless declarative experience without compromising the underlying performance of the C++ engine, the framework adopts a decoupled multi-tier architecture. The software ecosystem, provided as a Python package named `windflow_tide`, is logically partitioned into three main subsystems: the frontend `api`, the `codegen` engine, and the `runtime` manager.

This separation of concerns ensures that the logical formulation of a continuous query remains independent from its physical C++ translation and subsequent native execution. However, the framework is designed to function cohesively. The `api` module constructs the query's logical representation, the `codegen` module translates it into physical C++ components, and the `runtime` module orchestrates the compilation and execution phases.

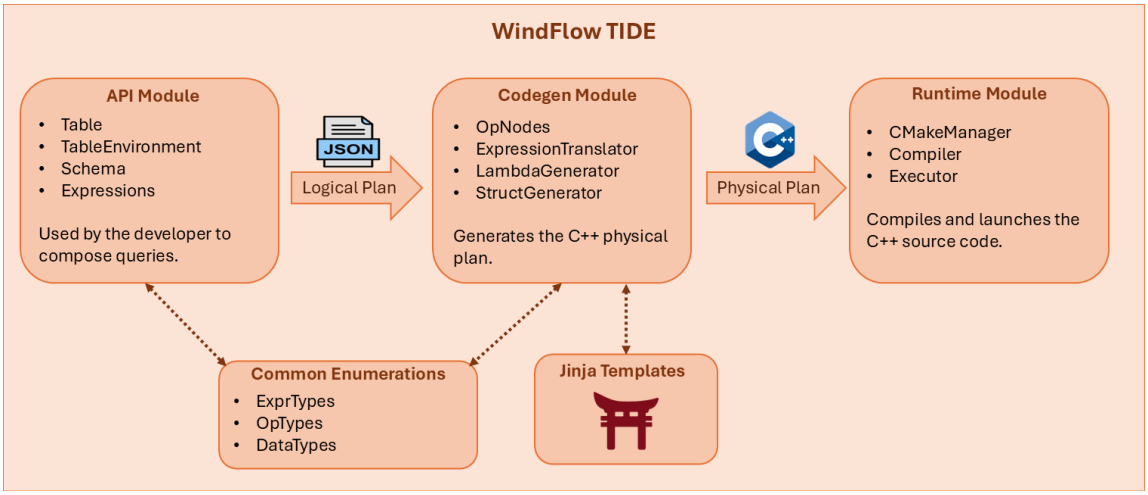


Figure 2.1: Overall architecture showing how different subsystems interact

2.1.1 The Python API Subsystem

The `api` module acts as the user-facing frontend of the framework. It exposes the Domain Specific Language (DSL) that allows developers to define schemas, connect to data sources, and compose relational operations (such as `where`, `select`, `group_by`, and `join`) using fluent method chaining.

Similarly to Flink, the API layer operates on a *lazy evaluation* paradigm where execution needs to be explicitly requested. This module is responsible for validating the logical tree, resolving dependencies, and serializing queries into an Abstract Syntax Trees (AST) formatted as a JSON files. These JSON artifacts serve as intermediate representations between the frontend API and the lower-level backend modules.

2.1.2 The Code Generation Subsystem

The `codegen` module represents the bridge between the Python logical realm and the native WindFlow execution. It ingests the JSON AST produced by the API and translates it into valid, high-performance C++17 source code. To manage the complexity of this translation, the subsystem is internally divided into specialized components:

- **JSON Parser and Operation Nodes:** The `JsonParser` reads the serialized AST and reconstructs the directed acyclic graph in memory. Instead of generic dictionaries, it instantiates a typed intermediate representation (IR) composed of `OpNode` subclasses (e.g., `WhereOpNode`, `JoinOpNode`).

- **Graph Explorer:** The `GraphExplorer` acts as the orchestrator of the compilation phase. It performs a post-order traversal of the IR graph, ensuring that upstream dependencies are processed before downstream consumers. For each node, it invokes the necessary translation services and accumulates the generated C++ topologies.

- **Schema Generator:** Invoked during the graph traversal, it maps logical data types to native C++ types and dynamically synthesizes the memory layouts (C++ `structs`) required for tuples. It applies strict deduplication strategies, ensuring that identical schemas across different operators reuse the same C++ structure to avoid compilation errors and minimize overhead.

- **Expression Translator and Lambda Generator:** These components work in tandem. The `ExpressionTranslator` recursively converts mathematical, logical, and column-referencing expressions into C++ strings. Subsequently, the `LambdaGenerator` packages these expressions into fully typed C++ lambdas required by the physical WindFlow builders.

During the traversal, the `GraphExplorer` relies on a unified Jinja2 [14] environment to inject the translated structs, lambdas, and builder configurations into template files (such as `main.cpp.jinja2`), producing the final, ready-to-compile C++ source code.

2.1.3 The Runtime Subsystem

Once the C++ source files are generated, the `runtime` module takes over to automate the build and execution lifecycle. The primary goal of this subsystem is to completely abstract the toolchain invocation, preventing the user from ever needing to manually interact with terminal commands.

First, it dynamically generates or updates a `CMakeLists.txt` configuration file, establishing the proper include directories and linking instructions to the WindFlow and FastFlow libraries. Next, it invokes the compiler.

Finally, upon a successful build, the runtime spawns the native executable as a detached, background process. The standard output and error streams are safely redirected to log files, and the control flow is immediately returned to the user through a `JobHandle` object. This handle allows the developer to non-blockingly monitor the execution, fetch logs, or terminate the native streaming process directly from the Python script.

2.2 WindFlow Table Operations

Instantiating WindFlow's native operators with relational logic directly is possible, but it requires generating both a dedicated configuration Builder and a complete C++ Functor struct for every single pipeline stage. To simplify the code generation process and reduce boilerplate, we expanded WindFlow with specific *Table Operations*. These are specialized wrappers around the original physical operators that abstract away the Functor definition; instead, they accept a simple lambda function and automatically apply it in accordance with relational semantics.

With this approach, the code generator only needs to emit the Table-Operation-Builder and its associated inline lambda logic (Section 4.2). This eliminates the necessity of synthesizing dozens of structurally identical C++ Functors that differ only slightly in their execution body.

Given the multi-tiered architecture of the framework, it is crucial to establish a strict taxonomy to disambiguate the different representations an operator assumes throughout its lifecycle:

- **API Operator:** A Python object from the frontend `api` module (e.g., `WhereOp`, `SelectOp`) used by the developer to compose queries. It is evaluated and eventually serialized into the JSON AST.
- **Operation Node (OpNode):** The typed intermediate representation used during the code generation phase. It is reconstructed from the serialized JSON and visited by the generation algorithms.
- **Table Operation:** The native C++ wrapper generated by the `codegen` module. It implements relational logic by exposing a simplified builder interface (e.g., `Select_Builder`) that binds generated lambdas to the underlying functors.
- **WindFlow Operator (wf-operator):** The final, physical C++ object instantiated at runtime and strictly appended to the native topology.

Chapter 3

Python API Syntax and Semantics

Building upon the architectural overview presented in the previous chapter, this chapter delves into the user-facing frontend of the framework: the `api` module. The primary objective of this module is to abstract the low-level complexities of the underlying WindFlow C++ library, offering developers a fluent, declarative DSL for continuous query composition. Similarly to Flink's Table API, WindFlows TIDE operates on a *lazy evaluation* paradigm, it does not execute data processing directly; instead, it focuses on constructing a logical representation of the pipeline. It statically validates the logical tree, resolves dependencies, and ultimately serializes the valid queries into a JSON artifact. This chapter explores the api's syntax and semantics, detailing its mechanisms from the central execution context down to the definition of data sources, relational expressions, and stateful multi-stream operations.

3.1 Execution Context and Environment Setup

The entry point of any application built with the framework is the `TableEnvironment`. Serving as the central orchestrator, this class acts simultaneously as the execution context and the catalog manager. It tracks registered sources, resolves table references during query composition, and eventually orchestrates the final AST serialization and C++ code generation phases upon the invocation of the `execute()` method.

```
    env = TableEnvironment(
        include_dir = Path("../includes"),
        par = 2,          # 1 by default
        policy= TimePolicy.INGRESS_TIME
    )

    #input files configuration
    file_config = ....

    #source registration
    tab = env.table_from_file(
        file_config,    #schema, format, timestamp order, etc..
```

```
        "source_table"  #name attributed to the table
    )

    #query composition
    q = tab.......

    #serialization, code generation and execution dispatch
    env.execute(q)
```

Listing 3.1: Abstract use of the TableEnvironment

When instantiating the `TableEnvironment`, the developer establishes the foundational execution parameters governing the resulting physical topology.

A critical configuration is the base degree of intra-operator parallelism, defined via the `par` argument. Every table-operation that supports parallelism greater than one will be built with intra-operator parallelism equal to `par`.

Another fundamental parameter is `include_dir`. Leveraging Python's standard `pathlib`, it instructs the internal runtime where to locate the WindFlow and FastFlow C++ headers during the automated Makefile generation.

Other more complex arguments require a more detailed explanation in specific sections: 3.1.1 for `policy` and 3.2.2 for `time_baseline`.

3.1.1 Time Policies

The framework requires the explicit declaration of a global time policy during the `TableEnvironment` initialization through the `TimePolicy` enumeration.

Three distinct policies are supported:

- `NO_POLICY`: The default configuration. Under this policy, the topology operates as a purely stateless or purely data-driven pipeline. It strictly prohibits the use of time-bounded windows or interval joins, and it rejects the definition of timestamp extraction within source tables.

- `INGRESS_TIME`: Uses WindFlow's native IngressTime mode in which timestamps are implicitly assigned by the system clock at the exact moment a tuple is generated. Like `NO_POLICY`, it does not allow explicit timestamp extraction in the source files, as time is hardware-dependent.

- `EVENT_TIME`: Uses WindFlow's native EventTime mode in which the temporal progress is dictated by the data itself rather than the wall-clock time. It requires every source table to declare a mandatory *time-column* to extract timestamps directly from the payload.

By centralizing this configuration, the `TableEnvironment` can enforce rigorous *fail-fast* validation. If a developer attempts to declare a time-windowed `GroupBy` operation while the environment is configured with `NO_POLICY`, the API intercepts the logical error during the validation phase, raising a Python exception before any AST serialization or C++ code generation occurs.

3.2 Stream Sources

With the execution context established, the first step in composing a pipeline is registering the input data streams. Since the framework relies on a *lazy evaluation* model, registering a source does not immediately allocate memory or parse files; it merely adds a logical reference to the environment's catalog.

Sources are declared using the `InputFileConfiguration` class, which encapsulates the physical metadata of the stream, and are registered via the `env.table_from_file()` method which returns a `Table` object.

```
1
2     #prior definition of the input schema
3     sensor_schema = ....
4
5     file_cfg = InputFileConfiguration(
6         file_path="data/sensor_stream.csv",
7         schema=sensor_schema,
8         #currently only CSV files are supported
9         format=FileFormat.CSV,
10        #whether or not to skip the header line
11        has_header=True,
12        #which column represents the timestamp in Event Time
13        time_col="timestamp_col"
14
15        #plus others that will be covered later
16    )
17
18    sensor_table = env.table_from_file(file_cfg, "sensor_stream")
```

Listing 3.2: Configuring and registering a CSV stream source

Currently, the framework supports exclusively CSV files as data sources. While the underlying WindFlow engine offers sophisticated I/O connectors for distributed message brokers (such as Apache Kafka) and key-value stores (such as RocksDB), the primary objective of this thesis is the design of a high-level relational API. Consequently while extensions are possible, physical ingestion mechanisms were intentionally kept straightforward to maintain focus on the abstract query syntax and semantics.

3.2.1 Schemas and Types

Data flowing through the physical WindFlow pipelines must be strictly typed to allow the generation of C++ `structs`. WindFlow TIDE provides a `SchemaBuilder` to define the structure of the incoming tuples. Developers map the columns of their dataset to a set of supported `DataTypes` (e.g., `STRING`, `INT`, `FLOAT`, `DOUBLE`, `BOOLEAN`).

Because the underlying C++ source builder parses data positionally to maximize ingestion throughput, the logical sequence of columns declared in the source `Schema` must strictly mirror the physical column order of the original CSV file.

```
    #sensor readings schema that will be used in running examples
    sensor_schema = (SchemaBuilder()
        .add_column("sensor_id", DataTypes.STRING)
        .add_column("timestamp", TimeFormats.ISO8601)
        .add_column("temperature", DataTypes.DOUBLE)
        .add_column("humidity", DataTypes.DOUBLE)
        #finalizes the building and returns the Schema object
        .build()
    )
```

Listing 3.3: Example of schema definition

Internally, a `Schema` is modeled as an immutable collection of `Field` objects. Each field encapsulates a logical name, its corresponding type, and, optionally, the abstract syntax tree expression that derived it (see 3.3).

To unify standard scalar types and temporal formats, the API relies on a common base class: the `TypeDescriptor`. Both `DataTypes` and `TimeFormats` are enumeration subclasses implementing it. `TypeDescriptor` exposes essential properties for static analysissuch as `is_numeric`, `is_boolean`, and `is_temporal`allowing the framework to reason polymorphically about types during query composition.

By treating types as first-class citizens in the Python API, the framework can perform rigorous static type-checking during the drafting phase. Simultaneously, the backend code generator accesses the `TypeDescriptor` definitions to synthesize the exact native C++ types (e.g., `int32_t`, `double`, `std::string`) required for the generated physical structures.

A pivotal design choice is the inclusion of temporal formats directly within the type system (e.g., `TIMESTAMP_ISO8601`). Instead of treating time as a generic string requiring custom user-defined lambdas for parsing, the API recognizes temporal columns at the schema level. During compilation, the code generator automatically maps these logical temporal types to a native `uint64_t` C++ type (which represents microseconds since UNIX epoch) and injects the corresponding parsing function directly into the CSV reader logic.

3.2.2 Extracting a Timestamp in EventTime

When the environment operates under `TimePolicy.EVENT_TIME`, temporal progression is strictly dictated by the data payload. In this scenario, the configuration requires the declaration of a `time_col` argument, which must exactly match the name of a temporal column (a column with a `TimeFormats` type) previously defined in the schema.

Inspired by Apache Flink's architecture, the designated timestamp column serves a dual purpose:

1. **Relational Attribute:** It exists as a standard, queryable field within the logical record payload, allowing the developer to perform standard projections or filtering.

2. **System Metadata (Timestamp):** It acts as the engine's internal driving force. The C++ source builder automatically extracts this value, injecting the explicit timestamp metadata required to govern downstream watermarks and window temporal boundaries.

In WindFlow, data elements are not propagated as raw structs; instead, every tuple circulates within an internal runtime wrapper. This wrapper carries the essential system metadataspecifically, the watermarks and the internal timestamp utilized by the runtime to govern stateful operators.

The domain of these internal system timestamps is strictly defined as a range of integers representing the elapsed microseconds from an abstract instant zero (the logical start of the execution). Consequently, raw absolute timestamps extracted from the payload (such as large UNIX epoch values) cannot be directly injected into the wrapper's metadata. They must be mapped into the relative system time domain.

To ensure a consistent translation and avoid chronological misalignments across multiple streams, TIDE enforces a unified normalization strategy governed by a single `time_baseline` (or *global epoch* during codegen), which is explicitly configured at the `TableEnvironment` level. This global parameter guarantees a shared linear time scale across the entire topology.

To ensure correct temporal shifts and avoid negative timestamps, this baseline must be carefully chosen to be less than or equal to the absolute minimum timestamp present across all registered input datasets:

$$\forall i \in \mathcal{F}, \quad \text{time_baseline} \leq \min(t_i) \tag{3.1}$$

where $\mathcal{F}$ represents the set of all input files and t_i denotes the temporal values within the i-th file.

Choosing the baseline requires either some preprocessing to find the minimum timestamp or prior knowledge regarding when data collection started.

```
    env = TableEnvironment(
        include_dir=Path("../includes"),
        par=2,
        policy=TimePolicy.EVENT_TIME,
        time_baseline=(
            "2026-09-01T00:00:00.000Z",      #01/09/26 at midnight
            TimeFormats.ISO8601
        )
    )
```

Listing 3.4: Configuring the time baseline for complete TableEnvironment initialization

During the ingestion phase, the automatically generated C++ parsing lambda translates the incoming temporal string into an absolute microsecond value and applies a linear shift:

$$\text{normalized_ts} = \text{raw_timestamp} - \text{time_baseline} \tag{3.2}$$

This simple, deterministic subtraction guarantees that the entire execution graph operates on a safe, unified low-bound temporal domain.

3.2.3 Out-of-Order Data and Durations

When a pipeline operates under `EVENT_TIME` policy, data streams might be out-of-order ($i < j \not\Rightarrow t_i \leq t_j$). To guarantee correct temporal evaluations without prematurely discarding late tuples, the underlying stream processing engine requires a robust watermarking strategy. Taking inspiration from Flink [5], we introduce a conservative slack delay (δ) that shifts the watermark relative to the maximum observed timestamp:

$$w = \max(t) - \delta \tag{3.3}$$

In TIDE, this δ tolerance is explicitly configured for each data source using the `delay` parameter within the `InputFileConfiguration`. Setting the `order` flag to `False` and providing a delay parameter instructs the C++ source builder to generate safe watermarks that accommodate out-of-order network traffic.

To provide a declarative and highly ergonomic interface for defining these delays—as well as window sizes and sliding intervals—the API introduces the `Duration` class. Rather than forcing the developer to manually calculate massive integer values in microseconds (a practice prone to arithmetic errors and poor code readability), the framework abstracts time into a pair of *value* and *time_unit* (e.g., `SECONDS`, `MINUTES`,

HOURS).

```
    file_cfg = InputFileConfiguration(
        path="data/sensor_stream.csv",
        schema=sensor_schema,
        format=FileFormat.CSV,
        has_header=True,
        time_col="timestamp",

        #timestamp ordering configuration
        order=False,
        delay=Duration.minutes(2)    # Slack delay (delta)
    )
```

Listing 3.5: Configuring an out-of-order stream source with a 2-minute delay using the Duration class.

Developers instantiate `Duration` objects through fluent static factory methods, such as `Duration.milliseconds(500)` or `Duration.days(1)`.

The native WindFlow engine operates exclusively on 64-bit unsigned integers (`uint64_t`) representing continuous microseconds. To completely decouple the AST serialization from complex unit conversion logic, the `Duration` class ultimately computes and exposes its microsecond equivalent.

3.2.4 Source Parallelization

To maximize ingestion throughput, TIDE allows source operators to be scaled with intra-operator parallelism ($P > 1$). However, scaling a file source introduces the challenge of partitioning the dataset across multiple concurrent worker threads without relying on a centralized, bottleneck-prone split coordinator.

By default, if no specific chunk size is provided to the configuration, the API employs a macroscopic partitioning strategy. It calculates the total physical size of the input file and performs a ceiling division, assigning a single, continuous, equally sized block of bytes to each thread. For example, with $P = 2$, the first thread processes the first half of the file, while the second thread simultaneously processes the latter half.

While optimal for purely stateless transformations, this default strategy introduces severe inefficiencies when operating under `EVENT_TIME` policy. Because downstream operators compute their active event-time clock as the strict minimum watermark across all incoming parallel channels, temporal progression becomes severely bottlenecked.

$$W_{\text{current}} = \min_i W_i \tag{3.4}$$

The thread assigned to the first half of the file processes the oldest timestamps, while the thread assigned to the second half immediately reads events from the distant future.

Consequently, the downstream global watermark remains anchored to the past until the first thread completely finishes its massive block. This temporal skew prevents windows from closing, delaying result emission and causing catastrophic memory bloat, as stateful operators are forced to buffer the second thread's future events indefinitely.

To resolve this temporal misalignment, the `InputFileConfiguration` exposes the `split_size` parameter, enabling an *interleaved splitting* strategy. Instead of partitioning the file into P macroscopic halves, the file is segmented into numerous smaller, fixed-size byte chunks. The framework then distributes these splits across the available thread replicas in a round-robin fashion.

```
    # Configuring a parallel source with interleaved splits
    file_cfg = InputFileConfiguration(
        path="data/sensor_stream.csv",
        schema=sensor_schema,
        format=FileFormat.CSV,
        has_header=True,
        time_col="timestamp_col",
        order=False,
        delay=Duration.minutes(2),

        # Interleaved chunk size
        split_size=SplitSize.megabytes(64)
    )
```

Listing 3.6: Configuring a parallel source with 64 MB interleaved splits to maintain watermark alignment.

By processing smaller, interleaved chunks, the worker threads advance through the physical file in tandem. Their local watermarks remain chronologically aligned, allowing the downstream global watermark to progress smoothly. This ensures timely window evaluations and strongly bounds memory consumption, both for strictly ordered datasets and out-of-order streams.

The API fully abstracts the complexity of physical byte-level parallel I/O. The underlying C++ backend automatically manages boundary alignmentssuch as correctly resolving and skipping partial CSV lines cut across chunk bordersensuring that the logical data integrity is perfectly maintained regardless of the chosen `split_size`.

3.3 Customizable Expressions and Type Checking

In native WindFlow, applying a filter or projecting a new column requires the developer to explicitly write and compile a C++ lambda function encapsulated within a dedicated Functor object. While this guarantees maximum runtime performance, it breaks the declarative nature of a high-level API.

To bridge this gap, TIDE introduces a robust, purely declarative expression system. Expressions are immutable, stateless Python objects that represent a logical tree of operations. Rather than executing the computation immediately, these objects recursively build an AST when combined. This AST is subsequently traversed, statically type-checked against the input schemas, and ultimately translated into C++ code by the `codegen` backend.

3.3.1 Expression Grammar and Operator Overloading

The composition of expressions follows a strict, recursive grammar. The API leverages Python's *magic methods* (e.g., `__add__`, `__gt__`) to overload standard language operators, allowing users to write complex mathematical and logical conditions using a fluent and natural syntax.

The syntax supported by TIDE can be formalized using the following Backus-Naur Form (BNF) notation:

$$
\begin{aligned}
\langle \text{Expression} \rangle ::= {} & \langle \text{Literal} \rangle \mid \text{col}(\langle \text{String} \rangle) \mid \langle \text{Agg} \rangle \mid \text{current_time}() \\
& \mid \langle \text{Expression} \rangle \langle \text{BinOp} \rangle \langle \text{Expression} \rangle \\
& \mid \langle \text{UnaryOp} \rangle \langle \text{Expression} \rangle \\
\langle \text{Literal} \rangle ::= {} & \text{lit}(\langle \text{Value} \rangle, \langle \text{DataType} \rangle) \\
\langle \text{Agg} \rangle ::= {} & \text{count}() \mid \langle \text{AggFunc} \rangle(\langle \text{Expression} \rangle) \\
\langle \text{AggFunc} \rangle ::= {} & \text{sum} \mid \text{avg} \mid \text{min} \mid \text{max} \\
\langle \text{BinOp} \rangle ::= {} & \texttt{+} \mid \texttt{-} \mid \texttt{*} \mid \texttt{/} \\
& \mid \texttt{>} \mid \texttt{>=} \mid \texttt{<} \mid \texttt{<=} \mid \texttt{==} \mid \texttt{!=} \\
& \mid \texttt{\&} \mid \texttt{|} \\
\langle \text{UnaryOp} \rangle ::= {} & \texttt{\~}
\end{aligned}
$$

In this section standard arithmetic and boolean expressions will be covered in detail; although some of these considerations also apply to `AggregateExpression` those will be covered in Section 3.6.

The leaves of the abstract syntax tree are always either constants (`LiteralExpression`) or references to existing schema fields (`ColRefExpression`, instantiated via the `col()` helper). These terminal nodes can be recursively combined using binary or unary operators resulting in `BinaryOpExpression` (BOE) and `UnaryOpExpression` (UOE) subtrees.

Because Python does not allow the overloading of the built-in `and`, `or`, and `not` keywords (which strictly evaluate the boolean truth value of an object rather than returning a new object), the API co-opts Python's bitwise operators to represent logical conjunction, disjunction, and negation:

- The ampersand (`&`) is used for logical `AND`.
- The pipe (`|`) is used for logical `OR`.
- The tilde (`~`) is used for logical `NOT`.

By relying on this grammatical structure, a filtering condition such as evaluating a temperature threshold of a specific sensor is naturally expressed in Python as:

```
    cond = (col("temperature") > 30.5) & (col("sensor_id") == "S01")
```

Listing 3.7: Example of composing a binary AST using overloaded logical and comparison operators.

Behind the scenes, this single line of code instantiates a deeply nested BOE tree, entirely decoupled from the physical data, ready to be statically validated against the table's schema during query compilation.

During the query serialization phase, TIDE traverses this tree and serializes it into a JSON AST. This JSON artifact serves as the strict, language-agnostic contract between the frontend API and the C++ code generator subsystem.

For instance, consider the expression in listing 3.7. When serialized, the resulting expression node is transformed into a deeply nested dictionary, capturing the logical operation types, operands, and explicitly inferred data types:

```
{
  "expr_type": "BINARY_OP",
  "data_type": "BOOL",
  "name": "temperature_gt_30.5_and_sensor_id_eq_S01",
  "op": "&&",
  "left": {
    "expr_type": "BINARY_OP",
    "data_type": "BOOL",
    "name": "temperature_gt_30.5",
    "op": ">",
    "left": {
```

```
      "expr_type": "COL_REF",
      "data_type": "DOUBLE",
      "name": "temperature"
    },
    "right": {
      "expr_type": "LITERAL",
      "data_type": "DOUBLE",
      "name": "30.5",
      "value": 30.5
    }
  },
  "right": {
    "expr_type": "BINARY_OP",
    "data_type": "BOOL",
    "name": "sensor_id_eq_S01",
    "op": "==",
    "left": {
      "expr_type": "COL_REF",
      "data_type": "STRING",
      "name": "sensor_id"
    },
    "right": {
      "expr_type": "LITERAL",
      "data_type": "STRING",
      "name": "S01",
      "value": "S01"
    }
  }
}
```

Listing 3.8: Serialized JSON representation of a binary expression tree containing logical operators, column references, and literals.

Given the representation length for such a simple condition, the JSON ASTs will be omitted from now on.

Every node in this JSON hierarchy explicitly declares its `data_type`. This metadata is crucial: it is not inferred dynamically during the final C++ compilation, but is rigorously calculated and injected a priori by the Python frontend through a static Type Checking mechanism.

3.3.2 Static Type Checking and Type Coercion

As observed in the serialized AST, every logical node ,whether a column reference an operator, or a simple constant, is strictly associated with a resolved `data_type`. Because the downstream WindFlow C++ engine operates on statically typed data

structures, the TIDE must meticulously infer and validate the type of every expression before executing the pipeline.

This validation relies on a polymorphic `get_type(schema)` method implemented by every expression node, which recursively infers the output type of the tree from the bottom up.

Implicit Literal Inference The leaves of the AST are the foundation of this type inference mechanism. When a developer combines a column reference with a raw Python scalar (e.g., `col("temperature") > 30.5`), the framework invokes an internal `_to_expr()` helper. This method intercepts the native scalar and implicitly wraps it into a `LiteralExpression`.

Simultaneously, an internal `_infer_literal_type()` function maps the dynamic Python primitive to its corresponding `DataTypes`. For example, Python `int` objects are mapped to `DataTypes.INT`, `float` to `DOUBLE`, `str` to `STRING`, and `bool` to `BOOLEAN`. This automated boxing completely shields the developer from manually declaring types for constants, maintaining the API's declarative fluency while preserving strict typing.

Polymorphic Type Rules and Registries Once the leaves are typed, intermediate binary and unary nodes must compute their resulting output types based on their specific operation. To maintain a clean and extensible architecture the API delegates type inference to a set of rule registries, such as `BINARY_TYPE_RULES` and `UNARY_TYPE_RULES` accessed from the `resolve_(binary or unary)_type` method of `BinaryExprOp` and `UnaryExprOp` classes.

These dictionaries map formal operator enumerations (e.g., `BinExprOp.GT`) to dedicated resolving functions:

- **Logical Operators (`AND`, `OR`, `NOT`):** The resolver statically verifies that all operands are inherently `BOOLEAN`, raising a Python `TypeError` otherwise. The resulting operation always yields a `BOOLEAN`.
- **Comparison Operators (`>`, `==`, etc.):** The resolver enforces that operands either share the exact same type or are both numeric. The output type is invariably `BOOLEAN`.
- **Arithmetic Operators (`+`, `*`, etc.):** The resolver guarantees that both operands belong to the numeric family. However, the output type depends on the specific combination of the inputs.

Such an approach is used multiple times in the API to allow different implementations of methods, as well as code reuse, from objects of the same class without requiring an extensive class hierarchy that could lead to over-engineering.

```
    #entry point method
    def resolve_binary_type(
        self, t_left: TypeDescriptor, t_right: TypeDescriptor
    ) -> TypeDescriptor:
        target = BINARY_TYPE_RULES[self]
        return target(t_left, t_right)

    #enum -> typechecking strategy
    BINARY_TYPE_RULES: Dict[
        BinExprOp,
        Callable[[TypeDescriptor, TypeDescriptor], TypeDescriptor]
    ] = {
        BinExprOp.PLUS: _resolve_arithmetic,
        BinExprOp.MINUS: _resolve_arithmetic,
        BinExprOp.MULT: _resolve_arithmetic,
        BinExprOp.DIV: _resolve_arithmetic,
        BinExprOp.GT: _resolve_comparison,
        BinExprOp.LT: _resolve_comparison,
        BinExprOp.GE: _resolve_comparison,
        BinExprOp.LE: _resolve_comparison,
        BinExprOp.EQ: _resolve_comparison,
        BinExprOp.NE: _resolve_comparison,
        BinExprOp.AND: _resolve_logical,
        BinExprOp.OR: _resolve_logical,
    }
```

Listing 3.9: Implementation of the polymorphic type resolution for binary expressions.

Type Coercion To guarantee mathematical safety and prevent precision loss in the generated C++ code, the API implements a deterministic type coercion policy.

When evaluating arithmetic expressions, the resolver invokes the `most_general_number` utility function. This method evaluates an internal priority hierarchy among numeric types, where floating-point structures naturally take precedence over integers. The function automatically promotes the resulting expression to the wider type, ensuring that the backend code generator correctly casts variables and averts runtime truncation faults.

```
    _NUMERIC_PRIORITY = {
        DataTypes.INT: 1,
        DataTypes.BIGINT: 2,
        DataTypes.UBIGINT: 2,
        DataTypes.FLOAT: 3,
        DataTypes.DOUBLE: 4,
    }

```

```
    @staticmethod
    def most_general_number(
        type1: TypeDescriptor,
        type2: TypeDescriptor
    ) -> DataTypes:
        if (not isinstance(type1, DataTypes)) or
            (not isinstance(type2, DataTypes)):
             raise TypeError(f"....")

        if (not type1.is_number) or (not type2.is_number):
            raise TypeError(f"....")

        #returns the one with highest priority
        return max(type1, type2, key=lambda t: _NUMERIC_PRIORITY[t])
```

Listing 3.10: Implementation of the type coercion logic. The framework resolves the widest data type using a priority dictionary to avert runtime C++ truncation faults.

Ultimately, this rigorous static analysis enforces the *Fail-Fast* principle. If an invalid operationsuch as adding an integer to a boolean flagis attempted, the API intercepts the semantic error immediately during the Python drafting phase, providing clear stack traces before any JSON serialization or C++ compilation is triggered.

3.4 Query Composition and the Draft Abstraction

The core developer experience of WindFlow TIDE revolves around fluent method chaining from the `Table` class. However, executing a sequence of transformations on a continuous stream requires carefully managing the logical state of the query before it is submitted to the C++ backend. To achieve this without mutating the original `Table` objects, the framework delegates query construction to a transient accumulator object: the `Draft`.

When a developer invokes a non-terminal transformation on a `Table` (such as filtering or deduplicating), the API transparently spawns a `Draft` object. Rather than constructing a physical topology immediately, this object acts as a mutable builder for the *logical plan* DAG. It sequentially appends *API Operators* (e.g., `WhereOp`, `DistinctOp`) into a temporary branch of the logical plan. The draft remains open and mutable until a terminal projection, specifically the `select()` method, is invoked. This invocation finalizes the topology, statically evaluates the newly derived schema, and returns a new immutable `Query` object (which inherits from `Table`), ready for further composition or execution dispatch.

With this approach, each `Table` and `Query` object remains strictly immutable, as only the ephemeral `Draft` tracks the intermediate topological mutations during the query formulation.

3.4.1 Projections, Filters and Deduplication

To illustrate the query composition process, the following section relies on a practical running example: extracting all unique temperature readings below 15°C and converting them to Kelvin.

Assuming the environment setup described in section 3.2, the first step is to filter out each temperature above 15 degrees Celsius, this is achieved by applying a `where` operator with a boolean predicate.

```
    cond = col("temperature") < 15

    my_query = (tab
        .where(cond)
        #.where(col("temperature") < 15)
    )
```

Listing 3.11: Using a where with a predicate

The `where` method applies a filter based on an expression that evaluates to a boolean on the query's current schema.

Then we can convert each temperature to a Kelvin by using a `select` method. The `select` applies projections removing attributes or calculating new ones based on an expression.

```
    cond = col("temperature") < 15

    to_kelvin = col("temperature") + 257.15

    my_query = (tab
        .where(cond)
        .select( #all other fields are lost
            to_kelvin.alias("k_temperature")    #aliasing
        )
    )
```

Listing 3.12: Using a select to apply an aliased expression

An important aspect of expressions used in a `select` is the `alias` method, which returns a shallow copy of the expression with a different name. The new name will be attributed to a column in the new schema post-selection, thus it is highly recommended to assign aliases to avoid the generation of arbitrary default identifiers.

Since it is common to change a Table schema, a specific set of operations has been provided to avoid having to use `select` every time.

```
    #renames a set of columns with a dictionary
    renamed_sensors = tab.rename_columns(
        {"sensor_id": "sensor_name"}
    )

    #keeps old columns and adds new ones from expressions
    extended_sensors = tab.add_columns(
        (col("humidity") + col("temperature")).alias("t_plus_h")
    )

    #removes the provided columns
    dropped_sensors = tab.drop_columns(
        "temperature", "humidity"
    )
```

All these methods are implemented with a `select` so they can be used during query composition as well.

Now we only need to deduplicate all the converted temperatures with a `distinct`.

```
    cond = col("temperature") < 15

    to_kelvin = col("temperature") + 257.15

    my_query = (tab
        .where(cond)
        .select( #all other fields are lost
            to_kelvin.alias("k_temperature")    #aliasing
        )
        .distinct()
    )
```

Listing 3.14: Using a distinct to deduplicate

The `distinct` deduplicates each tuple based on all of its fields meaning that if we want to deduplicate only by "temperature" we need to drop all other fields beforehand.

```
    cond = col("temperature") < 15

    to_kelvin = col("temperature") + 257.15

    my_query = (tab
        .where(cond)
        .distinct() #on the whole schema
        .select(
```

```
            to_kelvin.alias("k_temperature")     #aliasing
        )
    )
```

Listing 3.15: A distinct that considers more fields than wanted including the time-column

A query like the one in Listing 3.15 would result in a deduplication on the whole schema. Another critical factor in assuring a correct deduplication is the presence of the time-column in event time. If the timestamp is considered during deduplication, all tuples will result unique unless a specific event is recorded more than once in the same instant. Because of this, the time-column should always be dropped before deduplicating.

For ease of use (and more reasons discussed in the next section) the API offers a shortcut to automatically insert a `distinct` after a `select`.

```
    cond = col("temperature") < 15

    to_kelvin = col("temperature") + 257.15

    my_query = (tab
        .where(cond)
        .select(
            to_kelvin.alias("k_temperature"),
            distinct=True        #keyword only
        )
    )
```

Listing 3.16: Another way to deduplicate after selection, by doing it directly in the select invocation.

State Retention and Memory Constraints Unlike stateless filters, the `distinct()` operator evaluates incoming tuples against a stateful registry of previously observed records. In the current version of TIDE, this operator acts globally over the continuous stream and does not enforce a Time-To-Live (TTL) or automatic state eviction policy.

Consequently, applying tuple-level deduplication over an unbounded, high-throughput stream requires caution: because every unique payload is perpetually retained, the internal memory footprint will grow monotonically. To prevent out-of-memory faults, developers should ensure that global deduplication is applied exclusively to streams with a strictly bounded cardinality, or opt to confine stateful operators within finite temporal windows.

3.4.2 Query Objects and Table References

While the previous section illustrated the logical sequence of operations to reshape a stream, it is equally important to understand how these transformations are logically linked and registered within the environment's catalog.

By default, the framework automatically generates sequential identifiers for newly derived queries. However, for debugging purposes and to maintain a readable execution graph, developers can explicitly assign a deterministic identifier to the resulting query using the `name_query()` method before finalizing the Draft.

Revisiting the running example, the complete formulation taking advantage of explicit naming and the integrated deduplication flag is as follows:

```
    cond = col("temperature") < 15
    to_kelvin = col("temperature") + 257.15

    my_query = (tab
        #user defined query_id
        .name_query("my_query")
        .where(cond)
        .select(
            to_kelvin.alias("k_temperature"),
            distinct=True
        )
    )
```

Listing 3.17: The complete running example featuring explicit query naming and an integrated distinct flag to prevent intermediate topology generation.

A crucial design detail in this snippet is the placement of the distinct evaluation. Because the `select()` method is a terminal operator that finalizes the current query, invoking a standalone `.distinct()` *after* a projection (like in Listing 3.14) would inadvertently force the API to spawn an entirely new intermediate `Query` object solely for the deduplication step. By embedding the `distinct=True` flag directly within the `select()` arguments, the deduplication is collapsed into the same topological branch, keeping the environment's catalog clean and the overall logical plan DAG strictly optimized.

To fully grasp how this fluent chain translates into a verifiable data structure, Figure 3.1 illustrates the three phases of the Draft lifecycle.

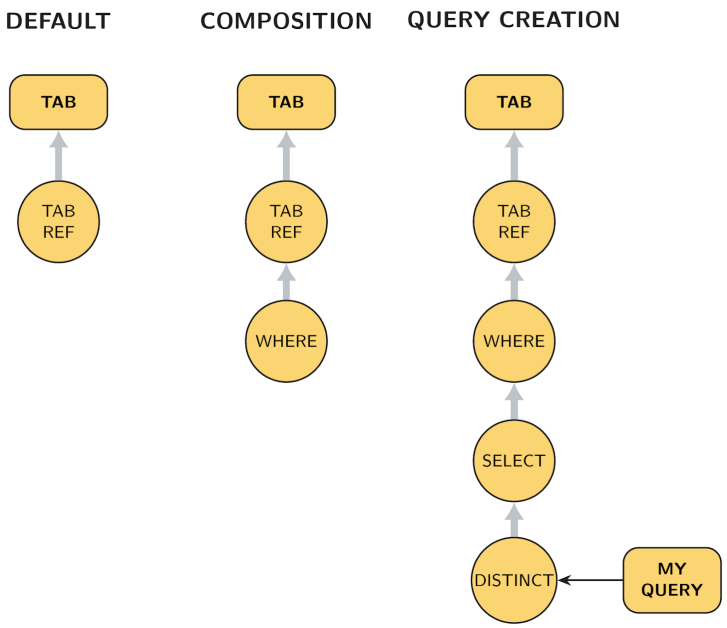


Figure 3.1: Topological evolution of the Draft during query composition. The final Query object encapsulates the root operator of the chain, while the initial `TableRefOp` symbolically links to the parent table.

The query composition mechanism relies on a bottom-up accumulation strategy:

1. **Default (Initialization):** As soon as a non-terminal method is invoked (or when the Draft is explicitly named), the framework automatically injects a `TableRefOp` at the base of the new DAG branch. This operator acts as a symbolic reference to the parent table.

2. **Composition:** Intermediate unary operators (such as `WhereOp`) are appended to the Draft. Each new operator stores a topological pointer to its predecessor, effectively forming a linked list of transformations.

3. **Query Creation:** The terminal method finalizes the chain. The API registers a new immutable `Query` object in the environment (named `"my_query"` in our example). This object does not store the data itself, but rather encapsulates a pointer to the root operator of its specific sub-DAG.

Late Binding and Subqueries The architectural cornerstone of this mechanism is the `TableRefOp`. Rather than storing hard memory pointers to the parent Table or immediately resolving the physical data source, a `TableRefOp` operates through *Late Binding*. It strictly records the textual `table_id` of its parent Table.

This design unlocks a powerful paradigm: because `Query` objects inherit from the base `Table` class and are registered in the environment via their textual IDs, they can seamlessly act as parent sources for downstream operations. When a new query is drafted on top of an existing one, the new `TableRefOp` simply points to the ID of the intermediate `Query`.

```
    #assuming "my_query" is the query built in the running example

    #composing a second query object from a different source table
    other_q = (other_tab
        .name_query("other_query")
        .where(...)
        .group_by(...)
        .select(...)
    )

    #nesting the two queries to perform a join
    my_joined_query = (my_query
        .name_query("my_joined_query")
        .join(other_q, ...)
        .select(...)
    )
```

Listing 3.18: Abstract example of reusing previously defined queries as subqueries.

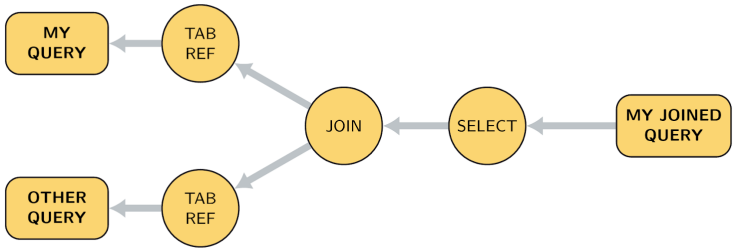


Figure 3.2: The `Draft` of my_joined_query from 3.18 pointing to the previously defined Queries with a `TabRef`.

Consequently, developers can modularize complex pipelines by creating reusable subqueries. The entire logical plan remains a lightweight, decoupled set of linked lists that the C++ code generator will recursively traverse and resolve only at execution time. During this traversal, the framework translates `TableRefOp` nodes either into physical file parsers (if they point to a `Source`) or into direct topological edges (if they point to another subquery).

3.5 Windows and Intervals

As established in the theoretical background (Chapter 1), computing stateful metrics over continuous, unbounded streams requires discretizing the dataflow into finite chunks. Without formal boundaries, aggregations would consume infinite memory, and relational joins would never emit matched tuples.

To model these boundaries ergonomically, TIDE provides two dedicated configuration objects: `Window` and `Interval`. Both abstractions heavily rely on the `Duration` value objects introduced in Section 3.2.3, effectively insulating the user from low-level microsecond arithmetic.

3.5.1 Window Specifications

A `Window` object dictates how an unbounded stream should be partitioned into discrete buckets before applying an aggregation. The API natively supports both time-driven and data-driven (count-based) discretization strategies, which can be instantiated through intuitive static factory methods.

- **Tumbling Windows:** These assign events to fixed, non-overlapping buckets of a specific size. Every event belongs to exactly one window.
- **Sliding Windows:** These assign events to overlapping buckets defined by a *size* and a *slide* step. Because the slide is typically smaller than the size, a single event can contribute to multiple concurrent windows.

Internally, the `Window` class tracks its `WindowType` (`TIME` or `COUNT`) and normalizes all size and slide parameters into raw integers. This unified representation allows the Python frontend to serialize window configurations transparently, leaving the C++ code generator to synthesize the appropriate `KeyedWindows` or `WindowJoin` physical operator during compilation.

```
    # A non-overlapping time window of 1 hour
    w_tumble_time = Window.createTBWindow(Duration.hours(1))

    # An overlapping time window (1 hour wide, sliding every 15
    minutes)
    w_slide_time = Window.createTBWindow(
        Duration.hours(1),
        Duration.minutes(15)
    )

    # A count window with size = 1000 and implicit slide = size
    w_tumble_count = Window.createCBWindow(1000)
```

Listing 3.19: Instantiating different window strategies using Duration objects and integer counts.

When these objects are eventually passed to a `group_by()` or `join()` operator, the frontend validates them against the `TableEnvironment`'s temporal policy, enforcing a *Fail-Fast* rejection if a time-based window is used under `TimePolicy.NO_POLICY`.

3.5.2 Relative Intervals

While windows partition a single stream into absolute buckets, correlating events across multiple streams often requires a relative approach. The `Interval` abstraction defines a relative temporal envelope around a specific event.

It is constructed using two `Duration` objects, a lower bound and an upper bound, which represent the tolerated temporal distance between matching tuples. Mathematically, for an event occurring at timestamp t, the interval defines a validity range of $[t + \Delta_{\text{lower}}, t + \Delta_{\text{upper}}]$.

```
    # Defines an envelope: from 5 minutes in the past to 2 minutes
    in the future
    join_interval = Interval(
        lower_bound=Duration.minutes(-5),
        upper_bound=Duration.minutes(2)
    )

    #both can be negative or both can be positive but
    # lower_bound has to be < upper_bound
```

Listing 3.20: Instantiating a relative temporal interval for cross-stream correlations.

This object serves as the core configuration for Interval Joins (discussed in Section 3.7.1), enabling the underlying engine to safely buffer events and automatically evict stale tuples once the temporal boundaries have been crossed.

3.6 GroupBy and Stateful Aggregations

Relational grouping constitutes the cornerstone of stateful dataflow analysis. To ensure strict semantic consistency, TIDE enforces a structural transformation on the dataset whenever a grouping operator is applied. Because data is partitioned, raw non-key attributes can no longer be evaluated individually and are actively discarded from the intermediate schema. Consequently, the output is strictly composed of the grouping keys and the newly computed aggregate metrics.

Rather than attempting to compute complex formulas and manage state within a single monolithic operator, the compilation phase decomposes the workload into two distinct physical phases:

1. **Accumulation (`GroupByOp`):** A stateful operator dedicated exclusively to maintaining the required grouping keys, managing window lifecycles, and incrementally updating primitive accumulators (e.g., `SUM`, `COUNT`).

2. **Finalization (`SelectOp`):** A stateless projection operator placed immediately downstream, responsible for evaluating algebraic formulas and applying final aliases.

```
    q = (tab
    #sets up keys and windows if present
    #drops all non key-fields that can only be used inside
   aggregations
        .group_by("sensor_id")
        #declares aggregations with a helper function
        .select(
            "sensor_id",
            avg("temperature")
        )
    )
```

Listing 3.21: Example of group_by used to calculate the average temperature per sensor

Because the aggregation expressions defined in the `select` method must be strictly passed down to configure the underlying `GroupByOp`, the API enforces a rigid topological contract: no intermediate operators can be injected between the accumulation and finalization phases.

To emulate constructs such as the `HAVING` SQL clause, developers can filter the aggregated results by simply chaining a standard `where()` operator after the finalizing `select()`. This simply spawns a new logical `Query` branch in the DAG, maintaining the strict stateless nature of the filtering phase.

```
    #number of readings by each sensor
    grouped_q = (tab
        .group_by("sensor_id")
        .select("sensor_id", count().alias("reading_num"))
    )

    #showing only the sensors with 100+ readings
    having_q = (grouped_q
        .where(col("reading_num") > 100)
        .select("sensor_id")
```

```
    )
```

Listing 3.22: Emulating a HAVING clause by filtering after aggregating.

3.6.1 Keys, Windows, and Global Aggregations

The topological behavior of the `GroupByOp` is fundamentally dictated by the presence of a temporal `Window` and the cardinality of its keys.

When a `Window` object is passed to the `group_by()` method, the operator acts as a discrete buffering mechanism: it accumulates state over time and only emits the finalized aggregates when the window boundaries are crossed (typically triggered by advancing watermarks). Conversely, if no window is provided, the operator performs a continuous, *global* aggregation. In this mode, the framework emits a continuous *changelog* stream, producing an updated aggregate tuple every time a new event modifies the internal state.

```
    time_w = Window.createTBWindow(Duration.hours(10))

    #keyed windowed aggregation with the keyword "window"
    kw = tab.group_by("sensor_id", window=time_w).select(...)

    #keyed global aggregation
    kg = tab.group_by("sensor_id").select(...)

    #unkeyed window aggregation
    uw = tab.group_by(window=time_w).select(...)

    #implicit unkeyed global aggregation
    ug = tab.select(avg("temperature"))
```

Listing 3.23: Comparison of keyed, unkeyed, windowed, and global grouping semantics.

Furthermore, the framework supports *Unkeyed Aggregations*, queries where metrics are computed over the entire stream without partitioning it by a specific attribute. In TIDE, invoking a `select()` containing aggregate functions without a prior `group_by()` implicitly generates a `GroupByOp` with an empty key list (`keys=[]`).

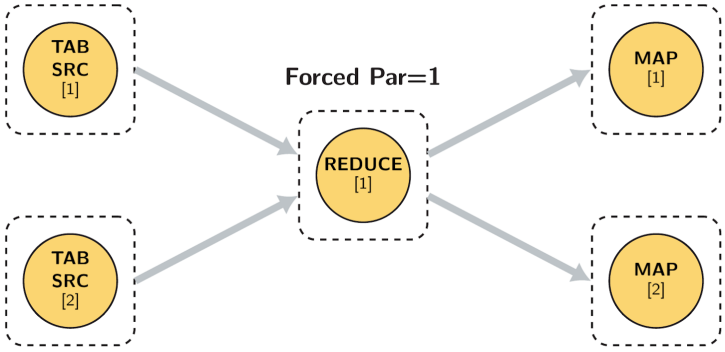


Figure 3.3: Physical plan view of an unkeyed-global-aggregation with forced parallelism to one.

While logically sound, unkeyed aggregations inherently force the underlying C++ execution graph to route all data to a single worker thread (`parallelism = 1`). As depicted in Figure 3.3, this creates a severe throughput bottleneck, but it is strictly necessary: distributing the state without a partitioning key would violate the correctness of the metric.

3.6.2 Aggregation Dependencies and Static Validation

The decoupling of accumulation and finalization requires a sophisticated translation mechanism during the Draft composition. When a developer invokes a `select()` following a grouping phase, the API executes a rigorous semantic check, leveraging the `validate_grouped()` method implemented by every `Expression` subclass.

This recursive function traverses the expression tree checking every `ColRefExpression` to verify that it corresponds strictly to an established grouping key. If a raw attribute is referenced outside of an aggregate function, the API enforces the *Fail-Fast* principle, raising an immediate `ValueError` before any JSON generation occurs.

```
    # 1. Invalid Query: Attempting to select a non-key raw column
    try:
        invalid_q = tab.group_by("sensor_id").select(
            col("sensor_id"),
            avg("temperature"),
            col("humidity")  # Raises ValueError: Not a grouping key!
        )
    except ValueError as e:
        print("Semantic Error Intercepted.")

    # 2. Valid Query: Grouping, Aliasing, and Post-Filtering
```

```
    valid_q = (tab
        .group_by("sensor_id", window=Window.createCBWindow(100))
        .select(
            col("sensor_id"),
            avg("temperature").alias("avg_temp")
        )
    )
```

Listing 3.24: Example of semantic validation during query drafting. The framework correctly intercepts invalid column references.

Once the projection is validated, the framework must bridge the gap between the declarative user expressions and the stateful C++ accumulators. This is achieved through dependency extraction and AST rewriting.

During drafting, the API invokes the `aggregation_dependencies()` method on the provided expressions. This recursively extracts all the required stateful functions (e.g., `AVG(temperature)`) from the tree. These requirements are injected into the intermediate schema of the `GroupByOp` using deterministic canonical names (e.g., `AVG_temperature`), intentionally stripping away any user-defined aliases to prevent state duplication and ensure a reliable mapping contract.

Subsequently, the `rewrite_grouped()` method recursively traverses the original AST and adapts it for the finalization phase. It replaces every `AggregateExpression` with a `ColRefExpression`, explicitly pointing to the canonical name emitted by the `GroupByOp`.

Because this rewriting happens recursively, developers can seamlessly embed aggregations within complex mathematical formulas (e.g., `(avg("temperature") * 1.8) + 32`). The `GroupByOp` strictly handles the stateful average evaluation, while the downstream `SelectOp` receives the pre-calculated `AVG_temperature` and purely executes the remaining scalar arithmetic, preserving any user-defined aliases for the final output schema.

3.7 Multi-Stream Operators

While unary operators and aggregations manipulate a single dataflow, complex streaming analytics often require correlating events from multiple independent sources. To facilitate this, TIDE exposes binary operators that accept another `Table` (or derived `Query`) instance as an input argument.

Because the API relies on a decoupled `Draft` state mechanism, developers can independently append operators to the logical branches of both the left and right tables before invoking a binary operation. This guarantees that streams are properly filtered and projected prior to physical unification, effectively minimizing the state

overhead.

```
    # Preparing the left branch (filtering and renaming)
    active_sensors = (sensor_tab
        .where(col("status") == "ACTIVE")
        .select(
            "sensor_id",
            "temperature"
        )
    )

    # Preparing the right branch (projection)
    metadata = (meta_tab
        .select(
            "sensor_id", "location", "model", "max_operating_temp"
        )

    # The two branches are now ready to be merged
```

Listing 3.25: Example of independently preparing two logical branches before applying a binary multi-stream operator.

This is useful as in the current state of the API there is no *Predicate Push-down* optimizations, the developers need to consciously put filters beforehand.

3.7.1 Relational Joins

Joining unbounded streams requires strict temporal boundaries to prevent infinite state accumulation. The `join()` method natively supports both `Window` and `Interval` attachments to constrain the evaluation scope.

- **Window Joins:** These discretize the continuous streams into distinct temporal buckets (e.g., Tumbling or Sliding windows) before joining. Records from the left and right branches are correlated exclusively if they fall into the exact same discrete time window.
- **Interval Joins:** These correlate each record on the left stream with any record on the right stream that arrives within a relative, sliding time envelope. Because this correlation timeframe is finite, the underlying engine safely evicts stale historical records as soon as the event-time watermark surpasses their upper temporal bound.

To guarantee deterministic output schemas and avert ambiguous column references during C++ code generation, the API enforces a strict schema constraint during the drafting phase: the specified join keys must be the only attributes sharing

the same name between the left and right schemas. Any other overlapping column names must be explicitly renamed or dropped via a `select()` projection prior to the join.

```
    # 1. Window Join: Matches tuples falling in the same exact 1-
    hour tumbling window
    windowed_join = active_sensors.join(
        other=metadata,
        on=["sensor_id"],    #join key/keys
        attachment=Window.createTBWindow(
            Duration.hours(1)
        )
    ).select(...)

    # 2. Interval Join: Matches tuples within a relative time
    envelope (-5m to +5m)
    interval_join = active_sensors.join(
        other=metadata,
        on=["sensor_id"],
        attachment=Interval(
            Duration.minutes(-5),
            Duration.minutes(5)
        )
    ).select(...)
```

Listing 3.26: Applying temporal boundaries to relational joins using Windows and Intervals.

Internal Predicates and Theta-Joins Beyond enforcing schema constraints, the `join()` method supports arbitrary boolean expressions through the `where` parameter. This allows developers to express complex *Theta-Joins*, such as comparing numerical values across the two streams without requiring a subsequent, independent filter operator.

From a topological standpoint, pushing the predicate directly into the join configuration yields significant performance benefits. During code generation, the condition is compiled directly into the C++ evaluation of the join operator. If the cross-stream predicate is not satisfied, the internal lambda immediately returns a `std::nullopt`. This guarantees that invalid tuple combinations are discarded in-place before any memory allocation or network emission occurs on the underlying FastFlow queues, resulting in zero threading overhead for unmatched records.

```
    # Cross-evaluating a metric (left) against a metadata threshold
    (right)
    join_cond = col("temperature") > col("max_operating_temp")

    interval_join = active_sensors.join(
```

```
        other=metadata,
        on=["sensor_id"],
        attachment=Interval(
            Duration.minutes(-5),
            Duration.minutes(5)
        ),
        where=join_cond      # Evaluated inside the C++ Join lambda
    ).select(...)
```

Listing 3.27: Applying a Theta-Join condition directly inside the operator to discard invalid matches before network emission.

Keyless Joins and DPMode If a join is invoked without specifying partitioning keys, the framework implicitly executes a Cross Join within the defined temporal boundaries. Unlike keyless aggregations that force a single-thread bottleneck, keyless joins maintain the configured topological concurrency by leveraging WindFlow's Data-Parallel Mode (`DPMode`). In this mode, the engine performs a *Symmetric Broadcast-Store* execution: it duplicates and broadcasts all incoming tuples from both the left and right streams to every parallel join replica.

While this prevents a single-thread bottleneck, `DPMode` scales poorly. The scalability is severely degraded by communication and synchronization overheads: broadcasting all data across the internal queues to every worker thread consumes massive bandwidth.

Consequently, while developers could theoretically leverage a keyless join with a filter to emulate a *Theta Join*, thereby bypassing the strict column-renaming constraints of the standard equi-join, this pattern is strongly discouraged due to its computational inefficiency.

3.7.2 Set Operators

Set operations merge divergent dataflows into a single unified stream. Unlike joins, which combine attributes horizontally, set operators append tuples vertically. Consequently, they require the left and right input schemas to be strictly identical in both field names and data types.

The API provides four relational set methods, differing fundamentally in their physical execution semantics:

- **`union_all()`:** A purely topological, stateless multiplexing operation. It seamlessly splices the upstream channels together with zero memory overhead, making it the most efficient set operator.

- **`union()`:** Performs a topological merge followed by a stateful deduplication stage to eliminate identical overlapping tuples.
- **`intersect_all()`:** A stateful, multi-set operation. It retains tuples that appear in both streams, respecting multiset multiplicity. The underlying engine injects tagging operators to mark stream provenance before securely matching records across the unified channel.
- **`intersect()`:** Similar to the multiset intersection, but enforces a strict deduplication on the resulting matched tuples.

```
warmer_q = (tab
    .where(col("temperature") > 20)
    .select("sensor_id", "temperature")
)

colder_q = (tab
    .where(col("temperature") < 25)
    .select("sensor_id", "temperature")
)

#merge the streams with duplicates in the 20<t<25 range
u_all = (warmer_q
    .union_all(colder_q)
    .select("sensor_id", "temperature")
)

#only the range between 20 and 25
intersection = (warmer_q
    .intersect(colder_q)
    .select("sensor_id", "temperature")
)

#deduplicating union
union = (warmer_q
    .union(colder_q)
    .select("sensor_id", "temperature")
)
```

Listing 3.28: Example of use of multiple set operations.

State Retention Warnings Much like the standalone `distinct()` operator, the stateful variants of set operations (`union`, `intersect`, `intersect_all`) evaluate incoming tuples against an ever-growing internal registry. Because the current version

of TIDE does not implement automatic state eviction or Time-To-Live (TTL) policies for set operators, invoking them on unbounded continuous streams will cause the memory footprint to grow monotonically. Developers should restrict these operations to bounded datasets or rely exclusively on `union_all()` for infinite dataflows to prevent out-of-memory faults.

3.8 Query Plan Serialization and Execution

To transition from a purely declarative Python Draft to a physical streaming pipeline, the logical plan must be serialized and passed to the compilation backend. This critical handover is orchestrated by the `TableEnvironment` through its primary entry point: the `execute()` method.

When a developer finalizes a stream topology, they invoke this method to trigger the entire Code-Generation and Compilation-Execution process.

```
    # Triggers the serialization and execution of the provided query
    env.execute(
        query=my_query,
        output_dir="./my_output",
        sync=False
    )
```

Listing 3.29: Invoking the execution of a finalized Query object through the TableEnvironment, triggering AST serialization and C++ compilation.

The `execute` method accepts several configuration parameters to control the dispatch lifecycle:

- `query`: The finalized target `Query` object representing the root of the desired dataflow.
- `output_dir`: The target filesystem directory where the environment will dump the serialized JSON artifacts and the generated code.
- `sync`: A boolean flag determining the execution mode. If `True`, the Python thread will block and wait for the C++ physical job to terminate; otherwise, it returns a `JobHandle` for asynchronous monitoring.

Once the parameters are processed, the method initiates the final validation of the DAG, serializes the tree into a JSON format, and invokes the C++ code-generator subsystem, ultimately launching the native streaming engine.

3.8.1 Query Dependencies and FromOp Resolution

Complex streaming applications often reuse the same intermediate queries across multiple downstream branches (e.g., a diamond-shaped DAG). If the `TableEnvironment` were to serialize the target query naively by recursively expanding every node, the resulting JSON would suffer from massive duplication, drastically increasing the nesting depth and reducing readability.

To prevent this, the API relies on a modular serialization strategy. Before generating any JSON, the environment executes a `_collect_referenced_queries` routine. This method traverses the logical DAG bottom-up, searching for `TableRefOp` nodes. Instead of expanding the graph into a monolithic structure, the environment collects all the required intermediate queries and serializes each one into its own distinct JSON file (named after its respective `query_id`). Consequently, the final serialized logical plan remains shallow and strictly references other query files by their identifiers.

Throughout the query composition, source tables are represented as `TableRefOp` nodes which only know the `table_id`. During serialization the physical I/O configurations must be injected into the AST. When the serializer encounters a `TableRefOp`, it evaluates its target identifier against the environment's catalog:

- If the identifier corresponds to a derived `Query`, the `TableRefOp` is preserved as a symbolic pointer to the corresponding JSON file.

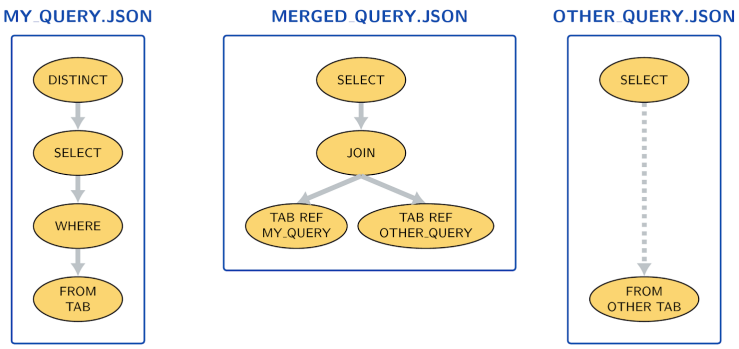


Figure 3.4: Serialized view of the proposed query in Figure 3.2

- If the identifier corresponds to a registered source table, the environment seamlessly replaces the `TableRefOp` with a `FromOp` which is given all the information of the Table's `InputFileConfiguration`.

The `FromOp` acts as the definitive physical leaf of the AST. It encapsulates the `InputFileConfiguration` object provided during the initial table registration, embedding all low-level I/O metadata (such as the file path, ingestion delays, byte-split

sizes, and the designated `time_col`) directly into the JSON. This guarantees that the downstream C++ code generator has all the precise physical details required to instantiate the parallel source threads, while keeping the Python drafting phase entirely agnostic to physical I/O constraints.

Chapter 4

Code Generation and Operations

As illustrated in the previous chapters, the `windflow_tide` framework relies on a strict separation of concerns between the logical definition of a query and its physical execution. While Chapter 3 focused on the `api` submodule and how a user can declaratively formulate pipelines up to the production of the logical plan, serialized in JSON format, this chapter explores the subsequent phase of the program's lifecycle: the translation of the logical plan into a high-performance application.

The `codegen` module bears the delicate task of parsing the generated JSON file, reconstructing the operation tree in memory, and synthesizing the C++ source code necessary for execution. This process leverages specialized constructors (*Table Operations*) that encapsulate the complexity of WindFlow's native builders, allowing relational logic to be injected directly as lambda functions into the C++ code.

To maintain a strict separation of concerns there are four specialized, stateless translation services separated from the parsing and optimization logic:

1. **`SchemaGenerator:`** Resolves logical types into deduplicated native C++ `structs`.

2. **`ExpressionTranslator:`** Converts logical AST dictionaries into C++ expression strings.

3. **`LambdaGenerator:`** Packages expressions into strictly typed `std::function` closures.

4. **`BuilderGenerator:`** Utilizes the generated lambdas and configuration parameters to return the finalized C++ builder declarations (such as the one in Listing 4.2).

Finally, the runtime module automates the compilation and execution process of the generated code, rendering the interaction with the native toolchain completely transparent to the end-user.

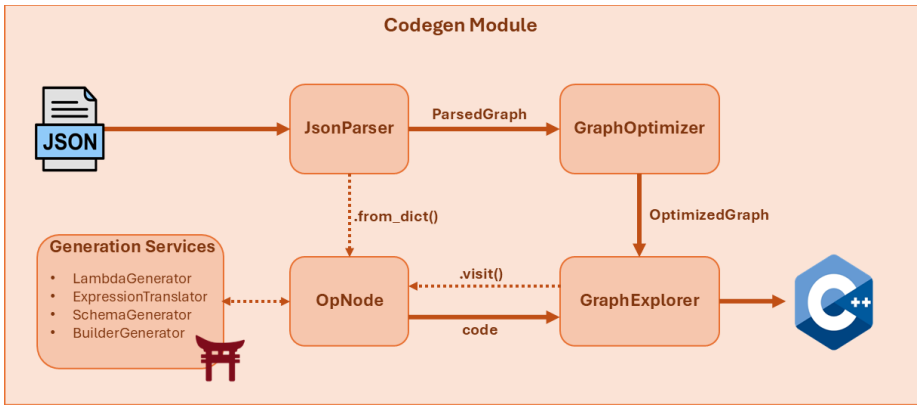


Figure 4.1: Codegen Module Architecture

The objective of this chapter is not to provide a line-by-line analysis of each operator's implementation, but rather to illustrate the architectural role of each component and the engineering solutions adopted to efficiently and safely bridge the conceptual gap between the Python abstraction and the underlying stream processing infrastructure.

4.1 Parsing the Json into the Operations Tree

The first responsibility of the `codegen` subsystem is to bridge the gap between the language-agnostic JSON artifacts produced by the frontend and the C++ generation engine. This transition is orchestrated by the `JsonParser`, which reads the serialized Abstract Syntax Tree (AST) from the filesystem and reconstructs it into a strongly typed, in-memory graph known as the *Operations Tree*.

Instead of manipulating raw, unvalidated Python dictionaries, the parser instantiates a hierarchy of intermediate representation (IR) objects. This object-oriented approach guarantees that the downstream code generators can interact with a uniform interface, confidently relying on pre-validated topological connections (parents and children) and explicitly defined output schemas.

4.1.1 Operation Tree and the Factory Pattern

The foundation of the parsed graph is the `OpNode` abstract base class. Every node in the reconstructed graph inherits from this base class, which centralizes the essential topological metadata: the unique `node_id`, the operational `op_type`, the output schema, and the bidirectional pointers to the node's `parents` and `children`.

To minimize code duplication and streamline the parsing logic, the hierarchy introduces intermediate abstract classes based on the operational arity and physical characteristics, such as `UnaryNode`, `BinaryNode`, and the mixin `WindowNode`.

These intermediate classes implement common extraction methods to parse shared attributes (like the dual input schemas for a binary join, or the size and slide parameters for a window) before passing the control to the concrete implementation.

The actual instantiation of these concrete nodes (e.g., `WhereOpNode`, `UnionOpNode`) is governed by the `OpNodeFactory`. This component implements the *Factory Method* design pattern: it maintains an internal registry mapping each `OpType` enumeration to its corresponding target class.

When the `JsonParser` traverses the raw JSON dictionaries, it delegates the instantiation of the intermediate objects to the `OpNodeFactory`. This approach completely decouples the JSON deserialization process which solely focuses on reconstructing the topological links from the internal semantic rules of individual operators. The synthesis of the C++ logic is deliberately deferred to a subsequent phase, where a dedicated *GraphExplorer* interacts with the fully instantiated nodes by invoking their respective `visit()` methods.

4.1.2 Linking Table References and Subquery Caching

As discussed in Section 3.8.1, the Python API serializes reusable subqueries into distinct JSON files, referencing them via `TABLE_REF` (or `TableRefOp`) nodes. The `JsonParser` must resolve these late-binding references during the operations graph reconstruction.

When the parser's recursive traversal encounters a `TABLE_REF` node, it extracts the target `source_id` and suspends the current query parsing to load the corresponding subquery JSON file. However, naively parsing the subquery every time it is referenced would inadvertently duplicate the logical branch.

```
#original query originating from the source table
base_q = tab.group_by(...).select(...)

#first branch derived from base_q
first_split = base_q.where(cond1).select(...)

#second branch derived from base_q
second_split = base_q.where(cond2).select(...)

#final query merging the results
merged_q = first_split.join(second_split, ...).select(...)
```

Listing 4.1: Abstract example of using a subquery multiple times in the same overall query

To prevent this, the `JsonParser` implements a strict caching mechanism. It maintains a `loaded_queries` set and a registry of `root_nodes`. When a `TABLE_REF` is evaluated, the parser first checks if the requested `source_id` has already been

processed. If it has, the parser simply retrieves the existing root `OpNode` from the registry and links it as a parent to the current operator, completely avoiding the duplication of the sub-graph. This ensures that, regardless of how many times a subquery is reused in the declarative frontend, it is represented exactly once in the memory graph.

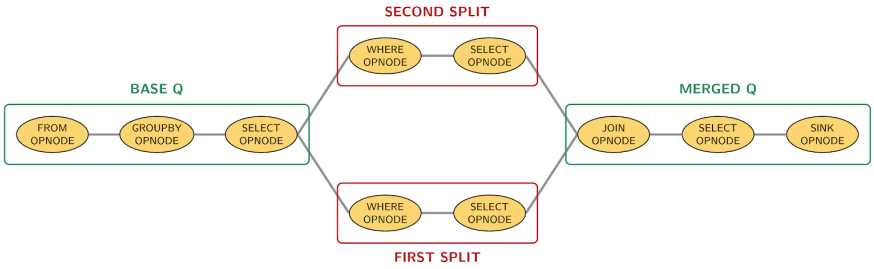


Figure 4.2: Logical Operation Tree derived from Listing 4.1. At the logic level, the `base_q` subgraph is represented exactly once.

The two divergent substreams depicted in Figure 4.2, originating from `first_split` and `second_split`, will eventually be merged using WindFlow's native pipeline merging mechanisms (Section 1.6.1).

However, while the logical graph remains safely deduplicated in Python, the operations within the `base_q` subgraph are currently duplicated at the physical level. This design choice was driven by the compiler's stateless post-order traversal, which guarantees topological correctness (Section 4.4) but inherently lacks a memorization mechanism to keep state information from previously visited nodes.

Although this redundancy could be natively avoided by introducing a stateful compilation context and leveraging WindFlow's built-in `multipipe` splitting capabilities, prioritizing the compiler's structural simplicity was deemed an acceptable trade-off for this first iteration.

This baseline approach introduces a noticeable computational and memory footprint: the source file is ingested and parsed multiple times, and downstream stateful operators (such as Windows and Joins) redundantly buffer identical tuples across disjoint memory spaces.

4.1.3 Post parsing Optimizations

Once the `JsonParser` has fully reconstructed and linked the `OpNode` graph, the framework exposes the graph to a post-parsing optimization phase. The goal of this phase is to analyze the topology and perform rule-based mutations to streamline the execution plan before any C++ code is generated.

The `GraphOptimizer` implements a fixed-point iteration loop. It applies a suite of optimization rules across the graph, utilizing linear snapshots and bitwise convergence checks to determine if the graph has ceased mutating. Currently, the primary rule implemented in the framework is the elimination of identity projections (*Identity Select Erasure*). If the optimizer detects a `SelectOpNode` that maps all input columns 1-to-1 without applying any transformations, renaming, or deduplication, it safely removes the node from the tree and rewires the parent and child pointers. This prevents the generation of useless C++ `Map` operators, reducing memory copies and minimizing thread context switches in the final WindFlow topology.

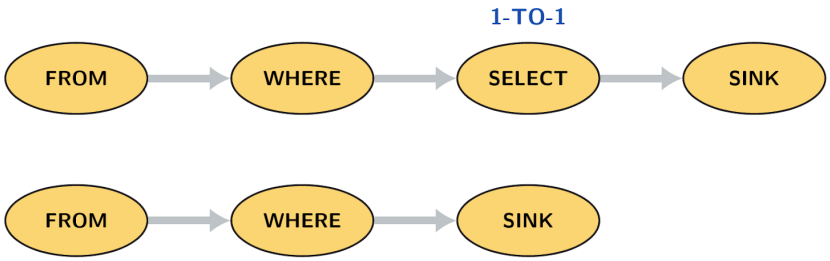


Figure 4.3: An example of *Identity Select Erasure*, the bottom graph is the result of the optimization.

While the current ruleset is intentionally lightweight, this architecture establishes a robust foundation for future extensions. The optimizer can be naturally expanded to support advanced relational heuristics, such as *Predicate Pushdown* (moving `WhereOpNode` filters as close to the `FromOpNode` as possible to reduce network traffic) or *Early Projection* (stripping unused columns prior to stateful operators to minimize memory consumption).

4.2 Operation Builders

Before detailing how the fully parsed Operations Tree is traversed and converted into a physical graph, it is essential to establish *how* the generated C++ code is structurally formulated. The `codegen` subsystem does not simply emit raw C++ code; rather, it synthesizes properly configured instances of WindFlow's builder objects.

As introduced in Section 2.2, the resulting program avoids exposing raw native WindFlow builders directly. Instead, it leverages specialized `Table Operations`: custom wrappers designed to instantiate physical WindFlow operators by securely encapsulating relational lambdas.

Python API Operator	Table Operation	Physical Operator	Notes
FromOp	Table_Source_Builder	Source	
.select(...)	Select_Builder	Map	
.where(...)	Where_Builder	Filter	
.group_by(window=w)	Windowed_Group_Builder	KeyedWindows	Forced parallelism to 1 if unkeyed.
.group_by(...)	Global_Group_Builder	Reduce	Forced parallelism to 1 if unkeyed.
.distinct()	Distinct_Builder	Filter (stateful)	No state cleaning.
.join(...)	Table_Window_Join_Builder or Table_Interval_Join_Builder	WindowJoin or IntervalJoin	If unkeyed supports parallelism with DPMode. Requires prior schema unification.
.union_all(...)	N/A	N/A	Purely topological MultiPipe merge.
.union(...)	N/A	N/A	UnionAll and a distinct.
.intersect(...)	Intersect_Builder	FlatMap(stateful)	Requires prior stream tagging. No state cleaning.
.intersect_all(...)	Intersect_All_Builder	FlatMap(stateful)	Requires prior stream tagging. No state cleaning.
N/A	Table_Sink_Builder	Sink	

Figure 4.4: Table detailing the what operator maps to each table-operation and physical operator.

For instance, a `SelectBuilder` yields a physical *Map* operator to apply user-defined projections, while a `GlobalGroupBuilder` instantiates a *Reduce* operator to maintain running aggregations.

```
    auto select_op = Select_Builder<SensorInput, SensorOutput>(
    //logic injected as a lambda
            [](const SensorInput& in) -> SensorOutput{
                SensorOutput out;
                out.sensor_id = in.sensor_id;
                out.temperature = in.temperature;
                return out;
            }
        )
        .withName("select_op")  // names the operator
        .withParallelism(2)     // sets intra-operator parallelism
```

```
12          .build();
```

Listing 4.2: A SelectBuilder returning a Map operator without the need for a dedicated functor. The projection drops all columns except temperature and sensor_id.

Each `Table Operation` requires distinct initialization parameters and exposes specialized configuration methods beyond the standard `withName` and `withParallelism`. For example, the `TableSourceBuilder`, responsible for instantiating the CSV parser, provides methods to distinguish between ingestion-time and event-time semantics, configure watermarking strategies for out-of-order streams, and define custom I/O split sizes.

While a line-by-line analysis of the internal C++ functors powering each `Table Operation` falls outside the scope of this work, the specific behavioral requirements and physical constraints of the most critical operators will be examined in the remainder of this chapter.

4.2.1 Logic Encapsulation in Lambdas

Native WindFlow processing stages mandate the definition of dedicated C++ `struct` or `class` functors, implemented by overloading the `operator()`, to encapsulate user logic. To bypass this boilerplate and enable a purely declarative frontend, the framework's `Table Operations` are designed to accept standard `std::function` closures. Translating the language-agnostic JSON representations into these compilable C++ lambdas is achieved through the coordinated effort of two specialized components: the `ExpressionTranslator` and the `LambdaGenerator`.

- `ExpressionTranslator`: It implements a recursive visit over the logical representation of `Expression` objects. When evaluating a composite operation, such as a `BINARY_OP`, the translator recursively traverses the nested left and right operands until it reaches the terminal leaves.

  It dynamically maps JSON logical operators to their native C++ counterparts (e.g., mapping logical `and` to `&&`) and rigorously formats `LITERAL` nodes. To guarantee type safety and prevent runtime faults, the translator applies strict C++ type suffixes, such as appending `f` to floating-point values, `LL` to big integers, or explicitly wrapping text in `std::string("...")`. Furthermore, every translated binary operation is automatically wrapped in parentheses, ensuring that the original mathematical and logical precedence of the Python expressions is flawlessly preserved in the generated code.

- `LambdaGenerator`: It consumes these translated strings and packages them into strongly typed anonymous functions. By leveraging the `SchemaGenerator`,

the framework accurately resolves the native struct's identifiers required for the lambda's input and output signatures. The `LambdaGenerator` then injects the types and the translated logic into specialized Jinja2 templates.

```
auto where_op = Where_Builder<SensorInput>(
        [](const SensorInput& in) -> bool {
            // col("temperature") > 30
            return (in.temperature > 30);
        }
    )
    .withName("hot_temperatures")
    .withParallelism(2)
    .build();
```

Listing 4.3: The LambdaGenerator automatically wraps the recursively translated boolean expression into a statically typed C++ closure, strictly bound to the deduced input struct.

This modular pipeline completely decouples the relational logic from the physical graph generation. For instance, a complex filtering condition crafted in the Python frontend is seamlessly lowered into an isolated, performant C++ closure, ready to be injected into the corresponding builder as seen in Listing 4.3.

4.2.2 Operation Peculiarities

While most relational operators (like `Map` or `Filter`) map 1-to-1 with their corresponding WindFlow builders, certain relational constructs present physical execution constraints that the native engine cannot resolve autonomously. To uphold the declarative abstraction of TIDE, the operation builders handle these edge cases by dynamically injecting auxiliary operators or restructuring the data types on the fly.

Union and Union All: TIDE does not rely on a dedicated C++ builder for set unions. The `UNION ALL` operation is purely topological: the code generator simply invokes WindFlow's `merge_multipipes` routine to splice the upstream branches together, incurring zero memory or processing overhead.

For a strict `UNION`, which requires stateful deduplication, the `UnionOpNode` automatically delegates the conflict resolution to the previously validated `DistinctOpNode` logic. This prevents code duplication within the compiler and leverages the existing hashing infrastructures to guarantee set semantics.

Schema Unification for Joins: WindFlow's native `Interval_Join` and `Window_Join` operators enforce a strict requirement: both the left and right incoming streams must share the exact same C++ `tuple_t` signature to manage the internal state buffers

correctly. Because relational joins frequently correlate heterogeneous schemas (e.g., matching a `SensorInput` with a `SensorMetadata`), the `JoinNode` builder intercepts this incompatibility.

Before instantiating the join, it uses the `SchemaGenerator` (see Section 4.3) to synthesize a unified struct containing the attributes of both branches. It then transparently injects two auxiliary `Map` operators (left and right *unifiers* implemented via a `SelectBuilder`) into the pipeline. These unifiers project the disparate upstream records into the unified struct, padding missing fields with default values.

This allows the inner join lambda to access the right attributes positionally since the native `IntervalJoin` and `WindowJoin` operators have a built in system to identify each tuple's stream of origin.

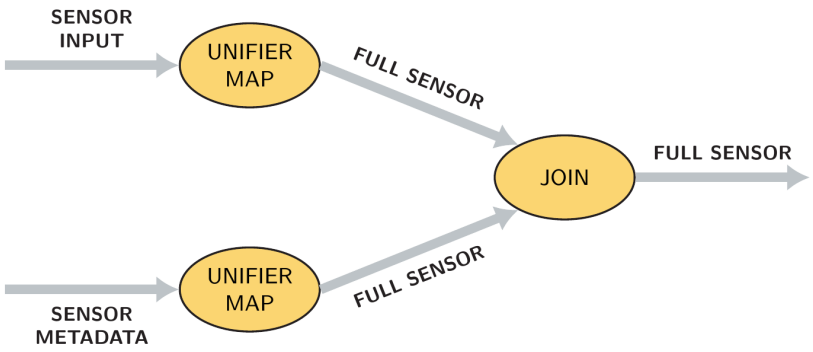


Figure 4.5: Automatically inserted maps in the physical plan to unify schemas before a join.

Tagged Tuples for Intersections: Contrary to native join operators, `INTERSECT` and `INTERSECT_ALL` operations, that are implemented with a `FlatMap`, demand precise tracking of stream provenance to compute intersections accurately.

`Map` operators are injected on both branches to wrap the incoming records inside a `Tagged_Tuple<T>`, explicitly marking the source channel (tag 0 for left, 1 for right). Subsequently, it emits the `Intersect_Builder`, whose internal stateful functor evaluates the tags to determine tuple convergence and properly filter the unified stream.

4.3 Struct Generation

Because WindFlow is a strongly-typed, native C++ library, any data flowing through its physical pipelines must be strictly encapsulated within `structs` validated

at compile time. The `codegen` compiler dynamically synthesizes these data types from the logical schemas defined by the user, aggregating them into a unified header file typically named `(query_id)_structs.hpp`.

Generated structs utilize automatically generated names derived from the original `query_id`, the `OpNode node_id`, and a global monotonic counter. For the sake of clarity, the structures presented in the following examples utilize simplified, custom identifiers rather than the actual auto-generated nomenclature.

4.3.1 Struct Deduplication

When translating a logical query plan into a physical execution graph, it is extremely common for cascading operators (such as chained `where` filters or `distinct`) to share identical input and output schemas. Synthesizing a distinct C++ `struct` for each individual node in the graph would inevitably lead to severe code bloat and, more critically, persistent type-mismatch errors during compilation due to WindFlow's strict template resolution.

To circumvent this, the framework enforces a *Struct Deduplication* mechanism based on canonical field signatures. When the schema generator analyzes an operator's output columns, it extracts their names and corresponding native C++ types. This composition is used to compute a unique identifier independent from column order.

```
    sensor_schema = (SchemaBuilder()
        .add_column("sensor_id", DataTypes.STRING)
        .add_column("timestamp", TimeFormats.ISO8601)
        .add_column("temperature", DataTypes.DOUBLE)
        .add_column("humidity", DataTypes.DOUBLE)
    )

    disordered_sensors = (SchemaBuilder()
        .add_column("humidity", DataTypes.DOUBLE)
        .add_column("sensor_id", DataTypes.STRING)
        .add_column("temperature", DataTypes.DOUBLE)
        .add_column("timestamp", TimeFormats.ISO8601)
    )

    #these two schemas are mapped to the same struct
```

Listing 4.4: Two identical high level schemas with different column order

If multiple relational operators yield logically equivalent schemas, the generated code will systematically refer to the same shared `struct`. This strategy guarantees seamless type alignment across intermediate operator chains, inherently minimizing

expensive runtime casting and optimizing the overall memory footprint at compile time.

```
    //column order depends on which schema is processed first
    struct sensor_struct {
        std::string sensor_id;
        uint64_t timestamp;
        double temperature;
        double humidity
    };
```

Listing 4.5: The struct that implements both schemas from Listing 4.4

4.3.2 KeyBy, Hashing and Windowing Requirements

WindFlow's stateful operators enforce rigorous structural constraints on the C++ `structs` employed as partitioning keys or window accumulators. Consequently, the schema generator must enrich these type definitions with specialized constructs mandated by the C++ standard and the engine's internal memory management.

Key Extraction and the `withKeyBy` Method: Stateful operations, such as `Join` or `GroupBy` aggregations, require key partitioning to correctly route tuples between operator's replicas. To correctly instantiate the physical builders in WindFlow, the generated code must provide an explicit extraction mechanism.

To achieve this, the framework synthesizes a dedicated C++ `struct` to represent the key and generates a specific lambda, defined as a *Key Extractor*. This function takes the current tuple as input, populates the relevant fields, and returns the key `struct`. Finally, this lambda is injected into the operation builder via the invocation of the `.withKeyBy()` method.

```
    auto window_group_2_op = Windowed_Group_Builder<SensorInput,
    GroupedSensors, SensorKey>(
            //aggregation logic in a lambda
            [](const SensorInput& in, GroupedSensor& out) -> void {
                out.sensor_id = in.sensor_id;
                out.COUNT += 1;
            }
        )
        .withName("count_by_sensor")
        .withTBWindow(43200000000ULL, 43200000000ULL)
        .withParallelism(2)
        .withKeyBy(
            //key extraction logic
            [](const SensorInput& in) -> SensorKey {
```

```
                SensorKey out;
                out.sensor_id = in.sensor_id;
                return out;
            }
        )
        //different build method
        .build_keyed();
```

Listing 4.6: The resulting Windowed-Group-Builder that counts how many readings a sensor recorded in a window. The attribute "sensor_id" is automatically extracted to make a key.

Hashing and Memory Routing: Once extracted, the key `struct` is utilized by WindFlow to perform hash-based partitioning, deterministically routing tuples across the P concurrent threads. For the generated key to function correctly within the engine's internal hash maps, simply declaring its fields is insufficient. The generated code must forcefully inject two architectural components:

1. An inline overload of the equality operator (`operator==`) within the `struct`, strictly necessary to allow runtime data structures to accurately resolve hash collisions.

2. A full specialization of the `std::hash<T>` template class, explicitly injected into the `std` namespace. This specialization computes a combined hash of all the fields (e.g., merging strings and floating-point values via logical XOR operations and bit-shifts), ensuring that tuples mapped to the same logical key are consistently routed to the identical worker replica.

```
    struct SensorKey {
        std::string sensor_id;

        // equivalence operator
        bool operator==(const SensorKey& other) const {
            return sensor_id == other.sensor_id;
        }
    };

    //hash method ingestion into std
    namespace std {
        template<>
        struct hash<SensorKey> {
            size_t operator()(const SensorKey& k) const {
                size_t h = 0;
                h ^= (std::hash<std::string>{}(k.sensor_id)
                    + 0x9e3779b9 + (h << 6) + (h >> 2)
```

```
                );
                return h;
            }
        };
    }
```

Listing 4.7: The key-struct needed for Listing 4.6 with hashing methods.

Non-key structs may also be generated with hash methods if a specific operation (e.g. `distinct` that uses a hashmap) demands it.

Window Constructors: Temporal aggregation operators, specifically `KeyedWindows`, dictate that the output struct acting as the window accumulator adheres to strict instantiation constraints. To comply with WindFlow's native object initialization routines, the generated struct must expose a precise set of overloaded constructors:

- An explicit default constructor (`= default`), necessary for basic memory allocation.
- A constructor tailored for non-keyed windows, which accepts the window identifier (`uint64_t win_id`) as its sole parameter.
- A specific constructor for partitioned (keyed) windows, which accepts a constant reference to the key `struct` and the temporal identifier (`win_id`). This allows the runtime to seamlessly propagate the key fields directly into the output accumulator upon initialization.

```
    struct GroupedSensors {
        std::string sensor_id;
        int64_t COUNT = 0;
        uint64_t win_id = 0;        // unique window id

        //default constructor
        GroupedSensors() = default;

        //unkeyed window constructor
        GroupedSensors(uint64_t _id)
            : win_id(_id) {}

        //keyed window constructor
        //it extracts the key from the key-struct
        GroupedSensors(const SensorKey& _key, uint64_t _id)
            : sensor_id(_key.sensor_id), win_id(_id) {}
    };
```

Listing 4.8: The output struct needed for Listing 4.6 with window-based constructors.

4.4 Visiting the Operation Tree

The final step before textually emitting the C++ source code is the physical resolution of the in-memory `OpNode` graph. This phase is orchestrated by the `GraphExplorer`, a stateful visitor that performs a post-order traversal of the parsed tree. By adopting a post-order recursive descent, the framework guarantees that all upstream dependencies (parents) are thoroughly processed, and their respective C++ constructs synthesized, before any downstream consumer is evaluated.

The `GraphExplorer` navigates the graph, delegating the actual synthesis of the C++ logic to the nodes themselves by invoking their overridden `visit()` method. All the generated code is accumulated to compose the complete C++ file.

4.4.1 VisitContext and VisitResult

Before invoking a node's `visit()` method, the explorer instantiates a `VisitContext` object. This dataclass tracks the dynamic topological state, consisting of the current C++ pipe variable (`pipe`), an array of upstream C++ structs (`parent_structs`), a list of physical channels that might require merging (`to_merge_pipes`) and a few global parameters passed down from the `TableEnviroment` (such as the degree of intra-operator parallelism `par`).

```
    @dataclass
    class VisitContext:
        """Common parameters needed to visit nodes."""
        #current pipe name
        pipe: str

        #input structs from the node parents
        parent_structs: List[CppStruct]

        #pipe to be merged by binary nodes
        to_merge_pipes: List[str]

        #generators
        sch_gen: SchemaGenerator
        expr_tl: ExpressionTranslator
        lambda_gen: LambdaGenerator
        build_gen: BuilderGenerator

        operations_counter: int      #monotonic counter
        par: int                     #intra-operator parallelism
        epoch_var: Optional[str]     #for the time_baseline

    #the visit abstact method in OpNode
```

```python
    @abstractmethod
    def visit(self, ctx: VisitContext) -> VisitResult:
        """
        Given the VisitContext each subclass returns a VisitResult
        """
        pass
```

Listing 4.9: Each OpNode concrete subclass has the same visit interface.

Crucially, the context also exposes references to the four generation services (`sch_gen`, `expr_tl`, `lambda_gen`, `build_gen`). This allows the `OpNode` to safely use the shared Jinja2 environment without importing it globally, preserving its nature as a pure intermediate representation.

Once the `OpNode` has utilized the generators to translate its internal dictionaries, it strictly returns a `VisitResult` object. This response contract safely encapsulates the resulting C++ type (`out_struct`), the accumulated C++ builder strings (`emitted_builders`), and the C++ instructions required to append the operator to the current pipeline (`pipe_addition`). The `GraphExplorer` extracts these fragments from the `VisitResult` and sequentially accumulates them into its internal registries, ensuring the global code emission remains perfectly ordered.

```python
    @dataclass
    class VisitResult:
        """Returned info from a node visit."""
        #deduplicated output struct
        out_struct: CppStruct

        #string to append to form pipelines
        pipe_additions: Dict[str, str]

        #emitted builders strings
        emitted_builders: List[str]
```

Listing 4.10: The VisitResult class that encapsulates all the generated C++ code from one node

4.4.2 Graph Construction and Pipe Routing

As the traversal climbs back from the leaf nodes toward the graph's root, the `GraphExplorer` must translate the logical edges into physical WindFlow `MultiPipe` connections. It dynamically maps this topology by maintaining an internal `pipe_counter`, a registry of pipe declarations (`pipes`), and an ordered accumulation list (`pipe_order`).

When the recursion reaches a root source node (a `FromOpNode`), a new physical processing branch is initialized (e.g., `auto pipe_1 = topology.add_source(...)`).

Subsequent unary operations along this linear branch are seamlessly appended to the corresponding C++ variable using WindFlow's fluent method chaining (e.g., `.add(where_1_op)`).

```
    // 1. Left Branch (first_split):
    //Pipe added by the FromOpNode
    auto& pipe_1 = topology.add_source(from_1_op)
        .add(group_2_op)            // base_q operations
        .add(select_3_op)
        .add(where_4_op)            //first_split operations
        .add(select_5_op)
        .add(left_unifier_6_op);  // injected unifier map

    // 2. Right Branch (second_split):
    auto& pipe_2 = topology.add_source(from_7_op)
        .add(group_8_op)
        .add(select_9_op)
        .add(where_10_op)
        .add(select_11_op)
        .add(right_unifier_12_op);

    // 3. Merge added by the JoinNode
    std::vector<wf::MultiPipe*> pipe_0_branches = {&pipe_1, &pipe_2
};
    auto* pipe_0_pointer = wf::merge_multipipes_func(&topology,
pipe_0_branches);

    // 4. Unified pipeline with the join
    auto& pipe_0 = (*pipe_0_pointer)
        .add(join_13_op)
        .add(select_14_op);
```

Listing 4.11: Topology composition needed for Listing 4.1. Pipe1 and Pipe2 were generated by `FromOpNodes` while Pipe0 by the `JoinOpNode`

When the traversal evaluates a binary operator (such as a relational `Join` or a set `Union`), the logical graph bifurcates, implying that the node possesses multiple parents originating from divergent processing pipelines. The `GraphExplorer` natively handles this by detecting multiple parents and assigning distinct, monotonic pipe identifiers to each upstream branch.

The target `OpNode` receives these divergent identifiers within the `to_merge_pipes` list of its `VisitContext`. Before emitting its own builder, the node invokes the `BuilderGenerator` to inject a C++ topological merge instruction leveraging WindFlow's native `merge_multipipes` routine. This efficiently splices the disparate branches into a single unified stream. The resulting C++ string is packed into the `VisitResult`,

allowing the `GraphExplorer` to safely neutralize use-before-declaration faults and variable redefinition errors without manually inspecting the underlying logic.

4.5 Compilation and Execution

Once the `GraphExplorer` has successfully traversed the entire Operation Tree, all the necessary C++ components, including deduplicated structs, operation builders, and topological merge instructions, are fully resolved and stored in memory. The final responsibility of the `codegen` subsystem is to assemble these disparate fragments into a cohesive, compilable C++ application. This is done by a last Jinja2 template that prints all the builders and pipe-additions in order in the `(query_id).main.cpp`.

Subsequently, the `runtime` subsystem takes over to orchestrate the compilation toolchain and the lifecycle of the native streaming process.

4.5.1 The Runtime Subsystem

With the C++ source files written to disk, the `TableEnvironment` delegates the actual deployment to the `runtime` subsystem. Because WindFlow applications are designed for high-performance, long-running continuous processing, blocking the Python interpreter during the execution of the pipeline would severely degrade the user experience and prevent the interactive monitoring of the dataflow.

To solve this, the `runtime` module implements a strictly decoupled, three-stage deployment architecture managed by a centralized `Executor` class:

1. Build System Configuration: The `CMakeManager` dynamically generates or updates a `CMakeLists.txt` file in the output directory. It automatically configures the necessary `include_directories` to link the WindFlow and FastFlow C++ libraries, sets the C++17 standard requirements, and appends a new `add_executable` target for the newly generated `(query_id)_main.cpp`.

2. Synchronous Compilation: Before initiating the stream processing, the `Compiler` module spawns a synchronous sub-process to invoke the native build toolchain (via `cmake configure` and `cmake -build`). This phase is intentionally blocking: it forces the Python environment to wait for the compilation to conclude.

3. Asynchronous Execution and Job Management: Upon a successful build, the `Executor` launches the resulting native binary. This execution is spawned as a detached background process (utilizing POSIX process groups via `start_new_session`) to prevent the Python parent script from hanging indefinitely. The standard output

and error streams of the WindFlow engine are safely redirected to persistent log files (e.g., `query_id.stdout.log`).

Finally, the control flow is immediately returned to the user through a lightweight `JobHandle` object. This handle exposes a non-blocking interface that allows the developer to programmatically verify if the stream is running, fetch the latest log outputs, or gracefully terminate the C++ background process (`SIGTERM`/`SIGKILL`) directly from the Python frontend, maintaining total control over the native execution lifecycle.

Chapter 5

Performance and Benchmarks

The previous chapters detailed the design and implementation of the TIDE, tracing the pipeline lifecycle from the logical Python frontend (Chapter 3) down to the physical C++ code generation and runtime engine (Chapter 4). While the proposed decoupled architecture successfully abstracts the complexities of native stream processing, it is imperative to validate that this high-level expressiveness does not compromise the underlying execution efficiency. To assess the robustness and physical capabilities of the framework, this chapter presents a comprehensive empirical evaluation of the system.

The performance analysis is conducted using the NEXMark benchmark [16], a widely recognized industry standard for evaluating relational continuous queries over data streams. The practical evaluation and dataset generation in this chapter rely on a more recent open-source implementation of the benchmark suite [13].

This evaluation focuses on testing the practical limits of the framework. Specifically, it measures the maximum throughput and scalability while tracking the memory footprint of stateful queries. Finally, to ensure the Python abstraction layer introduces negligible runtime overhead, the performance of the auto-generated pipelines is compared directly against handwritten native C++ baselines.

5.1 NEXMark Benchmark

NEXMark simulates the continuous operational activity of an online auction platform. Rather than relying on a static, pre-recorded dataset, the benchmark utilizes a deterministic generator that models complex interactions across a continuous stream of events.

The NEXMark domain is structured around three interconnected data streams that flow concurrently through the system:

- **Person (Users):** Represents new user registrations on the platform, accounting for approximately 2% of the total event stream.
- **Auction (Items):** Models new auction listings created by users, representing roughly 6% of the traffic.

- **Bid (Offers):** Comprises the dominant share of the workload (roughly 92% of total events), simulating real-time bidding activity on active auctions.

5.1.1 Data-Skew and Hot Entities

Real-world workloads rarely exhibit uniform data distributions. To realistically stress physical execution topologies, particularly hash-based partitioning and parallel worker thread allocation, NEXMark incorporates skewed distributions. Specifically, the generator introduces **Hot Entities** (such as *hot auctions* receiving a disproportionate share of bids or frequent *hot bidders*). This intentional data skew creates localized traffic spikes, allowing the benchmark to evaluate how effectively the underlying stream processing system handles load imbalances and thread contention across parallel operator subtasks.

5.1.2 Time Dilation and Dataset Generation

While foundational domain semantics follow the original specification [16], practical evaluation requires materializing these streams into physical CSV files to feed the Table API's file source builders, utilizing a modern open-source implementation of the benchmark suite [13].

A major challenge in benchmarking stream processing engines with continuous time-based operators (such as tumbling windows and interval joins) is temporal density. Under standard execution rates, benchmark streams compress millions of events into mere seconds of logical time, preventing long-term state accumulation and watermark progression over wide intervals. To overcome this limitation, a **time-dilation** methodology was applied during dataset generation.

By scaling the logical timestamps, short benchmark runs were stretched across a full calendar month of Event Time (anchored from September 1st, 2026). Furthermore, three distinct datasets were synthesized with varying scales; comprising 10 million, 30 million, and 70 million tuples respectively. This multi-scale approach enables a granular performance profile: massive datasets (70M) are leveraged to measure peak stateless throughput and parallel scalability, while smaller or moderately scaled datasets (10M and 30M) allow rigorous profiling of stateful memory footprints during memory-intensive interval joins and window aggregations.

5.2 Query Selection and Limitations

While the primary objective of NEXMark is to measure physical throughput and memory consumption, the benchmark also serves as a rigorous evaluation of a frame-

work's expressive power. Translating the standard NEXMark suite into WindFlow TIDE reveals the semantic boundaries of the current implementation.

Not all of the original benchmark queries can be natively expressed using the framework. This is not indicative of execution failures, but rather reflects the deliberate architectural boundaries of a pure relational API operating over a lightweight, append-only C++ streaming library. Before analyzing the performance of the supported query subset (Section 5.2.2) and the physical bottlenecks encountered in complex topological scenarios (Section 5.2.4), it is necessary to map the current expressiveness limitations.

5.2.1 Expressive Boundaries and Out-of-Scope Queries

The unsupported NEXMark queries can be categorized into four distinct architectural limitations, stemming either from the current boundaries of the Python Domain-Specific Language or from the physical constraints of the underlying stream processing paradigm.

- **Language and DSL Constraints (Q14, Q21, Q22):** Certain queries require procedural logic and scalar functions that fall outside standard relational algebra. Specifically, Queries 14 and 21 rely on conditional branching (e.g., `CASE WHEN` statements) to format output fields dynamically, while Query 22 requires advanced string manipulation functions (such as `split` or character counting). The current TIDE AST focuses entirely on relational topology and basic numeric/boolean expressions; thus, arbitrary scalar functions are currently out of scope.

- **Filtered and Distinct Aggregations (Q15, Q16, Q17):** These queries demand complex stateful accumulations, such as computing multiple metrics over different subsets of a window using the `FILTER (WHERE ...)` clause, or extracting unique occurrences using `COUNT(DISTINCT)`. In TIDE, C++ window accumulators are synthesized as lightweight, flat structures. Computing a distinct count would require instantiating dynamic Hash Tables inside the state of every single window, a feature not yet supported by the underlying code generator.

- **The "Keep Last" Semantics and Top-N (Q18, Q19):** Queries 18 and 19 require identifying the absolute winner of an auction or the top bidders using ranking functions (e.g., `ROW_NUMBER() OVER ... <= 1` or `ORDER BY ... LIMIT 1`). TIDE operates on a pure *append-only* streaming model. Without a changelog stream capable of emitting retraction messages (`-U` / `+U`) to invalidate previously emitted tuples, the framework cannot natively express "Keep

Last" semantics. (Note that TIDE's `DistinctOp` enforces a "Keep First" semantic).

- **Dynamic Windowing (Q11):** Query 11 tracks user sessions, grouping bids that occur within a dynamic timeframe of each other. This requires *Session Windows*, whose boundaries are data-driven rather than time-driven. TIDE's API currently restricts stateful discretization to rigid mathematical grids (Tumbling and Sliding windows) or fixed counts.

5.2.2 Supported Relational Subset

Having delineated the boundaries of the framework, this section examines the core subset of the NEXMark suite that TIDE natively supports.

To contextualize the declarative expressiveness of TIDE, each analyzed query is paired with its canonical benchmark formulation. For the first eight queries, the logical intent is presented using the original Continuous Query Language (CQL) syntax as defined in the foundational NEXMark technical report [16]. For the subsequent queries the reference statements are drawn directly from the Flink benchmark suite [13].

Query 1: Stateless Projections: Query 1 establishes the performance baseline by applying a stateless, one-to-one transformation to convert currency values.

```
SELECT Istream(auction, DOLTOEUR(price), bidder, datetime)
FROM bid [ROWS UNBOUNDED];
```

Listing 5.1: CQL formulation for Query 1.

In CQL semantics, the combination of the `[ROWS UNBOUNDED]` window and the `Istream` operator denotes a pure append-only flow where every incoming tuple is projected and emitted immediately without state accumulation.

```
#current exchange rate
dol_to_eur = col("price") * 0.87

q1 = (bid
    .select(
        "auction_id",
        dol_to_eur.alias("price_eur"),
        "bidder",
        "bid_dateTime"
    )
)
```

Listing 5.2: TIDE implementation for Query 1.

TIDE seamlessly replicates this behavior through a reactive pipeline comprising a source, a single `SelectOpNode`, and a sink.

Physically, Q1 serves as the benchmark's saturation point, measuring the maximum I/O throughput and serialization speed of the engine before introducing network or memory contention.

Query 2: Reactive Filtering and Selectivity Query 2 extends the stateless baseline by isolating bids associated with specific auction identifiers, introducing highly selective row-by-row filtering.

```
SELECT Rstream(auction, price)
FROM Bid [NOW]
WHERE auction = 1007 OR auction = 1020 OR auction = 2001
   OR auction = 2019 OR auction = 2087;
```

Listing 5.3: CQL formulation for Query 2.

The CQL `[NOW]` window extracts only the tuples arriving at the current logical instant, which the `Rstream` operator immediately materializes. In the context of relational stream processing, this specific combination translates to a purely stateless filter applied on a per-tuple basis, requiring no internal memory buffers or watermark synchronizations.

```
cond = (
    #parenthesis are needed
    (col("auction_id") == 1007)
    | (col("auction_id") == 1020)
    | (col("auction_id") == 2001)
    | (col("auction_id") == 2019)
    | (col("auction_id") == 2087)
)

q2 = (bid
    .where(cond)
    .select("auction_id", "price")
)
```

Listing 5.4: TIDE implementation for Query 2.

Query 3: Stateful Correlations and Key-Parallelism Query 3 elevates the complexity by introducing multi-stream correlations. It requires cross-referencing newly registered users with active auctions to suggest local items within specific categories.

```
SELECT Istream(P.name, P.city, P.state, A.id)
```

```
FROM Auction A [ROWS UNBOUNDED], Person P [ROWS UNBOUNDED]
WHERE A.seller = P.id
  AND (P.state = 'OR' OR P.state = 'ID' OR P.state = 'CA')
  AND A.category = 10;
```

Listing 5.5: CQL formulation for Query 3.

In the abstract CQL semantics, the `[ROWS UNBOUNDED]` windows over both the `Auction` and `Person` streams indicate a continuous Cartesian product over infinite history, filtered by an equality predicate.

```
p_cond = (col("state") == "OR") | (col("state") == "CA") | (col("
    state") == "ID")
a_cond = col("category") == 10

#renaming the seller column to person_id for the join
renamed_auction = (auction
    .where(a_cond)  #filter before the join for efficiency
    .rename_columns({"seller": "person_id"})
)

#interval covering the entire dataset to emulate UNBONDED
interval = Interval(Duration.days(-30), Duration.days(30))

q3 = (person
    .where(p_cond)
    .join(renamed_auction, on=["person_id"], attachment=interval)
    .select("name", "city", "state", "auction_id")
)
```

Listing 5.6: TIDE implementation for Query 3.

The TIDE implementation approximates the unbounded semantic by applying a symmetrical 30-day `Interval` attachment, which securely encapsulates the entire dilated benchmark timeline without breaking the C++ temporal arithmetic bounds.

By applying the highly selective `p_cond` and `a_cond` filters to the isolated branches *before* invoking the `join()` operator, the footprint of the state buffers maintained by the C++ engine is drastically minimized.

Query 8: Asymmetric Intervals Query 8 correlates newly registered users with the auctions they create shortly after joining the platform. This query specifically evaluates the framework's capability to correlate events across a relative time envelope rather than a fixed temporal grid.

```
SELECT Rstream(P.id, P.name, A.reserve)
FROM Person [RANGE 12 HOUR] P, Auction [RANGE 12 HOUR] A
```

```
WHERE P.id = A.seller;
```

Listing 5.7: CQL formulation for Query 8.

Contrary to Q3, Q8 is a proper [0, +12h] interval join. Testing both is necessary as Q3 filters out most tuples before the join operator. By comparing these two queries we will see the impact of the join memory buffers.

```
#time interval
interval = Interval(
    Duration.minutes(0),
    Duration.hours(12)
)

#renaming seller to allow for a keyed-join
renamed_auction = auction.rename_columns({"seller": "person_id"})

q8 = (person
    .join(renamed_auction, on=["person_id"], attachment=interval)
    .select("person_id", "name", "reserve")
)
```

Listing 5.8: TIDE implementation for Query 8.

Query 20: The Filter Join Pattern and State Minimization Query 20 expands a subset of bids with the detailed information of their corresponding auctions, focusing exclusively on auctions belonging to a specific category.

```
SELECT
    auction, bidder, price, channel, url, B.`dateTime`, B.extra,
    itemName, description, initialBid, reserve, A.`dateTime`,
   expires, seller, category, A.extra
FROM
    bid AS B INNER JOIN auction AS A on B.auction = A.id
WHERE A.category = 10;
```

Listing 5.9: Flink SQL formulation for Query 20.

Like Q3 this query requires an unbounded `INNER JOIN` which TIDE does not support. However this time we can narrow the interval needed based on domain knowledge: since an auction typically lasts around 8 hours, an asymmetric interval of `[0, +12h]` replicates the inner join semantics while guaranteeing deterministic state reclamation.

Bids made before the auction started or a long time after it ended will be discarded. Thus our implementation is an approximation of the original; we opted for this compromise as those discarded bids are usually not relevant.

```
#12h interval
interval = Interval(
    Duration.minutes(0),
    Duration.hours(12)
)

#filter and renaming
target_auctions = (auction
    .where(col("category") == 10)
    .rename_columns({
        "id": "auction_id",
        "extra": "auction_extra"
    })
)

#keyed interval join
q20 = (target_auctions
    .join(bid, on=["auction_id"], attachment=interval)
    .select(
        "auction_id", "bidder", "price", "channel", "url", "
   bid_dateTime",
        "extra", "itemName", "description", "initialBid", "reserve",
        "auction_dateTime", "expires", "seller", "category", "
   auction_extra"
    )
)
```

Listing 5.10: TIDE implementation for Query 20.

More importantly, Q20 serves as the primary benchmark for the *Filter Join* pattern. By pushing the predicate `category == 10` upstream of the `join()` operator, TIDE ensures that the underlying C++ join buffers only store the small fraction of auctions belonging to that specific category. When the massive stream of bids arrives at the right side of the join, bids referring to other categories simply probe the internal hash map, yield no matches, and are instantly discarded without triggering any memory allocation. Because of this Q20 is the only proposed stateful query that can process the full 70 million bids dataset without encountering memory allocations problems.

Query 12: Processing Time and System Clock Windows Query 12 introduces a fundamental shift in the temporal paradigm, moving from an event-driven progression to a system-driven execution. It measures the volume of bids submitted by each user within a fixed processing time limit, validating the engine's capacity to handle strict wall-clock windows.

```
SELECT
    bidder,
    count(*) as bid_count,
    window_start AS starttime,
    window_end AS endtime
FROM TABLE(
    -- 10 seconds tumbling process time window
        TUMBLE(TABLE B, DESCRIPTOR(p_time), INTERVAL '10' SECOND))
GROUP BY bidder, window_start, window_end;
```

Listing 5.11: Flink SQL formulation for Query 12.

Unlike the previously analyzed queries, which rely on the `EVENT_TIME` policy to extract logical timestamps directly from the data payload, Q12 evaluates the stream based on the hardware clock of the executing machine. In TIDE, transitioning to this paradigm requires initializing the `TableEnvironment` with `TimePolicy.INGRESS_TIME`.

```
#window based on system clock execution (1 second)
process_window = Window.createTBWindow(Duration.seconds(1))

q12 = (bid
    .name_query("ingress_time_windows")
    .group_by("bidder", window=process_window)
    .select("bidder", count().alias("processed"))
)
```

Listing 5.12: TIDE implementation for Query 12.

From a physical execution perspective, evaluating Q12 over static CSV files introduces a fascinating stress test for the engine. Because the file is read in paralel at maximum disk I/O speed, completely decoupled from the original 30-day logical event time, millions of records are ingested in a matter of seconds. A processing window of 1 s will thus capture a massive, unpredictable proportion of the dataset, dictated entirely by the operating system's context-switching and thread scheduling.

5.2.3 Successive Aggregations and Boundary Slicing

This section analyzes the semantic approximations required when dealing with continuous entity lifecycles (Boundary Slicing) and the structural limitations encountered when chaining multiple aggregations in a pure append-only engine.

Query 9: Dynamic Lifecycles and Boundary Slicing Query 9 requires identifying the absolute winning bid for each auction. This implies evaluating the maximum price strictly within the dynamic, event-driven lifespan of each individual

auction (from its creation until its `expires` timestamp).

```
SELECT
    id, price
FROM (
   SELECT A.*, B.auction, B.bidder, B.price, B.`dateTime` AS
   bid_dateTime,
   --over window and order by on price
     ROW_NUMBER() OVER (
        PARTITION BY A.id ORDER BY B.price DESC, B.`dateTime` ASC
    ) AS rownum
   FROM auction A, bid B
   WHERE A.id = B.auction
        AND B.`dateTime` BETWEEN A.`dateTime` AND A.expires
)
--keeps only the maximum price
WHERE rownum <= 1;
```

Listing 5.13: Flink SQL formulation for Query 9.

As the current TIDE implementation operates purely on an append-only paradigm without retraction mechanisms (e.g., `-U/+U` changelog messages) or *Over Window* constructs, natively replicating this Top-1 semantic requires specific architectural adaptations.

A naive approach would utilize a `keyed-global-groupBy` to compute a running maximum. While logically valid, this unwindowed operator emits a continuous changelog stream: it outputs a new tuple every single time the maximum price for an auction is updated. Rather than producing one definitive winner per auction, this floods the downstream topology with thousands of partial updates.

To strictly enforce the emission of a single, definitive result per entity, the system must forcefully discretize the stream using a `keyed-windowed-groupBy`. By restricting the aggregation period to a wide `Tumbling Window`, the operator emits the maximum value only upon window closure. However, because an auction's event-driven lifecycle (from `auction_dateTime` to `expires`) is asynchronous to rigid mathematical window boundaries, auctions crossing these cutoffs will inevitably output multiple results (one for each intersected window).

```
#an auction lasts about 8h so we match within a day
interval = Interval(Duration.seconds(0), Duration.hours(24))

#the bid must arrive after the auction is made and before it expires
valid_bid = (
    (col("bid_dateTime") < col("expires"))
    & (col("auction_dateTime") < col("bid_dateTime"))
)

```

```
q9 = (bid
    .name_query("winning_bids")
    #merges each bid with its auction
    .join(auction, on=["auction_id"],
        attachment=interval,
        where=valid_bid
    )
    #we consider every auction in a 2 days window
    #given the usual auction length many auctions fit entirely
   within the window
    .group_by("auction_id",
        window=Window.createTBWindow(Duration.days(2))
    )
    .select("auction_id", max("price").alias("winning_price"))
)
```

Listing 5.14: TIDE implementation for Query 9.

Despite this *Boundary Slicing* effect, the windowed discretization was deliberately chosen over the continuous global aggregation. As Q9 is primarily utilized as a nested subquery for successive chained aggregations (like in Q4), bounding the emission is structurally mandatory to prevent downstream operators from accumulating duplicate partial updates and corrupting subsequent algebraic formulas.

Query 4: Chained Aggregations and the Retraction Problem Query 4 computes the average of the winning bid prices for all auctions within each category. This requires a topological pipeline where the output of a first aggregation (the maximum price per auction) directly feeds a second continuous aggregation (the average per category).

```
SELECT Istream(AVG(Q.final))
FROM Category C, (
    --query9 implemented in cql plus category grouping
    SELECT Rstream(MAX(B.price) AS final, A.category)
    FROM Auction A [ROWS UNBOUNDED], Bid B [ROWS UNBOUNDED]
    WHERE A.id=B.auction
        AND B.datetime < A.expires
        AND B.datetime > A.datetime
    GROUP BY A.id, A.category
    ) Q
WHERE Q.category = C.id
GROUP BY C.id;
```

Listing 5.15: CQL formulation for Query 4.

```
q4 = (auction
```

```
    #keyed-global-group version of Q9
    .join(bid, "auction_id", attachment=interval, where=valid_bid)
    .group_by("auction_id", "category")
    .select("category", max("price").alias("final"))
    #chaining a second implicit query with a continuous aggregation
    .group_by("category")
    .select("category", avg("final").alias("avg_price"))
)
```

Listing 5.16: TIDE implementation for Query 4.

While TIDE safely compiles this chained topology (properly parallelizing the first stage via `auction_id` and the second via `category`), the physical execution yields numerically incorrect averages.

This failure isolates a fundamental architectural limit of WindFlow's current continuous *in-stream append/update* semantics. When the first `group_by` processes a bid of 50€ for Auction 100, it emits `MAX=50`. If a subsequent bid of 80€ arrives, it correctly updates its internal state and emits a new `MAX=80`. However, the second `group_by` receives both the 50€ and the 80€ tuples, treating them as two entirely separate auctions. Instead of replacing the old maximum, it accumulates them ($50 + 80 = 130$) and divides by two, emitting a corrupted average of 65€.

By using the `keyed-windowed-groupBy` version of Q9 this issue persists but it is less impacting as less auctions are going to output multiple maximums. Given this structural limitation, Q4 is excluded from the performance benchmarks, while Q9 is retained strictly in its windowed, discretized formulation.

5.2.4 Topological Bottlenecks

While TIDE successfully parallelizes keyed operations through deterministic hash routing, certain relational constructs force the physical dataflow to converge into a single execution thread. This section examines how emulating global predicates (such as the `ALL` keyword) introduces severe topological bottlenecks and memory contention.

Query 5: Emulating Global Predicates and Thread Collapse Query 5 identifies the "hot" auctions that received the highest number of bids in a given period. In standard SQL, finding a global maximum among grouped subsets is typically resolved using an `>= ALL` subquery.

```
SELECT Rstream(auction)
--number of bids per auction
FROM (SELECT B1.auction, count(*) AS num
      FROM Bid [RANGE 60 MINUTE SLIDE 1 MINUTE] B1
```

```
      GROUP BY B1.auction)
--if its higher than all counts its the maximum
WHERE num >= ALL (
                    SELECT count(*)
                    FROM Bid [RANGE 60 MINUTE SLIDE 1 MINUTE] B2
                    GROUP BY B2.auction
                );
```

Listing 5.17: CQL formulation for Query 5.

Because TIDE does not natively support the `ALL` predicate, the query must be algebraically restructured into a self-join: calculating the counts per auction, extracting the absolute maximum, and joining it back to the counts to keep only the maximum ones.

```
half_day = Window.createTBWindow(Duration.hours(12))

# count bids per auction in a 12-hour window
bids_per_auction = (bid
    .name_query("bids_counter_per_auction")
    .group_by(
        "auction_id", window=half_day
    )
    .select("auction_id", count().alias("bids_counter"))
)

#extract the maximum of all counts in a window
most_requested = (bids_per_auction
    .name_query("max_bid_count")
    #keyeless group has a forced parallelism to 1
    .group_by(window=half_day)
    .select(max("bids_counter").alias("max_count"))
)

#interval to absorb the allignment delay
join_interval = Interval(Duration.hours(0), Duration.hours(13))

#by joining with the absolute max we keep the hot auctions
hot_auctions = (bids_per_auction
    .rename_columns({"bids_counter": "max_count"})
    .join(
        most_requested, on=["max_count"],
        attachment=join_interval
    )
    .select("auction_id", "max_count")
)
```

Listing 5.18: TIDE implementation for Query 5.

The second aggregation (`most_requested`) operates without a partitioning key. Unkeyed aggregations inherently force the C++ execution graph to route all intermediate data to a single worker thread (`parallelism = 1`). While mathematically necessary to compute a global metric, this destroys the pipeline's throughput.

Query 7: The Cost of Keyless Extractions A similar topological anti-pattern is encountered in Query 7, which seeks the bids featuring the absolute highest price in the last period. Like Q5, it requires extracting a global maximum via an unkeyed window (`parallelism = 1`) and joining it back to the raw, high-throughput bid stream. While Q7 can theoretically be expressed using a Keyless Theta-Join, evaluating a massive raw stream against a globally centralized maximum severely limits scalability and introduces prohibitive execution times on larger datasets. Due to these overlapping topological bottlenecks, Q7 is considered practically unsupported for the high-throughput performance benchmarks and is excluded from the final evaluation suite.

5.3 Experimental Setup

All empirical benchmarks and performance measurements were conducted on a shared high-performance server running Ubuntu 24.04 LTS (Linux Kernel 6.8.0). The underlying hardware is equipped with a dual-socket Intel Xeon E5-2695 v2 processor at 2.40GHz, featuring 24 physical cores and yielding a total of 48 logical threads via simultaneous multithreading. Because WindFlow relies on non-blocking concurrent queues and active thread pinning to maximize throughput, allocating the entirety of the logical cores can lead to severe CPU starvation and operating system unresponsiveness. Consequently, to ensure stable and reproducible measurements in a shared environment, the maximum parallelism for the execution graphs was strictly capped at 40 concurrent threads. This configuration provides a massive parallel execution environment while leaving sufficient computational headroom for OS background scheduling and I/O handling.

In terms of primary memory, the machine is provisioned with 64 GB of total RAM (of which approximately 62 GiB are effectively available for the application space). This extensive memory capacity is critical for the evaluation of the stateful topologies (such as Q5, Q8, and Q20). It provides a high ceiling to accurately profile the memory footprint of the generated C++ interval join buffers and window accumulators under heavy loads, ensuring that the benchmarks reflect pure in-memory

algorithmic efficiency without triggering disk swapping or Out-Of-Memory (OOM) terminations.

Finally, regarding the software stack, the logical frontend and the TIDE compiler were executed using Python 3.12.3, while the generated physical topologies and the underlying WindFlow runtime were compiled utilizing GCC 13.3.0. This configuration ensures optimal compilation times and full support for the modern C++17 features required by the auto-generated data structures and operator builders.

5.4 Benchmark Results

Having established the semantic boundaries of the framework and defined the hardware execution environment, this section presents the empirical evaluation of the TIDE compiler and its underlying runtime. The performance analysis is structured to validate both the absolute physical capabilities of the generated topologies and the relative efficiency of the declarative abstraction layer.

In section 5.4.1 we examine the execution of the supported NEXMark suite in TIDE. Beyond raw throughput and memory consumption, the evaluation incorporates a detailed analysis of *data skew* and topological imbalances. Subsequently, Section 5.4.2 provides a direct performance comparison between TIDE's auto-generated pipelines and equivalent baselines authored manually using WindFlow's native C++ builders.

Data Skew: In the context of this evaluation, *data skew* refers to the asymmetrical distribution of data streams and computational load across the parallel worker threads. This phenomenon is not a system anomaly, but rather an intrinsic characteristic of the NEXMark workload, which relies on non-uniform distributions to generate "Hot Entities" (e.g., *hot auctions*) to deliberately stress the system's hash-based partitioning (`KeyBy`) mechanisms.

Data skew was quantified by parsing the engine's execution logs to measure the exact volume of tuples processed and emitted by each individual sink thread.

Throughput Measurement: The overall processing performance is evaluated through the system's *throughput*, formally defined as the volume of tuples processed divided by the total execution time.

$$\text{Throughput} = \frac{N_{\text{tuples}}}{t_{\text{stop}} - t_{\text{start}}} \tag{5.1}$$

Where:

- t_{start} is the exact moment the `Source` operator ingests the first tuple.

- t_{stop} is the moment the `Sink` operator completely processes the last tuple and closes the stream.

5.4.1 TIDE Results and Data Skew

This section presents the empirical performance of the queries supported by the TIDE API. To provide a comprehensive diagnostic profile, the evaluation of each topology is accompanied by a dual-axis analysis: the absolute processing throughput (tuples per second) as a function of the parallelism degree, and the relative *data skew*, measuring the percentage of tuples processed by each individual worker thread. Where relevant, topological dataflow representations are referenced to explain how skewed entity distributions impact the internal parallel balance of the engine.

Query 1: Stateless Baseline and Linear Scalability Query 1 represents the absolute performance baseline of the framework. As a purely stateless, map-only projection that converts currency values, it bypasses internal state allocations, watermark synchronizations, and hash-routing overheads. To accurately measure the peak saturation point of the engine, the query was executed against the massive 70-million tuple dataset (Bid stream).

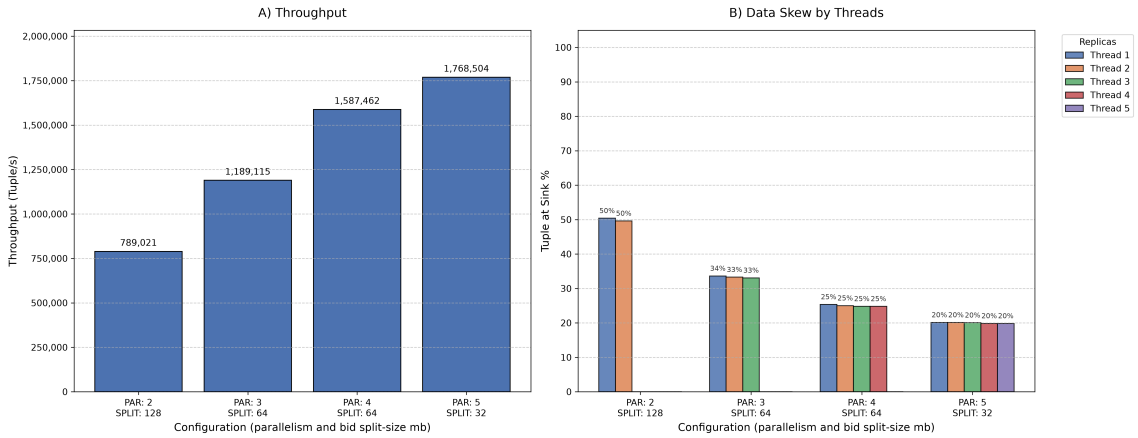


Figure 5.1: Performance profile for Query 1 on the 70M dataset. (A) Maximum processing throughput scaling linearly up to 5 threads. (B) Perfectly uniform data distribution at the sink, validating the source chunk-splitting mechanism.

As illustrated in Figure 5.1(A), the TIDE generated C++ topology exhibits near-perfect linear scalability. Starting from an initial throughput of 789,021 tuples/s at `par = 2`, the engine seamlessly scales to a peak of 1,768,504 tuples/s at `par = 5`. This metric confirms that the Python AST lowering introduces zero runtime overhead: the implicitly typed C++ lambda functions (e.g., the `double` conversion for the exchange rate) are evaluated at maximum CPU speed, and WindFlow's

non-blocking concurrent queues successfully absorb massive event ingestion without triggering backpressure.

The topological analysis of the load distribution, shown in Figure 5.1(B), reveals a perfectly uniform *data skew*. In a map-only pipeline, data is not routed through a `KeyBy` hash function. Therefore, the perfect balance observed at the sink is the direct result of the TIDE's `InputFileConfiguration`. By dividing the massive CSV into smaller physical chunks (`split_size`) and assigning them in a round-robin fashion to the reading threads, the source operator guarantees an equitable distribution of the raw tuples from the very beginning of the dataflow.

Query 2: Selective Filtering and Output Skew Query 2 evaluates the engine's capacity to process and discard millions of irrelevant events through a highly selective boolean predicate. Similar to the baseline, the topology was executed against the 70-million tuple dataset to measure peak filtering efficiency.

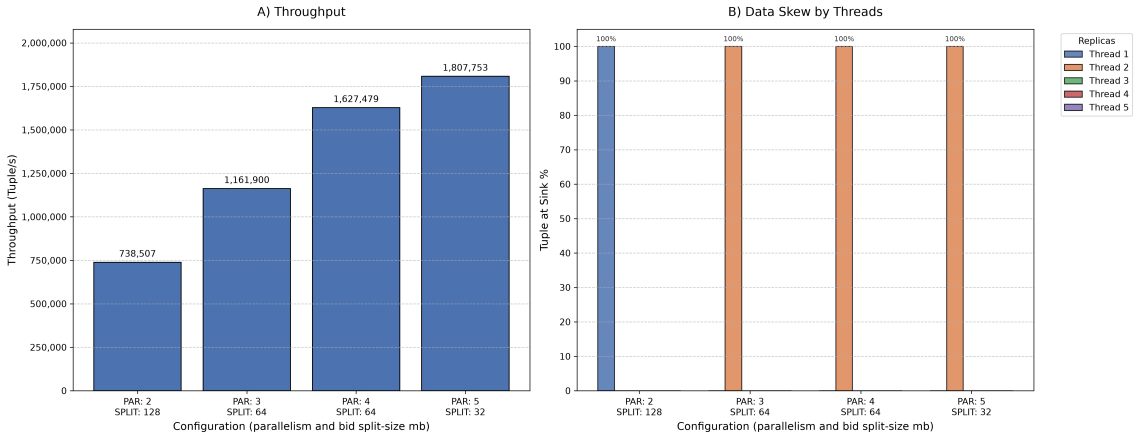


Figure 5.2: Performance profile for Query 2. (A) Linear throughput scaling up to 1.8M tuples/s, proving that parallel filtering introduces negligible overhead. (B) Extreme data skew at the sink, where a single worker emits 100% of the surviving tuples.

The throughput chart in Figure 5.2(A) confirms that the declarative filter maintains the same linear scalability observed in the stateless projection. The pipeline successfully scales from 738,507 tuples/s at `par = 2` to a peak of 1,807,753 tuples/s at `par = 5`. This metric confirms that the C++ `Where_Functor` compiled by TIDE evaluates the inline boolean conditions at maximum CPU speed, discarding the vast majority of the tuples without saturating the parallel workers.

However, the topological load distribution shown in Figure 5.2(B) presents a stark contrast to the perfect balance of Query 1. The data skew at the sink is completely unbalanced, with a single worker thread emitting 100% of the surviving tuples across all parallelism configurations.

This extreme output imbalance does not contradict the linear throughput scaling; rather, it highlights the distinction between computational skew and output skew. Because the source operator divides the dataset into physical chunks (`split_size`) distributed in round-robin, all worker threads concurrently ingest data and apply the `where` filter in parallel. The linear speedup in Figure 5.2(A) proves that the computational burden of evaluating the predicate is evenly shared across all cores.

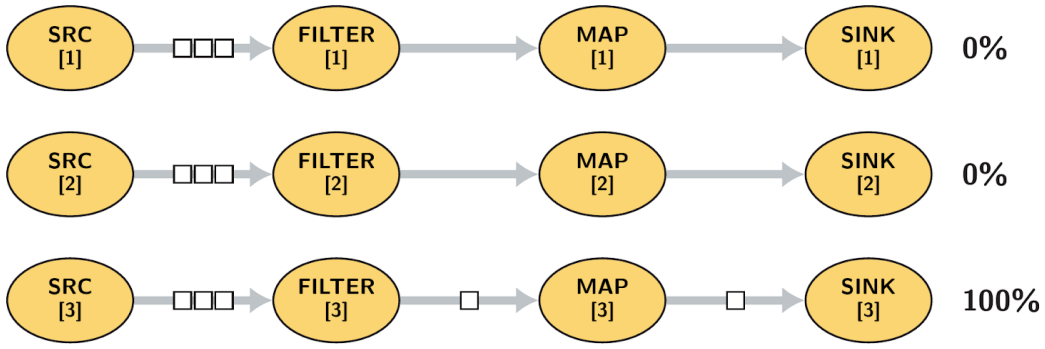


Figure 5.3: Internal view of filtered tuples.

The 100% skew observed at the sink simply reflects the statistical distribution of the dataset combined with the chunking strategy: all the extremely rare tuples matching the specific `auction_id` targets happened to reside within the physical data splits assigned to one specific worker thread. The pipeline avoids any true topological bottleneck, as the single-thread dominance occurs exclusively during the final materialization phase.

Query 3: Stateful Correlations and Data Skew in Joins Query 3 introduces multi-stream correlations by joining the filtered `Person` and `Auction` streams over a temporal interval. Unlike the stateless baselines, this topology exercises the framework's hash-routing mechanism for Key-Parallelism, testing stateful memory management under skewed workloads.

As shown in Figure 5.4(A), the throughput reaches a peak of approximately 1.34M tuples/s at `par = 3`, maintaining stable performance at `par = 4`. The transition from stateless processing to a stateful interval join introduces internal buffering and hash-lookup overheads, yet the pipeline maintains a robust execution rate.

The most distinctive characteristic appears in the load distribution chart of Figure 5.4(B). Unlike the uniform distribution of Query 1 or the simple output skew of Query 2, Query 3 exhibits a genuine *data skew* during the join phase (with Thread 2 handling roughly 88% of the output tuples at $par = 2$ and around 81% at $par =$

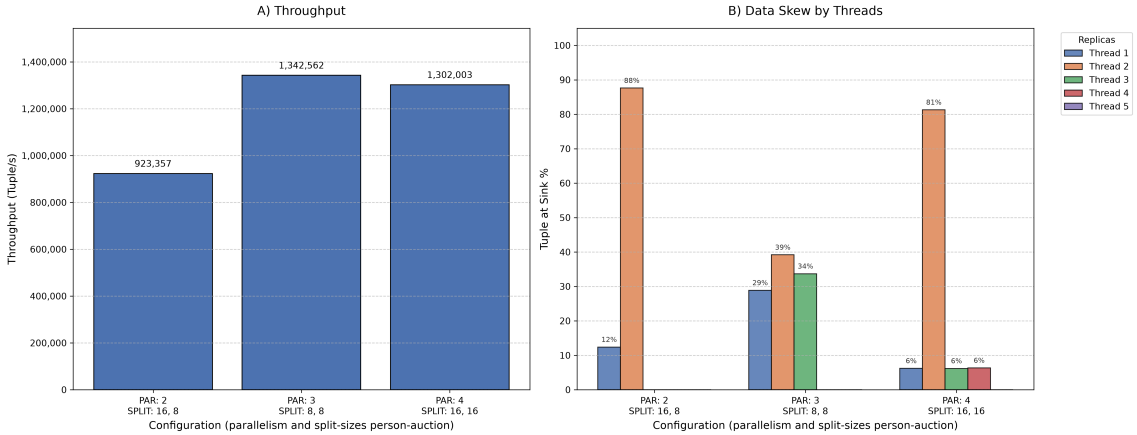


Figure 5.4: Performance profile for Query 3. (A) Throughput scaling across parallelism configurations. (B) Data skew at the sink, revealing significant load imbalance driven by hot entities during the join phase.

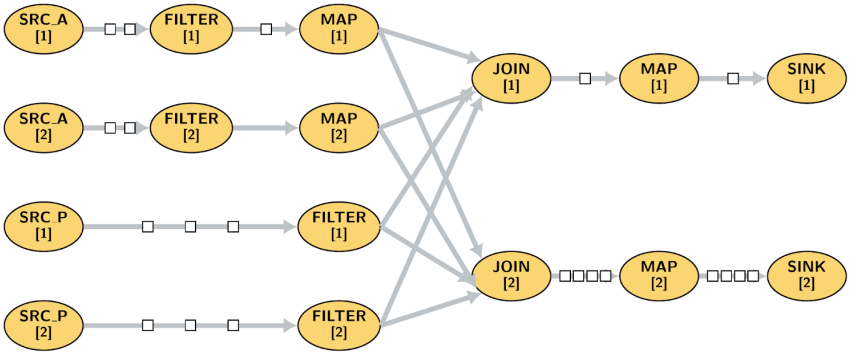


Figure 5.5: Internal view of join skew.

4). This imbalance is a direct consequence of real-world data distributions and "Hot Entities", specific users who register multiple listings, concentrating a disproportionate share of successful matches onto a single worker thread via hash-partitioning, as shown in Figure 5.5.

Despite this pronounced skew in the stateful join buffers, the high degree of parallelism prevents overall system degradation. Because the upstream filtering and projection stages operate concurrently across all available worker cores, the ingestion pipeline remains fluid, ensuring that the localized skew in the join operator does not cripple the global throughput.

Query 5: Unkeyed Extractions and Window Cascades Query 5 seeks to identify the auctions that recorded the highest number of bids within a sliding timeframe. In standard SQL, this is typically formulated using a `>= ALL` subquery. As

discussed in Section 5.2.4, lacking native support for the `ALL` predicate, the query was refactored into a self-join: counting bids per auction, extracting the global maximum count, and joining the peak value back to the original counts.

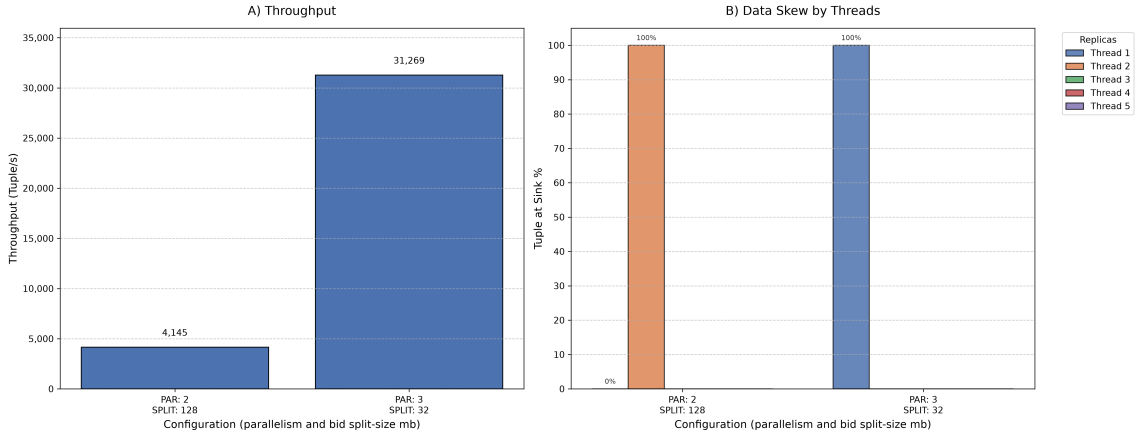


Figure 5.6: Performance profile for Query 5 on the 10M dataset. (A) Exceptionally low throughput caused by unkeyed topological bottlenecks.

Due to its extreme structural inefficiency, Q5 yielded the worst performance across the entire benchmark suite, forcing evaluation against the smallest 10-million tuple dataset (approx. 3 GB of raw CSV data). As shown in Figure 5.6(A), the throughput barely reaches 4,145 tuples/s at `par = 2`, and marginally improves to 31,209 tuples/s at `par = 3`.

This performance collapse is driven by two overlapping architectural bottlenecks. First, extracting the global maximum requires an *unkeyed aggregation*. As previously established, keyless grouping completely bypasses hash-routing, forcing the underlying C++ engine to serialize the entire dataflow into a single worker thread (`parallelism = 1`).

Secondly, this single-thread bottleneck is severely exacerbated by a *window-on-window* multiplication effect. The upstream grouping emits a continuous flow of local maximums for every single sliding window interval. The downstream global maximum operator is thus forced to process and buffer thousands of overlapping window outputs simultaneously. This cascading effect caused the active physical state to swell disproportionately, with the initial 3 GB dataset forcing the C++ join buffers to allocate up to 25 GB of physical RAM during execution.

Query 8: Hash Routing and Skew-Driven Bottlenecks Query 8 isolates the performance of the Keyed Interval Join by correlating users and their newly created auctions without the mitigating effect of heavy upstream selectivity. Because the engine does not discard the vast majority of tuples prior to the join operator (as

it did in Query 3), the physical throughput is almost entirely dictated by the hash-partitioning distribution and the resulting memory contention within the C++ join buffers.

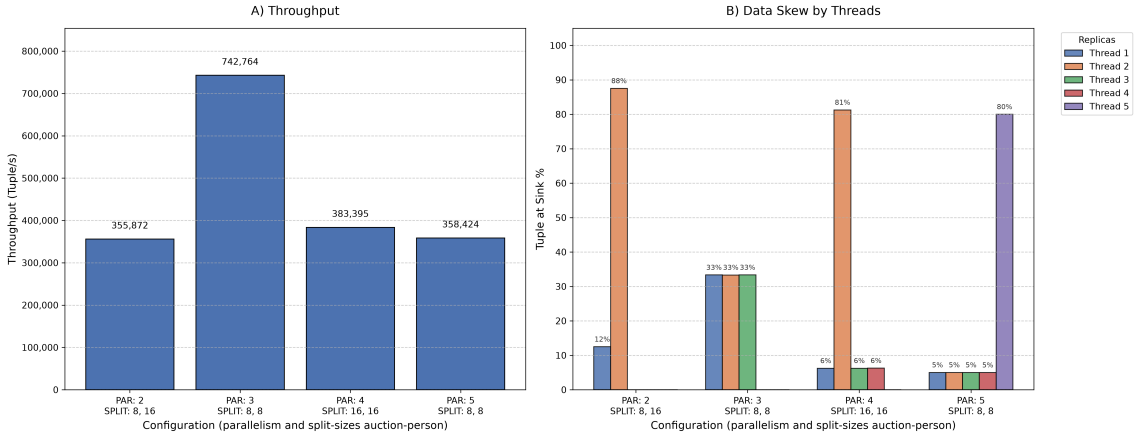


Figure 5.7: Performance profile for Query 8. (A) Throughput volatility across parallelism configurations. (B) Data skew at the sink, demonstrating a direct correlation between hash-based load balance and absolute pipeline throughput.

The performance profile in Figure 5.7(A) exhibits a highly volatile throughput behavior that correlates directly with the topological load distribution shown in Figure 5.7(B). At `par = 2`, the execution is severely bottlenecked, achieving only 355,872 tuples/s. This degradation is driven by extreme data skew: Thread 2 is forced to process approximately 88% of the output load. This imbalance arises naturally from the real-world distribution of data. The same `person_id` keys collide on the same worker thread, saturating its specific interval buffer while the other thread remains underutilized.

When scaled to `par = 3`, the throughput dramatically doubles, reaching a peak of 742,764 tuples/s. As illustrated in the skew chart, this configuration fortuitously achieves a perfect computational balance, with each of the three worker threads handling precisely 33% of the tuples.

However, increasing the parallelism further to `par = 4` and `par = 5` results in a significant performance regression, dropping the throughput back to 383,395 tuples/s and 358,424 tuples/s respectively. In these configurations, the hash distribution of the active entities once again heavily favors a single thread; the entire pipeline's ingestion rate is constrained by the processing speed of this single saturated join worker. This empirical behavior confirms that in stateful dataflows, hash-partitioning skew establishes the absolute ceiling for horizontal scalability.

Query 9: Complex Topologies and Skew Masking Query 9 represents one of the most structurally demanding operations in the benchmark, requiring a stateful `Interval Join` followed immediately by a `Keyed Tumbling Window` to extract the absolute maximum winning bid per auction. Due to the substantial memory required to buffer simultaneous unexpired auctions and their incoming bids, this topology was evaluated against the moderately scaled 30-million tuple dataset.

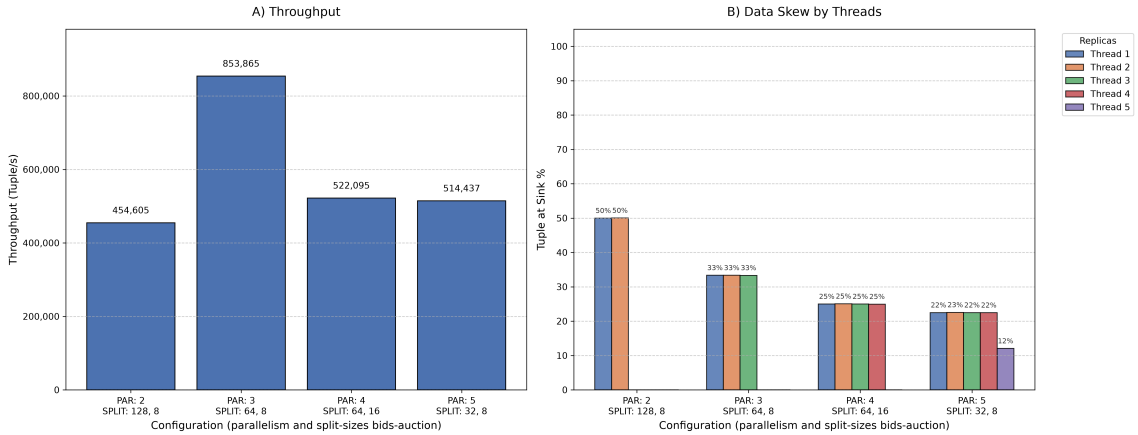


Figure 5.8: Performance profile for Query 9 on the 30M dataset. (A) Throughput peaking at 3 threads before experiencing degradation. (B) Data skew at the sink, exhibiting a deceptive output balance that masks internal computational bottlenecks.

As depicted in Figure 5.8(A), the pipeline achieves its maximum throughput of 853,865 tuples/s at `par = 3`. However, increasing the parallelism further triggers a sharp performance degradation, with throughput dropping to 522,095 tuples/s at `par = 4` and stagnating at 514,437 tuples/s at `par = 5`.

Analyzing the topological load distribution in Figure 5.8(B) introduces a fascinating architectural phenomenon: *Skew Masking*. The chart indicates that the final output tuples reaching the sink are distributed in a highly balanced manner across the worker threads.

While the sink distribution is uniform, the upstream execution is entirely unbalanced. Because the initial `Interval Join` routes events using `auction_id` as the partitioning key, "Hot Auctions" receiving disproportionate amounts of bids severely skew the load towards specific worker threads. This creates intense localized memory contention and processing delays within the C++ join buffers of the overloaded subtasks.

However, because the subsequent `Keyed Windows` operator compresses all those thousands of matched bids down to a single `MAX(price)` emission per auction, the massive volume disparity is completely neutralized before materialization. As illustrated in the Figure 5.9, the pipeline suffers from severe computational skew during

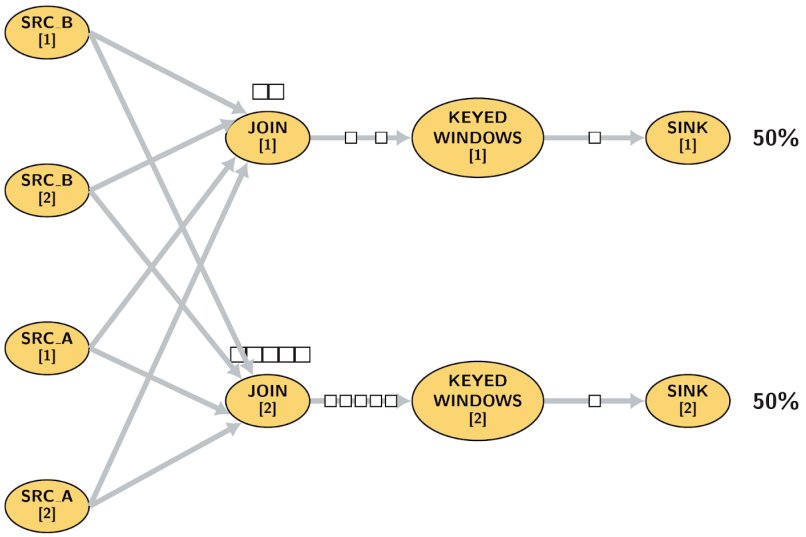


Figure 5.9: Internal view of the skew by the join that is nullified by the keyed windows operator.

the join phasedictating the absolute throughput drop observed at higher parallelism-seven though the final materialized results at the sink appear perfectly distributed.

Query 12: Keyed Windows and Balanced Hash Routing Query 12 evaluates a fully stateful continuous aggregation, counting the number of bids submitted by each user (`bidder`) within a processing window. Because this pipeline relies on a `KeyBy` operation without any upstream data reduction, it serves as a direct stress test for the engine's hash-routing mechanism on the 30-million tuple dataset.

As shown in Figure 5.10(A), the topology exhibits flawless linear scalability. The throughput starts at 750,881 tuples/s at `par = 2` and scales proportionally to reach a peak of 1,894,773 tuples/s at `par = 5`.

This optimal performance is entirely explained by the load distribution chart in Figure 5.10(B). Unlike the "Hot Auctions" that caused severe memory contention in Queries 8 and 9, the distribution of active `bidder` IDs in the dataset is vast and uniform. Consequently, the C++ hash-routing mechanism distributes the stateful workload perfectly across all available worker threads (achieving, for instance, an exact 20% load split per thread at `par = 5`). This perfectly balanced state allocation allows the parallel window operators to independently accumulate and emit results at maximum CPU speed, completely avoiding topological bottlenecks.

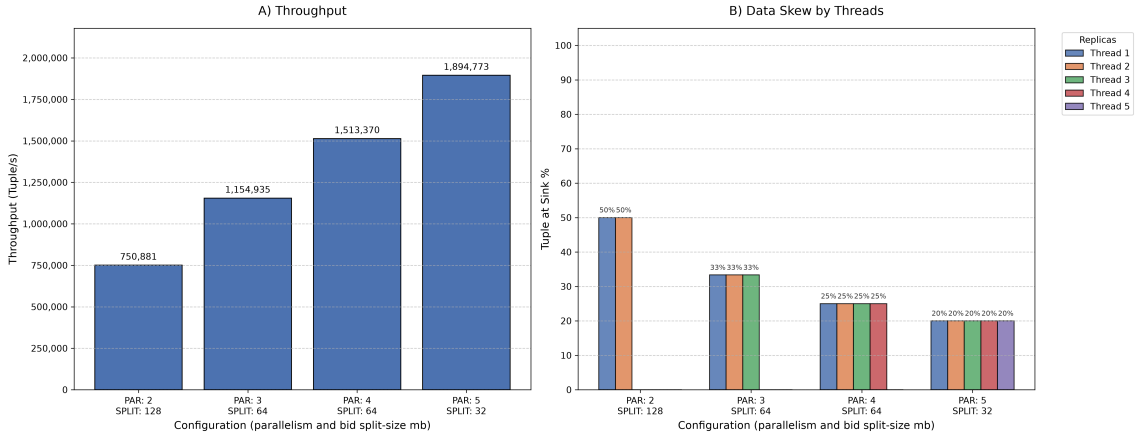


Figure 5.10: Performance profile for Query 12. (A) Perfect linear throughput scaling up to nearly 1.9M tuples/s. (B) Perfectly uniform data skew, demonstrating optimal hash distribution across the `bidder` keys.

Query 20: The Filter Join and Maximum State Retention Query 20 concludes the evaluation by executing a massive Interval Join between the Bid and Auction streams. To prevent immediate memory exhaustion on the full 70-million tuples dataset, the topology relies heavily on the *Filter Join* optimization pattern discussed in Section 5.2.2, pushing the highly selective `category == 10` predicate directly into the upstream extraction phase.

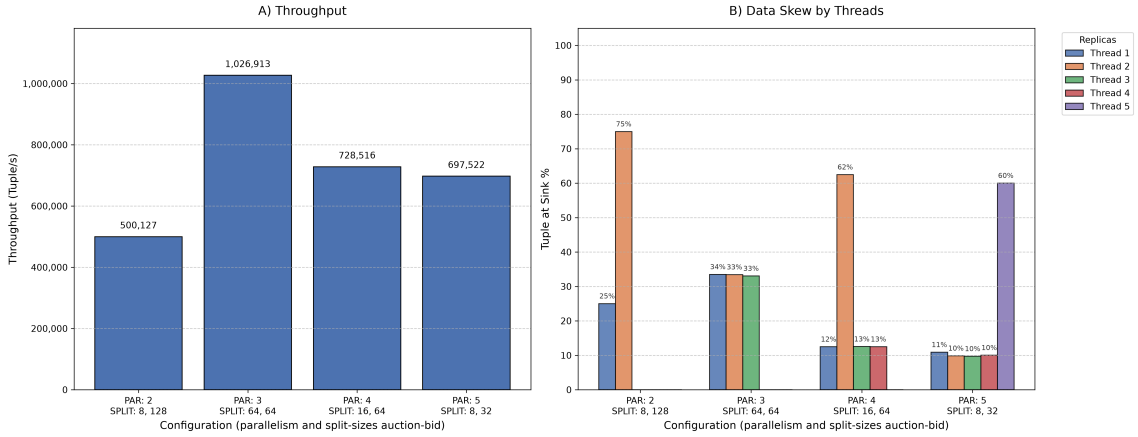


Figure 5.11: Performance profile for Query 20 under maximum dataset load (64.4M Bids × 4.2M Auctions). (A) Peak throughput achieved at `par = 3`, bounded by the sheer volume of memory state allocations. (B) Severe data skew in the C++ join buffers driven by clustered "Hot Auctions" matching the target category.

The throughput profile in Figure 5.11(A) reveals a distinct processing curve, reaching an absolute peak of 1,026,913 tuples/s at `par = 3`, before experiencing significant degradation at higher parallelism degrees (dropping below 700,000 tuples/s

at `par = 5`).

This performance ceiling is intrinsically tied to the immense physical state maintained by the C++ engine. Despite filtering out 95% of the auctions upstream, the join operator still retains thousands of active category-10 auctions across a massive asymmetrical envelope of [0m, +12h]. Consequently, the `Bid` stream forces the engine to buffer and index tens of millions of bids in RAM before the event-time watermarks finally surpass the 12-hour threshold required to trigger garbage collection. Empirical measurements during execution confirmed that the underlying worker threads allocated between 33 GB and 38 GB of physical RAM (depending on the parallelism configuration).

The degradation at higher parallelism levels is directly explained by the topological data skew shown in Figure 5.11(B). The distribution of bids targeting category-10 auctions is severely unbalanced, creating intense computational skew.

5.4.2 Native WindFlow Comparisons

The final evaluation isolates the computational overhead introduced by the TIDE's abstraction layer. To quantify the cost of translating a declarative Python AST into

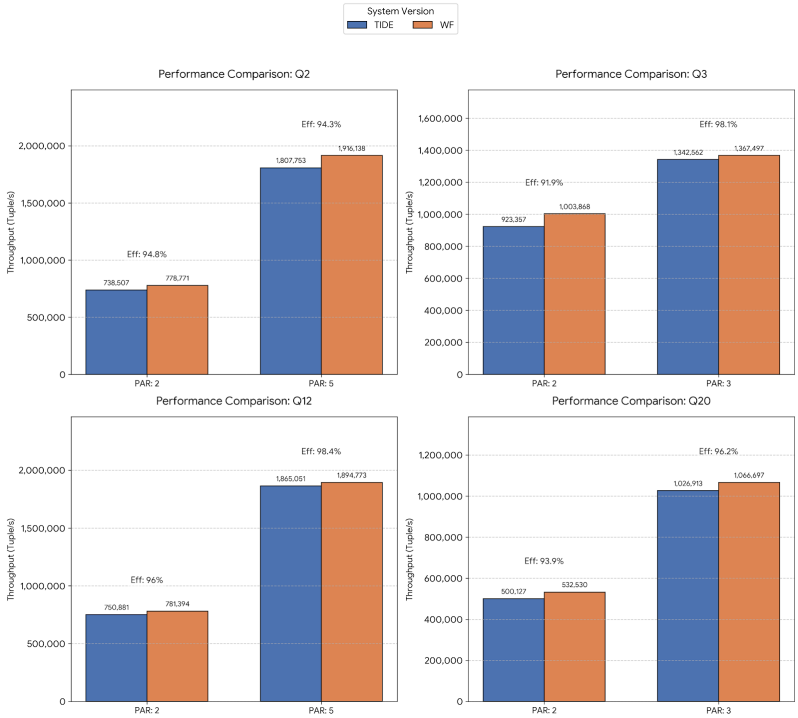


Figure 5.12: Performance comparison between TIDE auto-generated pipelines and native WindFlow C++ implementations. The efficiency metric highlights the near-zero overhead of the declarative abstraction.

executable C++, the auto-generated topologies were benchmarked against manually

optimized C++ pipelines authored directly using WindFlow's native operators. The comparison targets four representative queries (Q2 for stateless filtering, Q3 and Q20 for keyed interval joins, and Q12 for keyed windows), evaluated both at a baseline parallelism (`par = 2`) and at their respective peak scaling configurations.

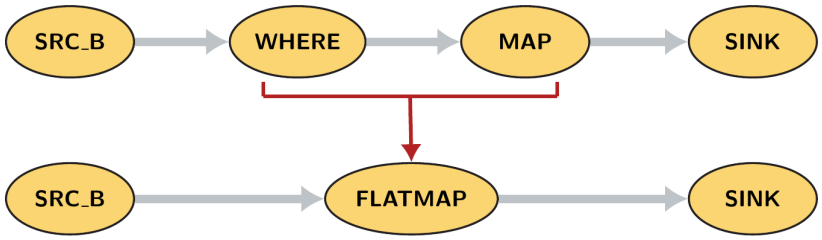


Figure 5.13: Simplified topologies for Q2 in TIDE (top) and native WindFlow (bottom). The filter and the map are fused.

As illustrated in Figure 5.12, the TIDE generated topologies maintain exceptional execution efficiency, consistently processing over 91% of the native throughput across all tested scenarios. At peak parallelism, the gap narrows even further, with Q3 reaching 98.1% efficiency and Q12 scaling up to 98.4%. These metrics demonstrate that the compilation mechanics described in Chapter 4 successfully lower the logical intents into highly performative inline C++ lambdas without introducing any bottlenecks.

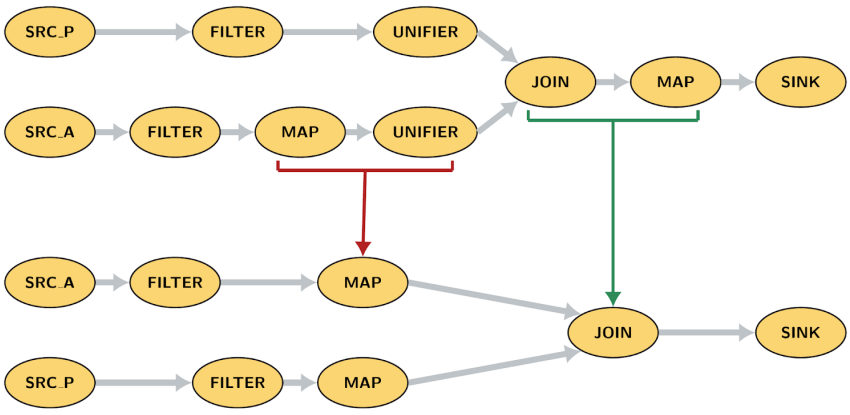


Figure 5.14: Simplified topologies for Q3 in TIDE (top) and native WindFlow (bottom). The select and unifier maps are fused and the join returns the already mapped struct.

The minimal performance gap (ranging from 1.6% to 8.1%) is entirely attributable to manual structural optimizations available to C++ programmers, specifically *Oper-*

ator Fusion. In native WindFlow development, a programmer can collapse adjacent logical steps to minimize intermediate struct allocations and lock-free queue interactions.

For instance, the native baseline for Query 2 utilizes a single custom `FlatMap` operator is used to evaluate the boolean filter and immediately apply the projection in-place (Figure 5.13).

Similarly, in the native implementation for Query 3, shown in Figure 5.14, the `join-unifier map` is fused with the `select` and the `IntervalJoin` operator outputs the right struct without the need for an additional map.

Despite lacking these handcrafted topological fusions, the framework absorbs massive data ingestion with a negligible abstraction penalty, conclusively validating the compiler's design for high-performance stream processing.

Conclusions

Summary of Contributions

The growing reliance on real-time analytics has cemented Data Stream Processing as a cornerstone of modern software architectures. However, developers operating in ultra-low latency domains face a difficult trade-off: they must choose between the high-level, declarative productivity of managed frameworks (e.g., Apache Flink on the JVM) and the raw execution speed provided by native C++ libraries like WindFlow. While WindFlow excels in exploiting shared-memory multi-core architectures, its reliance on low-level imperative programming, manual state management, and the lack of a dynamic type system drastically limits developer ergonomics and increases the likelihood of application errors.

This thesis presented the design, implementation, and empirical validation of TIDE (*Table Interface for Declarative Execution*), a declarative relational API for the WindFlow library. By adopting a decoupled three-tier architecturecomprising a Python frontend (`api`), a translation engine (`codegen`), and a C++ backend (`runtime`)TIDE successfully abstracts the complexities of continuous queries. The framework allows users to express complex topological intent using an SQL-like Domain Specific Language (DSL). The Python frontend enforces fail-fast static type checking and logical validation before serializing the pipeline into a JSON-based Abstract Syntax Tree (AST). The C++ compiler then dynamically synthesizes this AST into heavily optimized, deduplicated native code (via Jinja2 templating), seamlessly lowering the logical query into a physical WindFlow execution graph.

The empirical evaluation, conducted using the industry-standard NEXMark benchmark, validated both the robustness and the performance of the proposed solution. The results demonstrated that TIDE's declarative abstraction introduces negligible runtime overhead, maintaining over 91% (and peaking at over 98%) of the throughput of manually authored native C++ baselines, while effortlessly scaling to process millions of tuples per second across multiple parallel worker threads.

Future Works

While TIDE successfully establishes a robust foundation for declarative stream processing in WindFlow, several avenues for future development remain to expand

its expressive power and optimization capabilities.

Extended Relational Constructs and Connectors

The current DSL focuses on core relational algebra (filtering, projection, joining, and windowing). Future iterations should expand the grammar to support arbitrary scalar functions (e.g., advanced string manipulations and conditional `CASE WHEN` branching) and complex nested aggregations (e.g., `COUNT(DISTINCT)` and `FILTER`). Additionally, extending the source and sink builders to integrate seamlessly with external distributed message brokers, such as Apache Kafka, would significantly enhance the framework's applicability in production environments.

Query Caching and Branch Splitting

Currently, if a user needs to evaluate multiple distinct queries over the same initial dataset or intermediate result, the TIDE `Draft` abstraction dictates the instantiation of separate, parallel branches from the source. This implies redundant physical ingestion or duplicated intermediate computations. Implementing a true *Query Caching* mechanism would allow the compiler to recognize shared logical prefixes in the AST. Instead of duplicating the operators, the backend could synthesize topological splits (e.g., utilizing WindFlow's branching capabilities or custom routing functors) to compute the shared result once and broadcast it to multiple downstream branches, minimizing both I/O contention and redundant memory allocation.

Changelog Semantics and Retractions

Perhaps the most significant architectural limitation of the current implementation is its reliance on a pure *append-only* streaming paradigm. As observed during the benchmarking of successive aggregations (e.g., chaining maximum and average operations), the lack of retraction messages (`-U` / `+U`) prevents the accurate computation of dynamic, unbounded global states. Incorporating a true *Changelog Stream* semanticwhere operators can emit explicit updates or deletions to invalidate previously emitted recordswould unlock the capability to express complex analytical patterns natively. This includes supporting "Keep Last" semantics, Top-N dynamic rankings, and mathematically sound chained aggregations without relying strictly on rigid windowed boundaries.

Bibliography

[1] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[2] Marco Aldinucci, Marco Danelutto, Peter Kilpatrick, and Massimo Torquati. Fastflow: High-level and efficient streaming on multi-core. In *Programming Multi-Core and Many-Core Computing Systems*, pages 261–280. John Wiley & Sons, 2014.

[3] Henrique C. M. Andrade, Buğra Gedik, and Deepak S. Turaga. *Fundamentals of Stream Processing: Application Design, Systems, and Analytics*. Cambridge University Press, 2014.

[4] Apache Software Foundation. Apache Flink documentation. `https://nightlies.apache.org/flink/flink-docs-release-2.3/`.

[5] Apache Software Foundation. Apache Flink documentation: Windows - allowed lateness. `https://nightlies.apache.org/flink/flink-docs-release-2.3/docs/dev/datastream/operators/windows/#allowed-lateness`, 2026. Accessed: 2026-09-26.

[6] Arvind Arasu, Shivnath Babu, and Jennifer Widom. The CQL continuous query language: Semantic foundations and query execution. *The VLDB Journal*, 15(2):121–142, 2006.

[7] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*, pages 221–230. ACM, 2018.

[8] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 36(4):28–38, 2015.

[9] Duccio Lorenzoni (LorDigno). TIDE repository. `https://github.com/LorDigno/WindFlow-TIDE`.

[10] Fabian Hueske and Vasiliki Kalavri. *Stream Processing with Apache Flink: Fundamentals, Implementation, and Operation of Streaming Applications*. O'Reilly Media, 2019.

[11] Jin Li, David Maier, Kristin Tufte, Vassilis Papadimos, and Peter A. Tucker. No pane, no gain: efficient evaluation of sliding-window aggregates over data streams. *SIGMOD Record*, 34(1):39–44, mar 2005.

[12] Gabriele Mencagli, Massimo Torquati, A. Cardaci, A. Fais, L. Rinaldi, and Marco Danelutto. Windflow: High-speed continuous stream processing with parallel building blocks. *IEEE Transactions on Parallel and Distributed Systems*, 32(11):2748–2763, Nov 2021.

[13] Nexmark Contributors. NEXMark benchmark repository. `https://github.com/nexmark/nexmark`. Accessed: 2026-10-03.

[14] Pallets Projects. Jinja2 documentation. `https://jinja.palletsprojects.com/`, 2024. Accessed: 2026-09-25.

[15] Peter A. Tucker, David Maier, Tim Sheard, and Leonidas Fegaras. Exploiting punctuation in data stream query processing. In *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data (SIGMOD '03)*, pages 253–264, 2003.

[16] Peter A. Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. NEXMark – a benchmark for querying data streams. Technical Report CSE-02-012, OGI School of Science and Engineering at OHSU, 2002.

Ringraziamenti

Non ironicamente questa credo che sia la parte più difficile da scrivere, ci sono cose che vorrei dire e che dirò, cose che non dovrei dire ma che dirò comunque e cose che vorrei tanto riuscire a dire.
Per questo, miei cari venticinque lettori, vi dovrete accountentare della mia solita retorica di verità nascoste sotto un velo di sarcasmo, neanche troppo sarcastico, durante la scrittura col cuore in mano.

Ringrazio il mio retalore, il professore Gabriele Mencagli che nelle ultime settimane di studio "matto e disperatissimo" non mi ha detto di no ed è stato al passo permettendomi di essere qua oggi.

Alla miei genitori, che sia al di là della città o dopo qualche fermata ferroviaria, credo che ci siamo ritrovati sotto molteplici punti di vista negli ultimi anni. Ci sono stati dei momenti bassi ma c'è sempre stato qualcuno su cui contare e voglio che sappiate che anche io ci sarò per voi. Magari non per un mese a Careggi però, con quel posto "*ho smesso*".

A Fré che non so se mai leggerà queste righe. Grazie che mi hai aperto gli occhi su cosa potesse essere la mia vita, da quel cuginetto del ca**o che ti stava appiccicato in campeggio.

Ad Alessandro l'unico vero amico che ho avuto per la maggior parte della mia vita. A Luigi che sono contento che tu ti sia più "*garoso*" di quanto lo fossi io alla tua età. Grazie che senza di voi la vita montelupina sarabbe stata priva anche di quei pochi momenti di svago passati insieme.

Ad Andrea e a Pippo che sin da(i) fondamenti ci sono sempre stati nonostante negli anni si siano persi pezzi. Grazie per avere reso sopportabile quel periodo disastroso della mia vita che è stato il primo anno. La prossima volta però a *raccatta'* l'uva ci vai te Andre con Pippo e Jacopo.

Ad Alessandra che mi ha spinto ad andare avanti, a metterci impegno, che mi ha mostrato che anche io posso ho il diritto ad ambire al meglio, che *me lo merito*.

Grazie, senza di te certi periodi sarebbero stati infernali e non ce l'avrei mai fatta ad arrivare qua oggi in questo stato. Me la sono passata brutta in quetsi anni ma mi hai aiutato a ripartire da zero, per questo non voglio più sentirti dire che *non ti meriti* le cose, sei capace e speciale perchè nessuno ci era risucito in vent'anni ad aiutarmi come hai fatto te. Non ci sarà bisogno di dire che quando ci sarà bisogno anche io ci sarò.

A tutti quelli che non sono riuscito a trovare la forza o lo spazio per ringraziare personalmente. Tutti mi avete aiutato in questo viaggio che è la vita, la vostra presenza è la cosa più preziosa quando la solitudine è ciò di cui più ho paura al mondo.

Grazie!

P.S. Se alla dicussione per qualsivoglia motivo prendo un gesso e uso la lavagna durante la discussione Jacopo mi deve un Gallaccio. Grazie anche a te Pino per saper ascoltare, però magari la prossima volta non farmi aspettare mezzora al Duomo ok? *Non me lo merito.*